

Networking

K19G

2026-09-04

Table of contents

Networking	27
Curriculum map	27
Journey map	28
What “done” looks like	28
Lab ground rules	29
Related books	29
Formats	29
 Lab Platform	 30
Lab Host — macOS	31
 Lab Host — macOS	 32
Goals	32
Prerequisites	32
Virtualization and container runtime	32
Options	32
Checklist	33
Install Containerlab	33
Networking notes on macOS	33
First smoke test	33
Next	33
 Lab Host — Linux	 34
 Lab Host — Linux	 35
Goals	35
Prerequisites	35
Container runtime	35
Typical path (Debian/Ubuntu-style)	35
Kernel / modules	36
Install Containerlab	36
Host networking and firewall	36
First smoke test	36
Next	36
 Containerlab Fundamentals	 37
 Containerlab Fundamentals	 38
Learning goals	38

Prerequisites	38
How Containerlab fits together	39
Minimal topology: two Linux nodes	40
Triangle of FRR routers (book default pattern)	40
Startup config sketch (<code>config/r1/daemons + frr.conf</code>)	41
Lifecycle	42
Topology YAML anatomy	42
Kinds you will actually use	42
Links and endpoints	42
Multipoint / LAN segments	43
Management network	43
Inspect, exec, save	43
Lab directory layout (recommended)	44
Graph and documentation	44
Partial failure recovery	44
Predict → observe → fix lab	45
Setup	45
Predict	45
Observe	45
Fix inject	46
Harden	46
Common mistakes	46
Version awareness (mid-2026)	46
Security and shared hosts	47
Summary	47
Free Image Matrix & Resources	48
Free Image Matrix & Resources	49
Learning goals	49
Principles	49
Role matrix	49
What we deliberately de-emphasize	50
FRR as the workhorse	50
Linux endpoints	51
SR Linux community (optional track)	51
SONiC and VyOS (optional)	52
Sizing the lab host	52
RAM savers	52
Disk	53
Version pinning strategy	53
Resource planning worksheet	53
Network resource limits on the host	54
Documentation & community map	54
RFCs worth bookmarking (foundations)	54
Predict → observe: image sanity lab	55
Multi-implementation mindset	55
Ethics and redistribution	55

Summary	56
Linux Networking for Lab Hosts	57
Linux Networking for Lab Hosts	58
Learning goals	58
Learning goals in practice	58
Core objects	58
Network namespaces	59
veth pairs: virtual cables	59
Bridges: software switches	60
Routing between namespaces	61
Failure drill: disable forwarding	62
Failure drill: missing return route	62
VLAN subinterfaces (preview of trunks)	62
Policy leftovers that break labs	62
rp_filter (reverse path filtering)	62
Forwarding and firewall	63
Disable offloads when captures look weird (optional)	63
Mapping to Containerlab	63
Clean teardown habits	64
Example <code>down.sh</code>	64
Predict → observe → fix drill (checklist)	64
Useful one-liners cheatsheet	65
Security note for shared hosts	65
Summary	65
Lab Hygiene	66
Lab Hygiene	67
Learning goals	67
Why hygiene is a skill	67
Addressing plans	67
Template	68
Allocation strategy (personal lab supernets)	68
/30 vs /31 vs /24 on links	68
Naming conventions	69
Lab name	69
Node names	69
Interface names	69
Git branches / folders	69
Directory layout (canonical)	70
README skeleton	70
Git habits	71
Never commit	71
Diagram habit	71
Configuration hygiene	72
FRR example: comment intent	72

verify.sh hygiene	72
Cross-lab contamination	73
Time hygiene	73
Definition of done (lab)	73
Predict → observe → fix: hygiene drill	74
Collaboration norms (optional)	74
Summary	75
Networking Foundations	76
What Is Networking?	77
What Is Networking?	78
Learning goals	78
Building blocks	78
What problem networking solves	78
Switching vs routing (first cut)	79
Protocols as contracts	79
Why labs beat slideware	79
Observation toolkit (preview)	80
Checkpoint	80
Next	80
TCP/IP and the Layered Stack	81
TCP/IP and the Layered Stack	82
Learning goals	82
TCP/IP model (working map)	82
OSI as a legend (not a religion)	82
What each layer “owns”	83
Link	83
Internet (IP)	83
Transport	83
Application	83
Demultiplexing (up the stack)	84
Layered troubleshooting	84
Encapsulation (preview)	84
Checkpoint	84
Next	84
Packets, Frames, Encapsulation	85
Packets, Frames, Encapsulation	86
Learning goals	86
Vocabulary	86
Encapsulation path (send)	86
Decapsulation path (receive)	87
Maximum sizes and MTU	87

Captures: reading the onion	87
Tunnels and overlays (preview)	87
Lab sketch (two nodes)	88
Checkpoint	88
Next	88
Link Layer and Ethernet	89
Link Layer and Ethernet	90
Learning goals	90
Why link layer matters	90
Ethernet frame (simplified)	90
MAC addresses	91
Broadcast domain vs collision domain	91
Switch vs router (again)	91
MAC learning (preview)	91
Virtual Ethernet in labs	92
Common L2 failures (foundations list)	92
Checkpoint	92
Next	93
IPv4 Addressing and Subnetting	94
IPv4 Addressing and Subnetting	95
Learning goals	95
IPv4 structure	95
CIDR and masks	96
On-link decision	96
Private and documentation ranges	96
Subnetting for multi-tier labs	96
Point-to-point links	97
Summarization (intro)	97
VLSM	97
Host configuration checklist	99
Worked subnet: /26 from a /24	99
Private vs public (RFC 1918)	99
Paper drills	100
Checkpoint	100
Next	100
IPv6 Essentials	101
IPv6 Essentials	102
Learning goals	102
Why IPv6 in this book	102
Address format	102
Address kinds (must know)	103
Interface identifiers and /64	103
Linux static example	103

Neighbor Discovery (preview)	103
Dual-stack thinking	104
Checkpoint	104
Next	104
ARP, ND, and Local Resolution	105
ARP, ND, and Local Resolution	106
Learning goals	106
When local resolution happens	106
ARP (IPv4)	106
Incomplete / failed ARP	107
Neighbor Discovery (IPv6)	107
Gratuitous ARP and updates	107
Proxy ARP (awareness)	107
Lab sketch	107
Checkpoint	108
Next	108
ICMP and Path Diagnostics	109
ICMP and Path Diagnostics	110
Learning goals	110
ICMPv4 types (operator set)	110
ICMPv6	110
Interpreting ping	111
Traceroute mechanics (conceptual)	111
Path MTU Discovery (preview)	111
Filters and policy	111
Lab sketch	112
Checkpoint	112
Next	112
Transport: TCP and UDP	113
Transport: TCP and UDP	114
Learning goals	114
Ports and sockets	114
UDP	114
TCP	115
Three-way handshake (must know)	115
States (operator view)	115
TCP vs UDP selection	115
NAT and transports (preview)	116
Security basics	116
TCP state machine (operator view)	116
Three-way handshake and teardown (why captures look busy)	117
Lab sketch	117
Checkpoint	117

Next	117
Common Application Protocols	118
Common Application Protocols	119
Learning goals	119
Map (operator cheat sheet)	119
DNS	119
Roles	119
Query types (common)	120
Failure patterns	120
DHCP (IPv4)	120
HTTP and HTTPS	121
SSH	121
TLS in one paragraph	121
NTP and time	121
Lab sketch	122
Checkpoint	122
Next	122
Host Forwarding and Routing Intro	123
Host Forwarding and Routing Intro	124
Learning goals	124
Connected vs remote	124
Linux route table essentials	124
Longest prefix match	125
Default gateway	125
Static routes (host or simple router)	125
IP forwarding (Linux as router)	125
Policy routing (awareness)	126
Firewall vs routing	126
Lab sketch	126
Checkpoint	126
Next	126
Control, Data, and Management Planes	127
Control, Data, and Management Planes	128
Learning goals	128
Data plane	128
Control plane	128
Management plane	129
Interaction	129
Troubleshooting order (habit)	130
Lab habit	130
Checkpoint	130
Next	130

Layer 2	131
Ethernet & MAC Learning	132
Ethernet & MAC Learning	133
Learning goals	133
Concepts	134
Frame essentials	134
Learning and flooding	134
MAC addresses	134
ARP: glue for IPv4 on Ethernet	135
Linux bridge FDB	136
Lab A: two hosts, one bridge (hand or Containerlab)	136
Containerlab topology	136
Predict	137
Observe	137
Fix inject	137
Harden	138
Lab B: three hosts — flooding visibility	138
MAC moves and flaps	138
Common L2 failures (pre-VLAN)	138
Promiscuous mode and labs	138
Security digression (short)	139
Predict worksheet (paper)	139
FRR / NOS note	139
Summary	139
VLANs & Trunks	141
VLANs & Trunks	142
Learning goals	142
Concepts	143
Broadcast domains	143
802.1Q tag	143
Router roles	143
Addressing plan example	143
Lab topology (conceptual)	144
Containerlab sketch using Linux VLAN-aware bridge	144
config/sw1.sh (VLAN filtering bridge)	145
Predict → observe → fix	146
Predict	146
Observe	146
Failure drill: native / PVID mismatch	146
Failure drill: missing allowed VLAN	147
Router-on-a-stick preview	147
Verification checklist	147
verify.sh sketch	147
Design tips	148

Common mistakes	148
Summary	148
Loop Prevention	149
Loop Prevention	150
Learning goals	150
Why loops kill L2	150
Spanning tree behavior	150
Priority and ties	151
Port states (classic story)	151
Design alternatives (model maturity)	151
Lab: three switches, intentional loop	152
Topology YAML (Linux bridges)	152
Dangerous mode: dumb bridges without STP	153
Observe storm (careful)	153
Safer mode: enable STP on the bridge	153
Root placement drill	154
Failure drill: link down on active tree edge	154
BPDUs and protection concepts (awareness)	155
When not to extend L2	155
Predict worksheet	155
Verification checklist for STP labs	155
Summary	156
Link Aggregation	157
Link Aggregation	158
Learning goals	158
Concepts	158
One logical link	158
Static vs LACP	158
Hashing	159
Split brain / misconfig	159
Linux bonding quickstart	159
Containerlab topology (two paths, bond on routers)	160
config/r1-bond.sh	161
Predict → observe → fix	161
Predict	161
Observe	162
Failover drill	162
Mismatch drill	162
Hashing awareness drill	162
STP interaction	162
Operational checks (NOS-neutral)	163
Common mistakes	163
Summary	163
L2 Failure Drills	164

L2 Failure Drills	165
Learning goals	165
Baseline lab	165
Evidence kit	166
Drill catalog	166
Drill 1 — Access port wrong VLAN	166
Drill 2 — Trunk missing VLAN	166
Drill 3 — Native VLAN mismatch	167
Drill 4 — Cable / interface down	167
Drill 5 — Broadcast storm (controlled)	167
Drill 6 — STP blocked port confusion	168
Drill 7 — MAC move	168
Drill 8 — Duplicate IP	168
Drill 9 — Duplicate MAC (advanced)	169
Drill 10 — LAG member down	169
Drill 11 — Inter-VLAN path mistaken for L2	169
Drill 12 — Host firewall	170
Incident note template	170
Scorecard	170
Combined verify.sh ideas	171
Checkpoint: L2 competence	171
Summary	171
 Layer 3	 173
IP Addressing	174
IP Addressing	175
Learning goals	175
IPv4 structure	175
On-link decision	176
Subnetting for labs	176
Summarization	176
Paper drill	177
Special-use ranges (know them)	177
IPv6 essentials for dual-stack labs	177
Dual-stack habit	178
Linux configuration patterns	178
Persistent lab style	178
Multiple addresses	178
Lab: addressing plan + dual-stack hosts	178
Predict	179
Observe	179
Failure: overlapping subnets	180
Failure: bad summary (static preview)	180
Calculation practice (quick)	180
Host vs router addressing habits	180

Verification checklist	180
Summary	181
Routing Fundamentals	182
Routing Fundamentals	183
Learning goals	183
Concepts	183
Forwarding equation	183
RIB vs FIB	184
Longest prefix match	184
Metrics and distance (ideas)	184
Connected routes	184
Host vs router	185
Gateway ARP	185
Next-hop resolution failures	185
Asymmetric routing	185
Lab: competing prefixes	186
Predict / observe	187
Lab: router with two interfaces	187
ARP/ND deep enough for routing	188
TTL and traceroute	188
Policy preview	188
Predict → observe → fix drills	189
Drill: missing return route	189
Drill: wrong gateway MAC path	189
Drill: more-specific shadow	189
Cheatsheet	189
Summary	189
Static Routing Labs	191
Static Routing Labs	192
Learning goals	192
Topology: triangle with hosts	192
Containerlab topology	194
FRR config sketches	195
Linux-only alternative	195
Deploy and baseline	196
Predict	196
Observe	196
Floating statics	196
Failover drill	196
Return path drill (mandatory)	197
Blackhole and reject	197
verify.sh	197
Addressing.md excerpt	197
Default routes at edge	198

IPv6 statics (optional parallel)	198
Common mistakes	198
Summary	198
ICMP & Path MTU	200
ICMP & Path MTU	201
Learning goals	201
ICMP essentials (IPv4)	201
Path MTU discovery (idea)	202
Lab: MTU mismatch	202
Topology	202
Predict	203
Observe	203
Inject: drop ICMP unreachable on r1	204
Restore	204
Harden	204
Traceroute and ICMP filters	204
Capture workflow	204
iperf3 large transfers	205
IPv6 notes	205
Operational checklist	205
Predict worksheet	205
verify.sh additions	206
Summary	206
Interior Routing	207
OSPF Basics	208
OSPF Basics	209
Learning goals	209
Concepts	209
Distance-vector vs link-state	209
OSPF adjacency state machine (short)	209
Areas	210
Cost	210
Network types (awareness)	210
Addressing plan	210
Topology YAML	210
FRR daemons	211
FRR OSPF config (r1 example)	211
Deploy and verify	212
Predict	212
Neighbor not Full?	213
Cost change drill	213
Link failure drill	213
Passive interfaces / LAN notes	213

Verification checklist	214
verify.sh	214
Common mistakes	214
Beyond this chapter	215
Summary	215
BGP Basics	216
BGP Basics	217
Learning goals	217
Concepts	217
Autonomous systems	217
eBGP vs iBGP	218
Session state	218
Path selection overview	218
NLRI and attributes	218
Lab A: two AS eBGP	218
Topology	219
daemons	219
r1 frr.conf	220
r2	220
Verify session	220
Predict	221
Lab B: iBGP sketch (optional same day)	221
Session failure drill	221
Next-hop reachability drill	221
Dual-home teaser	222
verify.sh	222
Safety habits	222
Common mistakes	222
Summary	223
Policy & Filters	224
Policy & Filters	225
Learning goals	225
Why policy exists	225
Prefix lists (FRR)	226
Route-maps as idea	226
Lab: filter unauthorized prefixes	226
Customer r1 advertises extra junk	227
ISP r2 policy	227
Predict	227
Observe	227
Harden	227
Local-pref steer drill	228
Outbound preference (customer)	228

Redistribution loop awareness	228
Example static→OSPF with filter (sketch)	229
ACL-style packet filters vs routing policy	229
Distribute-list / filter-list ideas	229
Failure drills	229
Drill 1 — over-broad permit any	229
Drill 2 — accidental deny all	230
Drill 3 — next-hop-self missing (iBGP)	230
verify.sh policy checks	230
Policy design worksheet	230
Summary	231
Ops and Automation	232
Observability for Networks	233
Observability for Networks	234
Learning goals	234
Observability layers	234
Baseline: what “healthy” looks like	234
Interface and error signals	235
Control-plane signals	235
OSPF	235
BGP	236
Path quality	236
Packet capture practice	236
Capture points	236
Logging	237
FRR	237
Kernel	237
Metrics sketch (Prometheus-shaped thinking)	237
Minimal blackbox idea	237
Telemetry awareness (2026)	238
Incident timeline template	238
Lab drill: observe a BGP flap	238
Lab drill: silent MTU blackhole	239
Runbook snippet library	239
Privacy and safety	239
Summary	239
Automation & Lab-as-Code	240
Automation & Lab-as-Code	241
Learning goals	241
Principles	241
Directory layout (automation-ready)	241
Makefile pattern	242
up.sh / down.sh	242

Config strategies	243
Safe interactive explore → promote	243
Light Ansible (optional)	243
Templating addressing	244
verify.sh mature pattern	244
CI smoke (GitHub Actions sketch)	245
Image pins in automation	245
Idempotency drills	245
Secret hygiene	246
Graph and inventory export	246
Predict → observe → fix	246
Anti-patterns	246
Summary	247
Capstone Outline	248
Capstone Outline	249
Learning goals	249
Definition of done (all capstones)	249
Capstone C1 — Campus-style dual core (open images)	250
Intent	250
Skills exercised	250
Success metrics	250
Suggested stack	250
Capstone C2 — Dual-homed site to provider edge	251
Intent	251
Skills exercised	251
Success metrics	251
verify extras	251
Capstone C3 — Mini fabric snack	251
Intent	251
Skills exercised	252
Stretch (optional)	252
Capstone C4 — Incident week	252
Intent	252
Skills exercised	253
Success metrics	253
How to pick	253
Project template (copy)	253
DESIGN.md skeleton	254
RETRO.md prompts	254
Integration with the book spine	254
Beyond this book (next horizons)	254
Final checkpoint: “expert” as defined in the syllabus	255
Summary	255

Edge and Services	256
First-Hop Redundancy	257
First-Hop Redundancy	258
Learning goals	258
Concepts	258
The problem	258
First-hop redundancy idea	258
Protocol family (vendor-agnostic)	259
Priority, preempt, timers	259
What FHRP is <i>not</i>	259
ARP and the VIP	260
Addressing plan	260
Topology YAML	260
FRR daemons	261
FRR VRRP sketch (r1)	262
Linux alternative (keepalived)	262
Deploy and baseline	263
Predict (baseline)	263
Observe	263
Drill 1 — Master failure	263
Predict → observe → fix	263
Drill 2 — Split brain / LAN partition (concept)	264
Predict	264
Observe	264
Fix / harden	264
Drill 3 — Priority change live	265
Predict	265
Observe	265
Return path and asymmetric edges	265
Verification checklist	265
verify.sh sketch	266
Common mistakes	266
Optional SR Linux note (community)	266
Summary	266
DHCP Relay and NAT	268
DHCP Relay and NAT	269
Learning goals	269
Concepts — DHCP	269
Why not static everything?	269
Four-way dance (v4)	269
Critical fields	270
Relay failure modes	270
Concepts — NAT family	270
When NAT is appropriate in this book	270

Addressing plan (combined edge lab)	271
Topology YAML	271
Edge addressing and forward	272
DHCP server (dnsmasq) on dhcpsrv	273
Minimal same-subnet lab (sanity first)	273
Relay path (concept + command)	273
Predict (DHCP)	273
Observe	274
Fix	274
NAT — MASQUERADE on edge	274
Predict	274
Observe	275
Fix	275
Drill — break DHCP then NAT	275
FRR on the edge (optional co-location)	275
Verification checklist	276
Common mistakes	276
Summary	276

Name, Time, and Log Hygiene 277

Name, Time, and Log Hygiene 278

Learning goals	278
Why this chapter exists	278
Concepts — naming	279
Layers of DNS use	279
Records you actually need	279
Split horizon awareness	279
Concepts — time	279
Concepts — logs	280
Addressing plan (hygiene lab)	280
Topology YAML	280
Bring-up addressing (example)	281
DNS lab — dnsmasq authoritative snack	282
Predict	282
Observe / fix	282
Drill — DNS outage looks like network outage	282
Predict → observe → fix	282
Time sync — chrony sketch	283
Skew drill	283
Predict	283
Observe	283
Logging hygiene	284
Local journal pattern	284
syslog-style forward (concept)	284
What to log for network incidents	284
FRR logging (when routers are FRR)	284
Hygiene checklist (use every serious lab)	285

Predict → observe → fix mini-scenarios	285
Scenario A — “Git clone fails” in a lab VM	285
Scenario B — “Certificate expired” on brand-new lab CA	285
Scenario C — “It failed an hour ago” with no trace	285
Optional: name the lab graph	286
Common mistakes	286
Summary	286
Edge Dual-Home Lab	287
Edge Dual-Home Lab	288
Learning goals	288
Design choices (pick one primary)	288
Pattern A — Single CE, two eBGP uplinks (simpler)	288
Pattern B — Dual CE + VRRP (more realistic campus edge)	288
Addressing plan	289
Topology YAML	289
FRR daemons	290
CE1 config sketch	291
CE2 config differences	292
PE1 / PE2 sketch	292
Deploy and baseline verify	293
Predict (baseline)	293
Drill 1 — Primary PE session loss	294
Predict → observe → fix	294
Drill 2 — Force VRRP failover	294
Predict	294
Drill 3 — Policy rejection	295
Predict	295
Drill 4 — Name/time smoke (hygiene)	295
verify.sh	295
Definition of done (this lab)	295
Optional SR Linux PE	296
Common mistakes	296
Summary	296
Policy, QoS, and Hardening	297
ACL Thinking	298
ACL Thinking	299
Learning goals	299
Concepts	299
Three different “filters” people mix up	299
Policy statement template	299
First-match vs best-match	300
Stateful vs stateless	300
Direction and attachment	300

Written policy for the lab	300
Topology YAML	301
Addressing	302
Implement with iptables (readable teaching form)	302
nftables sketch (same policy)	303
Predict → observe → fix drills	304
Drill 1 — Allowed path	304
Drill 2 — Denied path	304
Drill 3 — SSH source restriction	304
Drill 4 — Asymmetric paths	304
Drill 5 — Wrong plane	305
FRR: route policy is still “ACL thinking”	305
Object-group idea (vendor-agnostic)	305
Change control for ACLs	305
Verification checklist	306
Common mistakes	306
Summary	306
Control-Plane Protection	307
Control-Plane Protection	308
Learning goals	308
Concepts	308
Two paths through a router	308
Why unprotected CPUs melt	309
Policy matrix (start simple)	309
Hardening adjacent controls	309
Lab topology	310
Addressing	311
Baseline routing (static snack)	311
Linux receive-path protection (teaching CoPP)	311
Predict → observe → fix	312
Drill 1 — Transit still works under “attack” to RE	312
Drill 2 — SSH from non-mgmt	313
Drill 3 — Over-tight BGP protection	313
Drill 4 — Peer-only OSPF	313
FRR-side levers (awareness)	313
Building a CoPP document	313
IPv6 notes	314
Optional SR Linux / NOS CoPP	314
Verification checklist	314
Common mistakes	314
Summary	315
QoS Vocabulary	316
QoS Vocabulary	317
Learning goals	317

Concepts — the pipeline	317
Vocabulary table	318
DSCP / PHB (operator view)	318
Policing vs shaping	319
Linux lab topology	319
Addressing	319
Experiment 1 — bottleneck without QoS	320
Predict	320
Observe	320
Experiment 2 — two classes with HTB	320
Predict	321
Observe	321
Fix	321
Experiment 3 — police vs shape feel	321
Marking sketch (DSCP)	321
Predict	322
Observe	322
What lab QoS cannot prove	322
FRR / NOS note	322
Design mini-checklist	322
Predict → observe → fix template	323
Common mistakes	323
Summary	323
Hardening Checklists	324
Hardening Checklists	325
Learning goals	325
How to use these lists	325
Checklist 0 — Lab hygiene (every topology)	325
Checklist 1 — Management plane	326
Linux probes	326
Checklist 2 — Control plane & routing	326
FRR probes	327
Checklist 3 — Data plane filters	327
Checklist 4 — Edge services	327
Checklist 5 — Secrets, users, supply chain	328
Checklist 6 — Resilience & abuse	328
Role profiles (quick)	328
Lab host (Containerlab hypervisor)	328
FRR edge CE	328
FRR P / fabric leaf	329
Linux bridge “switch”	329
Exception register template	329
Automating gates — <code>verify-hard.sh</code>	329
Predict → observe → fix (meta-drill)	330
Mapping to prior chapters	330
Optional SR Linux	330

Definition of done — hardened lab	331
Common mistakes	331
Summary	331
Overlays and Tunnels	332
Why Encapsulate	333
Why Encapsulate	334
Learning goals	334
The core picture	334
Why people encapsulate	335
Why <i>not</i> to encapsulate	335
Encapsulation as Russian dolls	336
Identifiers you must name	336
Failure domains	337
MTU tax	337
ECMP and entropy	338
Control plane vs data plane (again)	338
Mini lab — see encapsulation with tcpdump	338
GRE between a and b across under	339
Predict → observe → fix	339
Design questions checklist	339
Mapping to the journey map	340
Common mistakes	340
Summary	340
Site Tunnels	341
Site Tunnels	342
Learning goals	342
When site tunnels are the right tool	342
Topology — two sites + “Internet”	343
Addressing plan	343
Lab 1 — GRE + static routes	344
Predict	345
Observe	345
Fix	345
Lab 2 — MTU blackhole drill	345
Predict	345
Lab 3 — Underlay failure	346
Predict	346
Route-based vs policy-based VPN	346
Lab 4 — WireGuard route-based (if packages available)	346
Predict → observe	347
FRR over GRE (dynamic optional)	347
Policy-based IPsec awareness	347
Security notes (vendor-agnostic)	348

verify.sh sketch	348
Common mistakes	348
Summary	348
VXLAN Data Plane	349
VXLAN Data Plane	350
Learning goals	350
Concepts	350
Roles	350
Header (mental)	351
Data-plane only learning	351
Multicast vs head-end replication	351
When VXLAN is appropriate	351
Topology — two VTEPs + underlay + hosts	351
Addressing	352
VXLAN + bridge on leaf1	353
Deploy checks	354
Predict	354
Observe	354
Fix	354
Drill — MTU	355
Predict	355
Drill — wrong VNI isolation	355
Predict	355
Drill — MAC move / learning	355
Predict	355
Flood scope risk	356
FRR / SR Linux awareness	356
Verification checklist	356
Common mistakes	356
Summary	356
EVPN Control-Plane Intro	358
EVPN Control-Plane Intro	359
Learning goals	359
Why EVPN exists	359
Objects (vendor-agnostic)	360
Control vs data verification split	360
Minimal design story — 2 leaf	360
FRR capability note (mid-2026)	361
Topology sketch (EVPN lab)	361
RR config idea	362
Predict → observe → fix drills	363
Drill 1 — Session before service	363
Drill 2 — Host MAC appears as EVPN route	363
Drill 3 — Kill EVPN, keep underlay	363

Drill 4 — RT mismatch (concept)	363
L2 VNI vs L3 VNI (awareness)	363
Optional SR Linux community path	364
Security / isolation checklist	364
What this intro deliberately skips	364
Common mistakes	365
Summary	365
Overlay Lab	366
Overlay Lab	367
Learning goals	367
Pick a track	367
Definition of done (all tracks)	367
Track A — Dual-site GRE (reference build)	368
Success metrics	368
verify.sh	368
Failure matrix A	369
Track B — VXLAN two-leaf (reference build)	369
Success metrics	369
bringup excerpt	369
verify.sh B	369
Failure matrix B	370
Capture helper	370
Track C — EVPN intro (stretch)	370
Success metrics	370
Failure matrix C	370
Integrated “mini fabric snack” (optional merge)	371
Observability expectations	371
Hardening lite (overlay)	371
Lab report template	372
Predict → observe → fix (capstone habit)	372
Time-box guidance	372
Common mistakes	372
Summary	373
Fabrics and Multi-Area	374
Clos Leaf-Spine	375
Clos Leaf-Spine	376
Learning goals	376
Why Clos-style fabrics	376
Roles	377
Oversubscription (awareness)	377
Underlay protocol choices	378
Addressing plan (2×2)	378
Topology YAML	378

FRR daemons	380
OSPF underlay sketch (11)	380
eBGP underlay sketch (optional alternate)	381
Deploy and ECMP verify	381
Predict	381
Observe multipath	381
Drill 1 — Spine loss	382
Predict → observe → fix	382
Drill 2 — One leaf uplink down	382
Predict	382
Drill 3 — Polarization awareness	382
East-west vs north-south	383
Overlay placement (preview)	383
Verification checklist	383
Common mistakes	383
Summary	384
Multi-Area OSPF	385
Multi-Area OSPF	386
Learning goals	386
Concepts	386
Why areas	386
Backbone rule (practical)	386
Roles	387
LSA awareness (not flash cards)	387
Summarization	387
Topology	387
Topology YAML	388
FRR configs	389
r1 (area 1)	389
r3 (area 2)	389
r2 (ABR)	389
Deploy and baseline	390
Predict	390
Observe	390
Drill 1 — Summarization blackhole	391
Predict	391
Fix	391
Drill 2 — ABR failure	391
Predict	391
Drill 3 — Area mismatch	392
Predict	392
Stub area ideas (awareness)	392
Multi-area vs fabric BGP	392
Verification checklist	393
Common mistakes	393
Summary	393

Fabric with FRR	394
Fabric with FRR	395
Learning goals	395
Lab repo layout	395
Design defaults for this book	396
eBGP unnumbered (modern fabric style)	396
Numbered eBGP fabric (reliable teaching)	396
Spine s1 excerpt	397
Leaf l1 excerpt	397
OSPF alternate (same wiring)	398
Containerlab topology	398
Operational show kit	398
verify.sh	399
Failure scripts	399
Predict → observe → fix	399
Policy on fabric (light)	400
Adding overlay later (hook points)	400
Scale checklist 2×2 → 4×4	400
Observability	401
Optional SONiC / SR Linux leaf	401
Common mistakes	401
Summary	401
Multi-Implementation Lab	402
Multi-Implementation Lab	403
Learning goals	403
Design principle	403
Recommended topology	404
Addressing / ASN (fixed)	404
Topology YAML (FRR + optional srl)	404
FRR side	405
SR Linux mapping (conceptual)	406
Verification questions (dialect-free)	406
FRR answers	406
SR Linux answers (illustrative)	407
predict → observe → fix matrix	407
verify.sh (heterogeneous)	407
Overlay optional add-on	407
Hardening gates (reuse part 08)	408
Lab report template	408
When interop hurts	408
Common mistakes	409
Summary	409

Networking

Personal notes on learning **computer networking from first principles to expert NetOps**, with labs built as **code** in [Containerlab](#) using **open-source and free network images** only (FRR, Linux, optional SR Linux / SONiC / VyOS).

i Note

This book is **vendor-agnostic**. CLIs differ; protocols do not. Start with the **lab platform** (macOS/Linux + Containerlab), then a deep **networking foundations** pass (TCP/IP, addressing, core protocols), then L2 → L3 → routing → edge → policy → overlays → fabrics → ops. Topology figures are illustrated SVGs under [images/](#).

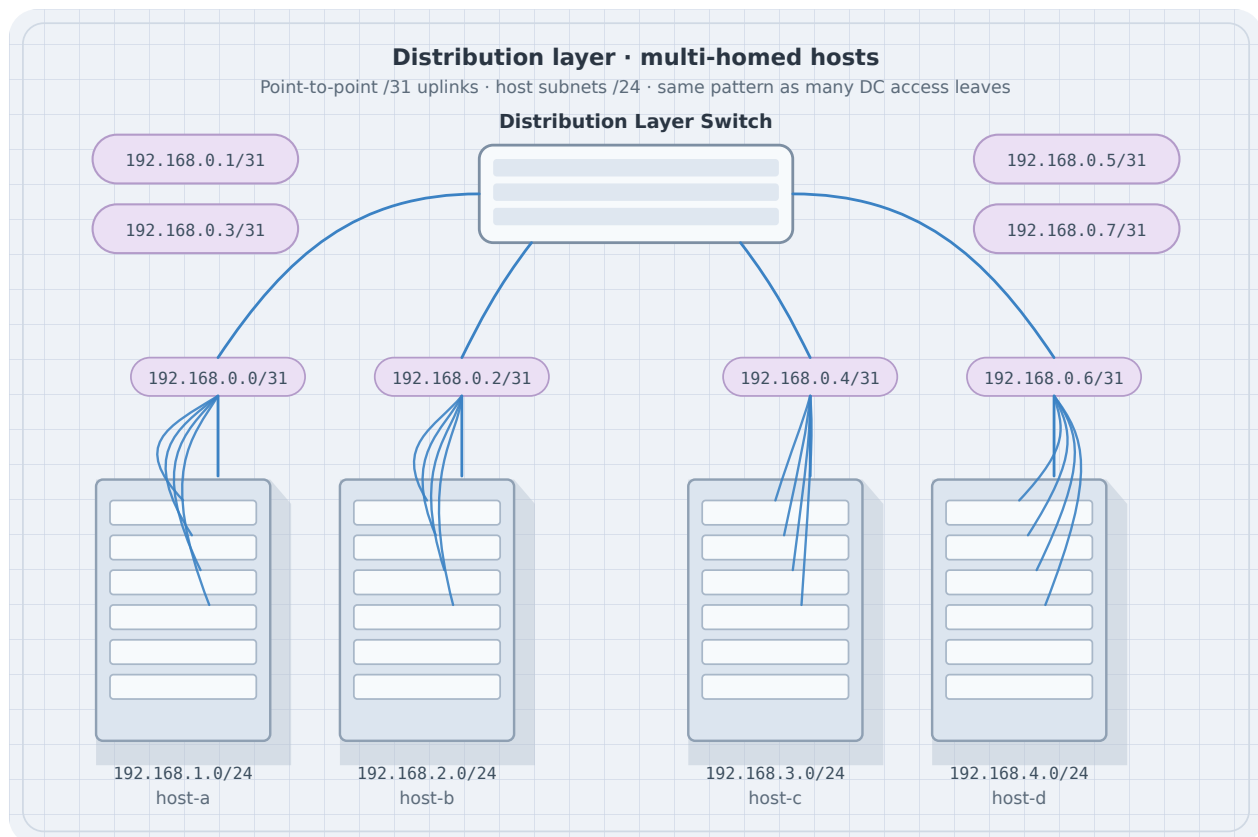


Figure 1: Distribution switch with multi-homed servers

Curriculum map

Part	Focus
01 Lab platform	macOS/Linux host setup, Containerlab, free images, host networking, hygiene
02 Networking foundations	TCP/IP stack, packets, Ethernet intro, IPv4/IPv6, ARP/ND, ICMP, TCP/UDP, common apps, host routing, planes
03 Layer 2	Ethernet/MAC deep, VLANs, loops, LAG, failure drills
04 Layer 3	Advanced addressing/routing labs, statics, ICMP/PMTU in multi-hop designs
05 Interior routing	OSPF, BGP, policy & filters
06 Ops & automation	Observability, lab-as-code, capstone outline
07 Edge & services	First-hop HA, DHCP/NAT in edge designs, hygiene, dual-home lab
08 Policy, QoS, hardening	ACLs, CoPP, QoS vocabulary, hardening
09 Overlays & tunnels	Encap, site tunnels, VXLAN, EVPN intro
10 Fabrics & multi-area	Clos, multi-area OSPF, FRR fabric, multi-NOS

Journey map

Lab platform (macOS / Linux + Containerlab)

- Networking foundations (TCP/IP, addressing, protocols)
- Layer 2 depth
- Layer 3 / routing depth
- Interior routing (OSPF, BGP)
- Edge & services
- Policy / QoS / hardening
- Overlays & tunnels
- Fabrics
- Ops & automation

What “done” looks like

- Design multi-site L2/L3 networks with clear underlay vs overlay roles
- Implement them in Containerlab with free images
- Prove behavior with tables, neighbors, and packet captures
- Break links and policies on purpose; restore and document
- Automate lab lifecycle

Lab ground rules

- Prefer **free/open** node kinds (Containerlab + FRR + Alpine/Debian)
- Every serious lab: topology file + addressing plan + failure scenario
- Capture evidence when it teaches something tables do not

Related books

Book	Overlap
Linux	Host networking commands (ip , ss , tcpdump , firewalls)
NixOS	Declarative network config on NixOS hosts
Go	net/http , probes, and load-test projects
VCS	Lab-as-code repos and CI for topology files

Formats

HTML, PDF, and EPUB via the monorepo portal when built.

Lab Platform

Lab Host — macOS

Lab Host — macOS

Prepare a **macOS** machine as the Containerlab control host: virtualization, container runtime, resources, and networking gotchas before you deploy topologies.

i Note

Stub chapter — outline for the lab-platform on-ramp. Expand with exact install commands and verified versions as you freeze a lab baseline.

Goals

- Run Docker-compatible containers with enough CPU/RAM for FRR + Linux nodes
- Install Containerlab and pull free lab images
- Understand macOS-specific limits (VM layer, file sharing, port publishing)

Prerequisites

- Recent macOS with virtualization support (Apple Silicon or Intel)
- Admin rights for Docker/Colima and Homebrew
- Disk space for images (plan multi-GB for NOS images)

Virtualization and container runtime

Options

Approach	Notes
Docker Desktop	Simple GUI; license/terms apply for some orgs
Colima + Docker CLI	Lightweight VM; common for CLI-first labs
Lima / other	Possible; keep one path documented for this book

Checklist

1. Install runtime and confirm `docker version` / `docker info`
2. Allocate CPU and memory for multi-node labs (start modest; scale up)
3. Confirm architecture (`linux/arm64` vs `linux/amd64`) matches images you pull

Install Containerlab

- Prefer the [Containerlab install docs](#) for the current macOS method
- Verify: `containerlab version`
- Confirm the binary can talk to your Docker socket

Networking notes on macOS

- Lab nodes run **inside a Linux VM** — host `localhost` port maps differ from native Linux
- Bridge and macvlan behaviors are not identical to bare-metal Linux
- Prefer topologies that do not assume bare-metal SR-IOV or host netns tricks on day one

First smoke test

1. Deploy a **two-node** FRR or Linux topology from Containerlab examples
2. `containerlab inspect` / `docker ps` — nodes up
3. `docker exec` into a node; `ping` the peer
4. Destroy the lab; confirm no orphan containers

Next

- Parallel path: [Lab Host — Linux](#)
- Then [Containerlab Fundamentals](#)

Lab Host — Linux

Lab Host — Linux

Prepare a **Linux** machine (bare metal or VM) as the Containerlab control host: kernel/container runtime, permissions, and host networking.

i Note

Stub chapter — outline for the lab-platform on-ramp. Expand with distro-specific package names as you freeze a baseline (Debian/Ubuntu recommended for docs).

Goals

- Install Docker or a Containerlab-supported runtime
- Install Containerlab and free images
- Avoid permission and firewall traps that break lab deploy

Prerequisites

- 64-bit Linux with virtualization if nested (for some images)
- Root or passwordless sudo for install steps
- Enough RAM/disk for your target topology scale

Container runtime

Typical path (Debian/Ubuntu-style)

1. Install Docker Engine (or Podman only if you know Containerlab support for your version)
2. Add your user to the `docker` group (or use root consistently in labs)
3. Enable and start the service; `docker info` succeeds

Kernel / modules

- Bridge, veth, and overlay-related features must work
- Nested virt: confirm if you need it for chosen NOS images

Install Containerlab

- Follow [Containerlab install](#) for Linux
- Verify: `containerlab version`
- Optional: bash completion

Host networking and firewall

- Local firewalls (ufw, firewalld, nftables) can block management or published ports
- Don't disable security blindly — open what the lab needs
- IP forwarding may be required for some designs later

First smoke test

1. Deploy a minimal two-node topology
2. Confirm links and management connectivity
3. Capture with `tcpdump` on a veth if useful
4. `containerlab destroy -t <file>` — clean teardown

Next

- macOS path: [Lab Host — macOS](#)
- Then [Containerlab Fundamentals](#)

Containerlab Fundamentals

Containerlab Fundamentals

[Containerlab](#) turns a YAML topology into runnable network labs: create nodes, wire virtual links, attach a management network, and destroy everything cleanly. This chapter is the operator's manual for the rest of the book's labs.

Learning goals

By the end of this chapter you can:

- Read and write a minimal `*.clab.yml` topology
- Deploy, inspect, connect, and destroy labs
- Choose free node **kinds** (FRR, Linux, etc.)
- Understand management network vs data-plane links
- Export a graph and use consistent naming
- Recover from partial deploys and leftover state

Prerequisites

- Linux host or VM with nested virt as needed (Docker or podman per Containerlab docs)
- `containerlab` installed ([install guide](#))
- Permission to run privileged containers / `sudo` as required
- Images pullable from public registries for free kinds

Check:

```
containerlab version
docker version    # or podman
```

How Containerlab fits together

```
topology.clab.yml
```

```
containerlab deploy
```

```
create containers (nodes)  
create veth/links between interfaces  
attach management network (SSH/mgmt IPs)  
optional startup configs / binds
```

Destroy reverses the graph. **Source of truth is the YAML** (plus mounted configs), not hand edits inside nodes.

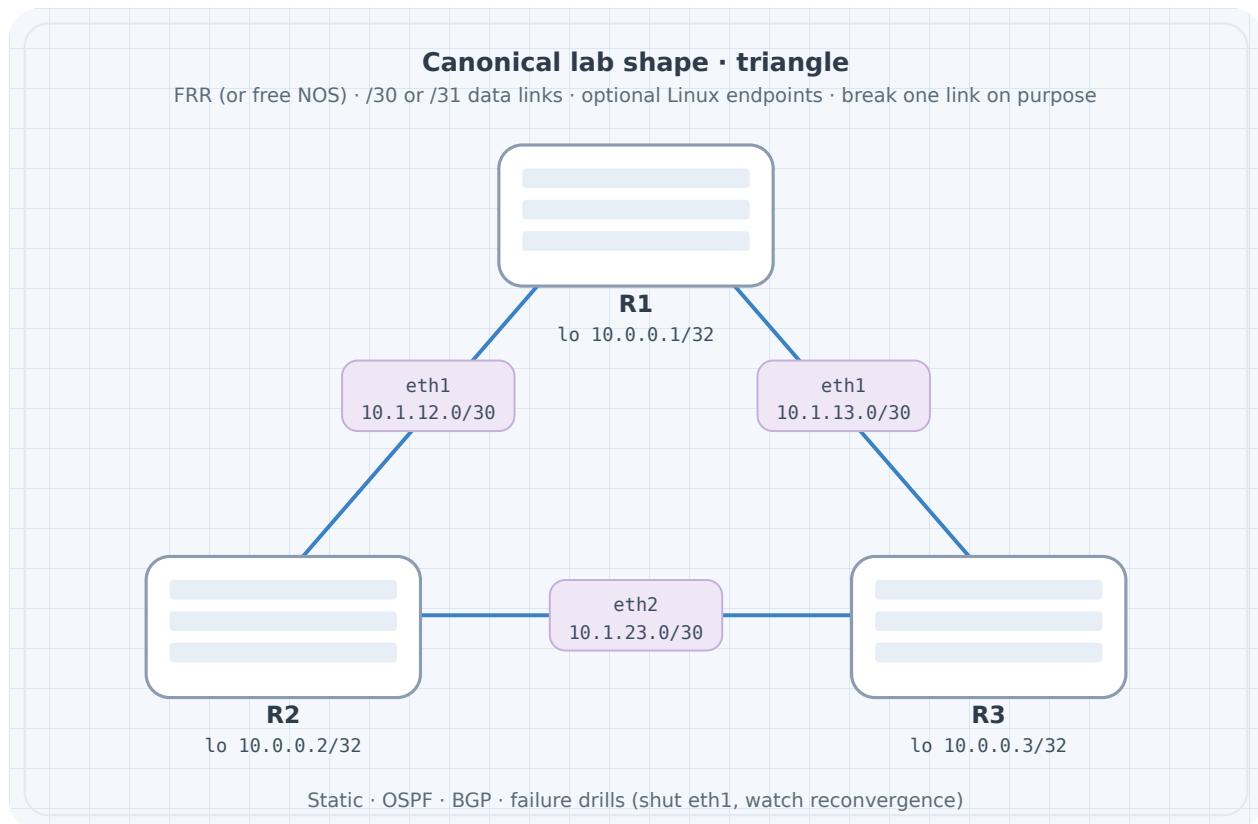


Figure 1: Triangle lab

Minimal topology: two Linux nodes

```
name: twoping

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
    h2:
      kind: linux
      image: alpine:3.20

  links:
    - endpoints: ["h1:eth1", "h2:eth1"]
```

Save as `twoping.clab.yml`.

```
sudo containerlab deploy -t twoping.clab.yml
sudo containerlab inspect -t twoping.clab.yml
```

Address the link and test (interfaces may need addressing—Alpine is raw):

```
# Names often: clab-twoping-h1
docker exec -it clab-twoping-h1 sh
# inside:
ip addr add 10.0.0.1/24 dev eth1
ip link set eth1 up
# on h2:
# ip addr add 10.0.0.2/24 dev eth1 && ip link set eth1 up
# ping 10.0.0.1
```

For real labs you bake addressing into startup scripts or use NOS images that apply configs. Pattern: **deploy** → **configure via files** → **verify**.

Triangle of FRR routers (book default pattern)

FRR containers give you a real control plane without licenses.

```
name: triangle

topology:
  nodes:
    r1:
      kind: linux
```



```

    image: quay.io/frROUTING/frr:10.2.1
    binds:
      - ./config/r1:/etc/frr
  r2:
    kind: linux
    image: quay.io/frROUTING/frr:10.2.1
    binds:
      - ./config/r2:/etc/frr
  r3:
    kind: linux
    image: quay.io/frROUTING/frr:10.2.1
    binds:
      - ./config/r3:/etc/frr

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]

```

Note: Pin image tags deliberately (example tag may differ—check current FRR container tags). Each node mounts its own `./config/rN` to `/etc/frr`.

Startup config sketch (config/r1/daemons + frr.conf)

FRR images expect daemons enabled. Example `frr.conf` ideas (interface names must match):

```

frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
  ip address 10.0.12.1/24
!
interface eth2
  ip address 10.0.13.1/24
!
ip route 0.0.0.0/0 10.0.12.2
!
line vty

```

Exact FRR container entrypoint behavior evolves—follow the image README. The **topology pattern** (binds + links) is the durable skill.

Lifecycle

```
sudo containerlab deploy -t triangle.clab.yml
sudo containerlab graph -t triangle.clab.yml # if supported in your version
sudo containerlab inspect -t triangle.clab.yml
docker exec -it clab-triangle-r1 vtysh
sudo containerlab destroy -t triangle.clab.yml --cleanup
```

Topology YAML anatomy

Key	Purpose
name	Lab name prefix for containers/namespaces
topology.nodes	Node inventory
topology.links	Point-to-point (or multipoint via extras)
kind	How Containerlab treats the node
image	Container image
binds	Host path mounts
exec	Commands after start (use sparingly; prefer files)
env	Environment variables
ports	Publish container ports to host (mgmt tools)
mgmt-ipv4	Static management address (optional)

Kinds you will actually use

Kind / image pattern	Role
linux + Alpine/Debian	Hosts, simple routers with FRR image
FRR container on linux kind	OSPF/BGP labs
sr1 (SR Linux community)	Modern NOS practice (free image path)
VyOS / SONiC images	Optional diversity

Always confirm **license-free** use for the image you pull. This book never requires paid virtual NOS licenses.

Links and endpoints

```
links:
  - endpoints: ["r1:eth1", "r2:eth1"]
```

Rules of thumb:

- Interface names must match what the NOS expects or what you configure
- Point-to-point links are veth pairs under the hood
- Do not reuse the same interface name twice on one node
- Leave `eth0` for management if the kind uses it for mgmt

Multipoint / LAN segments

Patterns include attaching multiple nodes via a bridge node or kind-specific multipoint links. Early labs stick to **point-to-point** for clarity; L2 chapters introduce switch nodes.

Management network

Containerlab creates a management network so you can SSH/API to nodes without using data-plane links.

```
sudo containerlab inspect -t triangle.clab.yml
# Note mgmt IPv4 addresses
ssh admin@<mgmt-ip> # credentials depend on image
```

Predict: Breaking a data-plane link does not remove SSH via mgmt (unless you broke the host docker network). That separation mirrors production OOB vs in-band—use it.

Inspect, exec, save

```
# Table of nodes and mgmt IPs
sudo containerlab inspect -t triangle.clab.yml

# Shell
docker exec -it clab-triangle-r1 bash

# FRR
docker exec -it clab-triangle-r1 vtysh -c 'show interface brief'

# Destroy
sudo containerlab destroy -t triangle.clab.yml --cleanup
```

`--cleanup` removes lab containers and lab-specific wiring; use it to avoid ghost interfaces.

Lab directory layout (recommended)

```
labs/triangle/  
  triangle.clab.yml  
  README.md  
  addressing.md  
  verify.sh  
  config/  
    r1/  
    r2/  
    r3/  
  captures/  
  
cd labs/triangle  
sudo containerlab deploy -t triangle.clab.yml  
./verify.sh
```

Graph and documentation

```
sudo containerlab graph -t triangle.clab.yml
```

Import diagrams into your journal. Combine with a hand-drawn addressing plan—graph alone lacks subnets.

Partial failure recovery

If deploy dies halfway:

```
sudo containerlab destroy -t triangle.clab.yml --cleanup  
docker ps -a | grep clab  
# remove stragglers if needed  
sudo containerlab deploy -t triangle.clab.yml
```

Also check:

```
docker images | head  
df -h          # disk full breaks pulls  
ip link | wc -l # too many leftover veths
```

Predict → observe → fix lab

Setup

Deploy `twoping` with addresses applied via `exec` for speed (acceptable for learning; later prefer binds):

```
name: twoping

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - ip addr add 10.0.0.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - ip addr add 10.0.0.2/24 dev eth1
        - ip link set eth1 up
  links:
    - endpoints: ["h1:eth1", "h2:eth1"]
```

Note: `exec` timing can race interface creation on some versions—if `eth1` missing, retry commands or use a small sleep in a script bind.

Predict

- `ping 10.0.0.2` from `h1` succeeds
- `ip neigh` shows REACHABLE for peer

Observe

```
docker exec clab-twoping-h1 ping -c 3 10.0.0.2
docker exec clab-twoping-h1 ip neigh
docker exec clab-twoping-h1 ip -br a
```

Fix inject

```
docker exec clab-twoping-h2 ip link set eth1 down
docker exec clab-twoping-h1 ping -c 2 10.0.0.2 # fail
docker exec clab-twoping-h2 ip link set eth1 up
```

Harden

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-twoping-h1 ping -c 2 -W 1 10.0.0.2
echo OK
```

Re-deploy from clean destroy and ensure `verify.sh` still passes.

Common mistakes

Mistake	Symptom	Fix
Wrong interface name in links	Deploy error or silent no-path	Match kind docs
Editing live node only	Lost on destroy	Binds + git
Forgetting <code>--cleanup</code>	Ghost links/names	Always cleanup when done
Huge images, tiny laptop	OOM kills	Smaller kinds, fewer nodes
Mixing lab names	Container name collisions	Unique <code>name</code> : per active lab

Version awareness (mid-2026)

Containerlab releases steadily. Skills that stay:

- Topology as code
- kinds + images + links + binds
- deploy / inspect / destroy lifecycle

Syntax extras (stages, components, interfaces block style) may appear in newer docs—read containerlab.dev for your installed version when YAML fails validation.

```
containerlab version
containerlab help deploy
```

Security and shared hosts

- Labs often run privileged networking—isolate lab hosts
- Do not expose mgmt SSH to the internet
- Pull images from trusted registries; pin digests for CI later

Summary

- Containerlab materializes YAML into namespaces, containers, and links
- Master deploy → inspect → exec → destroy –cleanup
- Prefer binds/startup configs over irreproducible exec-only hacks
- Keep per-lab directories with verify scripts
- FRR + Linux free images cover most of this book’s spine

Next: **free image matrix**—what to run without licenses, and how to size CPU/RAM.

Free Image Matrix & Resources

Free Image Matrix & Resources

The main path of this book uses **open-source and freely available** lab images only. No paid virtual NOS license is required to finish the curriculum. This chapter catalogs practical choices (as of mid-2026 practice) and how to size a lab host.

Learning goals

By the end of this chapter you can:

- Pick images for endpoints, routers, and optional modern NOS practice
- Estimate RAM/CPU for small vs medium topologies
- Avoid accidental license walls
- Pin versions for reproducibility
- Know where official docs and communities live

Principles

1. **Free to pull and run for learning** on the main path
2. **Documented interfaces** (Linux, FRR vtysh, open NOS models)
3. **Containerlab-friendly** kinds where possible
4. **Pin tags**—`latest` is for tourists

Always re-check the image's license and Containerlab kind docs before classroom or corporate use; policies change.

Role matrix

Role	Recommended default	Alternatives (still free/open path)
Endpoint host	<code>alpine:3.20</code> or Debian slim	Ubuntu minimal
Software router / FRR	<code>quay.io/frrouting/frr:<tag></code>	FRR on Debian + manual setup
Modern NOS feel	Nokia SR Linux community image via Containerlab kind	SONiC community images
Firewall / edge Swiss army	VyOS rolling (confirm current license terms)	Linux nftables node
Switch-like L2	Linux bridge / OVS container; FRR + bridge	SR Linux L2 features
Traffic tools	Alpine/Debian + <code>iperf3</code> , <code>tcpdump</code> , <code>mtr</code>	Dedicated trafficgen images
Capture analysis	Host <code>tshark</code> / Wireshark	Container with tshark
Automation control	Host Python/Ansible	CI runner container

What we deliberately de-emphasize

Avoid on main path	Why
Paid IOS/NX-OS/JUNOS virts	License friction; concepts unchanged
Exam dump topologies	Wrong optimization target
Huge multi-vendor museums	RAM death; low learning density

Paid images can be a **later footnote** if your employer provides them—not a gate.

FRR as the workhorse

FRRouting implements OSPF, BGP, IS-IS, statics, and more with a Cisco-like vtysh. Container images are the fastest path:

```
# Example - check quay for current stable tags
docker pull quay.io/frrouting/frr:10.2.1
```

Strengths: control-plane labs, policy, dual-stack

Limits: not a full merchant silicon simulator; L2 features depend on how you combine Linux bridging

Lab pattern:

```
# snippet
r1:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/r1:/etc/frr
```

Enable only needed daemons (bgpd=yes, ospfd=yes) in daemons file to save RAM.

Linux endpoints

Alpine is small and fast:

```
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils busybox-extras
```

Debian if you need a richer package set:

```
h1:
  kind: linux
  image: debian:bookworm-slim
  exec:
    - apt-get update && apt-get install -y iproute2 iputils-ping tcpdump iperf3 mtr-tiny
```

Tip: bake a personal lab image with tools preinstalled to save time:

```
FROM alpine:3.20
RUN apk add --no-cache iproute2 iputils tcpdump iperf3 mtr bind-tools
```

```
docker build -t local/lab-endpoint:1.0 .
```

Use local/lab-endpoint:1.0 in topologies.

SR Linux community (optional track)

Nokia SR Linux community images are commonly used with Containerlab for modern streaming telemetry and CLI/model exposure—without classic paid chassis IOS requirements. Follow current Containerlab **kind: srl** documentation for image names and credentials.

Use SR Linux when you want:

- YANG-ish / modern NOS workflows
- Leaf-spine demos closer to DC ops
- Comparison against FRR (“same OSPF idea, different dialect”)

Do not block L2/L3 fundamentals waiting for SR Linux. FRR+Linux is enough for competence.

SONiC and VyOS (optional)

Image family	Good for	Watch for
SONiC	DC NOS ideas, BGP underlay practice	Image size, platform assumptions
VyOS	Edge router feature bundle	Release/license wording over time

Treat both as **breadth**, not blockers.

Sizing the lab host

Rough starting points (conservative; measure yours):

Lab scale	Nodes (example)	RAM ballpark	CPU
Tiny	2-3 Alpine	2-4 GB free	2
Small	3× FRR + 2 hosts	8 GB system	4
Medium	6-8 FRR/SRL + hosts	16 GB	6-8
Fabric snack	2 spine + 4 leaf + hosts	32 GB happier	8+

```
free -h
nproc
docker stats --no-stream
```

RAM savers

- Prefer Alpine endpoints over full Ubuntu desktops
- Disable unused FRR daemons
- Destroy labs when idle

- Avoid multiple concurrent large labs
- Do not run heavy desktop + large lab + browser zoo on 8 GB

Disk

Images accumulate:

```
docker system df
docker image prune
# careful:
# docker system prune
```

Pin and keep only tags you need.

Version pinning strategy

```
# Good
image: quay.io/frrouting/frr:10.2.1
image: alpine:3.20

# Risky for books/CI
image: quay.io/frrouting/frr:latest
image: alpine:latest
```

In CI (later chapter), pin digests:

```
image: quay.io/frrouting/frr:10.2.1@sha256:....
```

Record pins in each lab's README.

Resource planning worksheet

Before a new topology, write:

```
## Sizing
Nodes:
- 3 x FRR (~X MB each estimated)
- 2 x Alpine
Total RAM budget: ...
Host free RAM: ...
Fallback if OOM: remove r3, collapse to chain
```

If deploy dies with OOM kills:

```
dmesg -T | tail -50
docker ps -a
```

Network resource limits on the host

Resource	Symptom when exhausted
RAM	Random node death
CPU	Protocol timers flaky, loss
Inotify/watches	Rare editor/CI issues
Interface count	Fail to create veth
Docker IP pool	Mgmt network allocate fail

```
ip link | wc -l
docker network ls
```

Destroy cleaned labs free veths.

Documentation & community map

Resource	Use
containerlab.dev	Topology reference, kinds
docs.frouting.org	OSPF/BGP/commands
RFCs (IETF)	Authoritative protocol behavior
Linux <code>man ip</code> , <code>man bridge</code>	Host dataplane
Lab repos using Containerlab	Inspiration—verify licenses

RFCs worth bookmarking (foundations)

Topic	Starting points
IP	RFC 791 (historic), RFC 8200 (IPv6)
ICMP	RFC 792, RFC 4443
ARP	RFC 826
OSPF	RFC 2328 (v2)
BGP	RFC 4271
Ethernet/VLAN	IEEE world + RFC 5517 context notes

You do not memorize RFCs; you **use them when models conflict**.

Predict → observe: image sanity lab

```
name: imgcheck

topology:
  nodes:
    frr1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
    a1:
      kind: linux
      image: alpine:3.20
  links:
    - endpoints: ["frr1:eth1", "a1:eth1"]
```

Predict: Both containers running; `vttysh` answers on `frr1`; Alpine has `ip` after `apk` or use `busybox` `ip` if present.

```
sudo containerlab deploy -t imgcheck.clab.yml
docker exec clab-imgcheck-frr1 vtysh -c 'show version'
docker exec clab-imgcheck-a1 uname -a
sudo containerlab destroy -t imgcheck.clab.yml --cleanup
```

Harden: Document the exact tags that work on your machine in a personal `IMAGES.md`.

Multi-implementation mindset

Later design labs may mix FRR + Linux + optional SR Linux. Same OSPF adjacency model; different show commands. Keep a translation scrap:

Intent	FRR	Linux
Addresses	<code>show interface</code>	<code>ip -br a</code>
Routes	<code>show ip route</code>	<code>ip route</code>
BGP summary	<code>show bgp summary</code>	n/a or <code>bird/etc</code>

Ethics and redistribution

- Do not redistribute proprietary images
- Respect registry rate limits and ToS
- Prefer building lab content others can run **without** secrets or licenses

Summary

- Default stack: Containerlab + FRR + Alpine/Debian
- Optional free NOS: SR Linux community, SONiC, VyOS—with eyes open
- Size hosts honestly; destroy idle labs
- Pin image tags; record what works
- Official docs + RFCs beat random CLI folklore

Next: **lab hygiene**—addressing plans, naming, git layout, and diagram habits so multi-lab life stays sane.

Linux Networking for Lab Hosts

Linux Networking for Lab Hosts

Containerlab sits on Linux networking primitives: namespaces, veth pairs, bridges, routes, and basic filtering. If you only drive YAML without understanding the host, debugging feels like magic. This chapter builds the host-side intuition you need before (and during) Containerlab work.

Learning goals

By the end of this chapter you can:

- Explain how network namespaces isolate stacks
- Create veth pairs and bridges by hand
- Add addresses and routes; verify with `ip route get`
- Use `tcpdump` inside a namespace
- Map Containerlab links to veth/bridge concepts
- Apply minimal filter awareness (`nft`/`iptables`, `rp_filter`)

Learning goals in practice

You will build a **two-namespace router lab** without Containerlab, then tear it down. That is the same physics Containerlab orchestrates for you.

Core objects

Object	Role
Interface	L2 endpoint (physical, veth, bridge, vlan subint)
Network namespace	Isolated network stack (interfaces, routes, sockets)
veth pair	Virtual cable: two ends, often in different namespaces
Bridge	Software switch in the kernel

Object	Role
Route	FIB entry: destination → device / next hop
Rule	Policy routing (advanced; optional here)

```
ip -br link
ip -br addr
ip netns list
ip route
```

Network namespaces

A namespace process sees the **root** namespace. Lab nodes (containers) each get a namespace.

```
# Create two namespaces
sudo ip netns add ns1
sudo ip netns add ns2
ip netns list

# Run a command inside
sudo ip netns exec ns1 ip link
sudo ip netns exec ns1 ping -c1 127.0.0.1
```

Loopback exists per namespace; bring it up if you care about local sockets:

```
sudo ip netns exec ns1 ip link set lo up
```

veth pairs: virtual cables

```
# Create pair veth-a <-> veth-b
sudo ip link add veth-a type veth peer name veth-b

# Move ends into namespaces
sudo ip link set veth-a netns ns1
sudo ip link set veth-b netns ns2

# Name + address inside each ns
sudo ip netns exec ns1 ip link set veth-a name eth0
sudo ip netns exec ns2 ip link set veth-b name eth0

sudo ip netns exec ns1 ip addr add 10.0.0.1/24 dev eth0
sudo ip netns exec ns2 ip addr add 10.0.0.2/24 dev eth0
```

```
sudo ip netns exec ns1 ip link set eth0 up
sudo ip netns exec ns2 ip link set eth0 up
```

```
# Test L3 on-link
sudo ip netns exec ns1 ping -c 2 10.0.0.2
```

Predict: ARP resolves; ICMP works; no router required because same subnet.

Observe:

```
sudo ip netns exec ns1 ip neigh
sudo ip netns exec ns1 tcpdump -ni eth0 -c 10 arp or icmp &
sudo ip netns exec ns1 ping -c 2 10.0.0.2
```

Bridges: software switches

Connect multiple endpoints as if on a switch:

```
sudo ip netns add h1
sudo ip netns add h2
sudo ip netns add h3

sudo ip link add br0 type bridge
sudo ip link set br0 up

# veth for each host: host side in ns, peer on host attached to bridge
for n in 1 2 3; do
    sudo ip link add veth-h$n type veth peer name veth-b$n
    sudo ip link set veth-h$n netns h$n
    sudo ip netns exec h$n ip link set veth-h$n name eth0
    sudo ip netns exec h$n ip addr add 10.10.0.$n/24 dev eth0
    sudo ip netns exec h$n ip link set eth0 up
    sudo ip netns exec h$n ip link set lo up
    sudo ip link set veth-b$n master br0
    sudo ip link set veth-b$n up
done

sudo ip netns exec h1 ping -c 2 10.10.0.2
sudo ip netns exec h1 ping -c 2 10.10.0.3
```

Bridge FDB (MAC table):

```
bridge fdb show br br0
# or
ip neigh # not the same; FDB is L2 forward database
```

```
bridge link show
```

After pings, you should see learned MACs on the bridge ports.

Routing between namespaces

Classic: two LANs and a router namespace.

```
h1 (10.1.0.10/24) -- veth -- r (10.1.0.1 + 10.2.0.1) -- veth -- h2 (10.2.0.10/24)
```

```
sudo ip netns add h1
sudo ip netns add h2
sudo ip netns add r

# h1 <-> r
sudo ip link add v1 type veth peer name r1
sudo ip link set v1 netns h1
sudo ip link set r1 netns r
sudo ip netns exec h1 ip link set v1 name eth0
sudo ip netns exec r ip link set r1 name eth1
sudo ip netns exec h1 ip addr add 10.1.0.10/24 dev eth0
sudo ip netns exec r ip addr add 10.1.0.1/24 dev eth1
sudo ip netns exec h1 ip link set eth0 up
sudo ip netns exec r ip link set eth1 up
sudo ip netns exec h1 ip link set lo up
sudo ip netns exec r ip link set lo up

# h2 <-> r
sudo ip link add v2 type veth peer name r2
sudo ip link set v2 netns h2
sudo ip link set r2 netns r
sudo ip netns exec h2 ip link set v2 name eth0
sudo ip netns exec r ip link set r2 name eth2
sudo ip netns exec h2 ip addr add 10.2.0.10/24 dev eth0
sudo ip netns exec r ip addr add 10.2.0.1/24 dev eth2
sudo ip netns exec h2 ip link set eth0 up
sudo ip netns exec r ip link set eth2 up
sudo ip netns exec h2 ip link set lo up

# Enable forwarding on router ns
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=1

# Default routes on hosts
sudo ip netns exec h1 ip route add default via 10.1.0.1
sudo ip netns exec h2 ip route add default via 10.2.0.1
```

```
# Test
sudo ip netns exec h1 ping -c 3 10.2.0.10
sudo ip netns exec h1 traceroute -n 10.2.0.10
```

Predict → observe:

```
sudo ip netns exec h1 ip route get 10.2.0.10
sudo ip netns exec r ip route get 10.2.0.10 from 10.1.0.10 iif eth1
sudo ip netns exec r ip -s link show eth1
```

Failure drill: disable forwarding

```
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=0
sudo ip netns exec h1 ping -c 2 10.2.0.10 # expect fail
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=1
```

Harden note: container routers must allow forwarding; images often set this, but verify.

Failure drill: missing return route

If the “router” only had a connected route on one side (misconfigured static elsewhere), one-way traffic appears. With connected routes on both interfaces above, both directions work. Later static chapters amplify return-path bugs.

VLAN subinterfaces (preview of trunks)

```
# On a namespace with eth0
sudo ip netns exec h1 ip link add link eth0 name eth0.10 type vlan id 10
sudo ip netns exec h1 ip addr add 10.10.10.1/24 dev eth0.10
sudo ip netns exec h1 ip link set eth0.10 up
```

Both ends need matching VLAN IDs. Bridges can be VLAN-aware (`vlan_filtering`); Containerlab L2 labs often use OVS or NOS switches—concepts transfer.

Policy leftovers that break labs

rp_filter (reverse path filtering)

Strict reverse-path checks drop asymmetric legitimate lab traffic.

```
sysctl net.ipv4.conf.all.rp_filter
sysctl net.ipv4.conf.default.rp_filter
# Per-iface:
# sysctl net.ipv4.conf.eth0.rp_filter
```

If mysterious one-way drops appear on Linux routers, check `rp_filter` and counters.

Forwarding and firewall

```
# nftables quick view
sudo nft list ruleset
# legacy
sudo iptables -L -n -v
sudo iptables -t nat -L -n -v
```

Host firewalls (especially Docker’s chains) can interact with lab bridges. When a lab “should work” and does not, inspect host filter tables—not only node configs.

Disable offloads when captures look weird (optional)

Checksum offload can make `tcpdump` show wrong checksums on local veth. Prefer understanding this over panicking:

```
sudo ethtool -k veth0 | grep sum || true
```

Mapping to Containerlab

You did by hand	Containerlab roughly does
<code>ip netns add</code>	Creates container network namespaces
veth pairs	Links between nodes
bridges / OVS	Multipoint wiring, mgmt network
addressing	From topology + image startup
<code>ip_forward</code>	Inside router nodes

When Containerlab deploy fails or links are dark, drop to:

```
ip netns list
# Containerlab often names netsns like clab-<lab>-<node>
sudo ip netns exec clab-mylab-r1 ip -br a
```

Exact naming depends on lab name and version; `containerlab inspect` helps.

```
sudo containerlab inspect -t topology.clab.yml
```

Clean teardown habits

Namespaces and veths litter a host if you exit mid-lab.

```
# Example cleanup for the objects you created
sudo ip netns del h1 2>/dev/null || true
sudo ip netns del h2 2>/dev/null || true
sudo ip netns del r 2>/dev/null || true
sudo ip netns del ns1 2>/dev/null || true
sudo ip netns del ns2 2>/dev/null || true
sudo ip link del br0 2>/dev/null || true
# dangling veths usually die with netns; if not:
ip -br link | grep veth || true
```

Prefer scripted `up.sh` / `down.sh` even for hand labs.

Example `down.sh`

```
#!/usr/bin/env bash
set -euo pipefail
for ns in h1 h2 r ns1 ns2; do
    sudo ip netns del "$ns" 2>/dev/null || true
done
sudo ip link del br0 2>/dev/null || true
echo "cleaned"
```

Predict → observe → fix drill (checklist)

Lab: three hosts on br0 as above.

1. **Predict:** h1→h3 uses bridge flood then learn; FDB populates.
2. **Observe:** bridge fdb show, tcpdump on veth-b1.
3. **Fix inject:** `sudo ip link set veth-b3 down` — h3 unreachable; FDB ages/fails.
4. **Harden:** document “access port down vs cable unplug” equivalence in labs.

Useful one-liners cheatsheet

```
# Watch neigh events
ip monitor neigh

# Route decision
ip route get 1.1.1.1 from 10.1.0.10 iif eth0

# Stats
ip -s -s link show eth0

# All addresses every ns (rough)
for ns in $(ip netns list | awk '{print $1}'); do
    echo "=====$ns===="
    sudo ip netns exec "$ns" ip -br a
done
```

Security note for shared hosts

Namespace labs are powerful. On a shared machine:

- Do not bridge lab interfaces onto trusted production networks without filters
- Clean up after sessions
- Avoid enabling `ip_forward` on the **root** namespace unless you intend the host to route

Containerlab management networks still need isolation discipline on corporate laptops.

Summary

- Namespaces isolate full network stacks—foundation of container networking
- veth pairs are virtual cables; bridges are virtual switches
- Router namespaces need addressing, up state, forwarding, and routes both ways
- Host firewall and `rp_filter` are common “invisible” lab breakers
- Hand-building once makes Containerlab YAML readable and debuggable

Next: **Containerlab fundamentals**—topology YAML, kinds, links, lifecycle—on top of these primitives.

Lab Hygiene

Lab Hygiene

Brilliant topologies rot without hygiene: colliding subnets, random hostnames, uncommitted click-ops, and mystery cables. This chapter is the professional layer on top of Containerlab—how to keep labs reproducible, reviewable, and kind to future you.

Learning goals

By the end of this chapter you can:

- Write an addressing plan before deploy
- Apply consistent node and interface naming
- Structure a lab repo/directory for git
- Keep diagrams and inventories in sync
- Run a personal definition-of-done for every lab
- Avoid cross-lab contamination on one host

Why hygiene is a skill

Without hygiene	With hygiene
“Which 10.0.0.0/24 is this?”	Documented per-lab blocks
Works once after 40 hand edits	Destroy/deploy/verify loop
Cannot explain a failure	Journal + captures
Peer cannot run your lab	README + pins + scripts

Hygiene is not bureaucracy; it is how lab craft scales.

Addressing plans

Never invent IPs mid-SSH.

Template

```
# Addressing - lab triangle-ospf
```

```
| Node | Interface | Address | Notes |
|-----|-----|-----|-----|
| r1 | eth1 | 10.0.12.1/24 | to r2 |
| r2 | eth1 | 10.0.12.2/24 | to r1 |
| r1 | eth2 | 10.0.13.1/24 | to r3 |
| r3 | eth2 | 10.0.13.2/24 | to r1 |
| r2 | eth2 | 10.0.23.1/24 | to r3 |
| r3 | eth1 | 10.0.23.2/24 | to r2 |
| h1 | eth1 | 192.168.1.10/24 | gw 192.168.1.1 (r1 lo/lan) |
```

Loopbacks:

```
| Node | lo |
|-----|-----|
| r1 | 1.1.1.1/32 |
| r2 | 2.2.2.2/32 |
| r3 | 3.3.3.3/32 |
```

VLANs: n/a

ASNs: n/a

Mgmt: Containerlab default pool

Allocation strategy (personal lab supernets)

Pick a **lab supernet** you will not confuse with home LAN:

Use	Example block
Home LAN (real)	192.168.1.0/24 (do not reuse in labs if VPN splits break)
Lab underlay P2P	10.0.0.0/16 carved into /24 or /30
Lab customer VRFs	10.10.0.0/16
Loopbacks	10.255.0.0/24 as /32s
“Internet” sim	203.0.113.0/24 (documentation range) / 192.0.2.0/24

Documentation ranges (RFC 5737) are great for examples in prose; inside isolated labs private space is fine—just **write it down**.

/30 vs /31 vs /24 on links

Choice	When
/24	Beginners, lots of headroom, wasteful

Choice	When
/30	Classic P2P
/31	Efficient P2P (ensure both NOS support)

Be consistent within a lab.

Naming conventions

Lab name

```
name: l2-vlan-trunk # short, hyphenated, unique among active labs
```

Container names become `clab-<name>-<node>`.

Node names

Pattern	Examples
Role + number	r1, r2, sw1, h1
Role + site	edge-a, core-1
Avoid	test, node, foo, router

Interface names

Stick to what the image uses (`eth1`, `eth2`... for many Linux kinds). Document any breakout or rename.

Git branches / folders

```
labs/
  03-l2-vlan-trunk/
  04-static-triangle/
  05-ospf-single-area/
```

Numeric prefixes match book order when useful.

Directory layout (canonical)

```
labs/04-static-triangle/  
  README.md           # intent, pins, how to run  
  topology.clab.yml  
  addressing.md  
  journal.md  
  verify.sh  
  up.sh               # optional deploy wrapper  
  down.sh             # destroy --cleanup  
  config/  
    r1/  
    r2/  
    h1/  
  captures/  
  diagrams/  
    logical.png       # or .svg / .drawio
```

README skeleton

```
# Static triangle  
  
## Intent  
Three FRR routers, static routes, dual hosts ping.  
  
## Images  
- quay.io/frrouting/frr:10.2.1  
- alpine:3.20  
  
## Run  
sudo containerlab deploy -t topology.clab.yml  
./verify.sh  
sudo containerlab destroy -t topology.clab.yml --cleanup  
  
## Failure drills  
- shut transit link r1-r2  
- remove return route on r3
```

Git habits

```
git status
git add labs/04-static-triangle
git commit -m "lab: static triangle with verify.sh"
```

Commit:

- Topology YAML
- Configs
- addressing.md
- verify.sh
- journal highlights (not necessarily huge pcaps)

Usually ignore:

- Large pcaps (store sparse samples or regenerate)
- Ephemeral logs

```
# example snippet
labs/**/captures/*.pcap
!labs/**/captures/.gitkeep
```

Never commit

- Production credentials
- Private keys
- Proprietary images

Diagram habit

Minimum viable diagram set:

1. **Logical** — nodes and links
2. **Addressing** — subnets/VLAN IDs on links
3. **Failure** — which link you shut for the drill

ASCII is valid:

```
h1--r1-----r2--h2
  \      /
   \    /
    r3
```

Illustrated topology SVGs (self-contained light canvas) match this book's lab figures under `images/`; conceptual models may stay monochrome. See `images/README.md`.

Configuration hygiene

Do	Don't
Edit files under <code>config/</code>	Treasure unique vtysh-only state
Reload/restart from files	Assume memory is durable
Diff configs in git	Copy-paste from chat without review
Comment tricky policy lines	Leave “magic” prefix-lists

FRR example: comment intent

```
! block lab RFC1918 from leaking to "isp" peer
ip prefix-list LAB-PRIV seq 5 permit 10.0.0.0/8 le 32
```

verify.sh hygiene

- `set -euo pipefail`
- Explicit container names
- Timeouts on ping
- Exit non-zero on failure
- Print which check failed

```
#!/usr/bin/env bash
set -euo pipefail
PASS=0
fail() { echo "FAIL: $" ">&2; exit 1; }

docker exec clab-static-h1 ping -c 2 -W 1 10.2.0.10 \
|| fail "h1 cannot reach h2"
```



```
echo "OK: all checks passed"
```

Cross-lab contamination

Symptoms:

- Wrong SSH target (same hostname different lab)
- Stale docker networks
- IP confusion in your brain

Mitigations:

```
sudo containerlab inspect --all 2>/dev/null || true
docker ps --format 'table {{.Names}}\t{{.Status}}' | grep clab || true
# Only one "heavy" lab running when learning
sudo containerlab destroy -t old.clab.yml --cleanup
```

Use unique lab **name**: fields always.

Time hygiene

Put dates in journals; note Containerlab and image versions:

```
Date: 2026-08-04
containerlab: 0.xx.x
frr image: 10.2.1
kernel: $(uname -r)
```

Future breakage is often a version drift story.

Definition of done (lab)

- ☐ Intent one-liner in README
- ☐ addressing.md complete
- ☐ topology deploys clean on fresh destroy
- ☐ verify.sh green

- ☐ One failure drill documented
- ☐ Diagrams match YAML
- ☐ Secrets absent
- ☐ Image tags pinned
- ☐ Committed to git

Predict → observe → fix: hygiene drill

1. **Predict:** A peer can clone your lab folder and pass `verify.sh` without asking you questions.
2. **Observe:** Remove your personal aliases; follow only README.
3. **Fix:** Gaps become README bullets.
4. **Harden:** Add `up.sh/down.sh`.

```
#!/usr/bin/env bash
# up.sh
set -euo pipefail
sudo containerlab deploy -t topology.clab.yml
./verify.sh
```

```
#!/usr/bin/env bash
# down.sh
set -euo pipefail
sudo containerlab destroy -t topology.clab.yml --cleanup
```

Collaboration norms (optional)

If sharing labs:

- Prefer open images only
- Include expected verify output sample
- Note host requirements (RAM)
- License your configs clearly (e.g. MIT notes)

Summary

- Address first, deploy second
- Name labs and nodes for humans and `docker ps`
- Canonical directory: topology, configs, verify, journal, diagrams
- Git is part of networking practice
- One clean destroy/deploy is worth ten fragile demos
- Definition of done prevents “almost labs”

Part checkpoint: topology-as-code should feel like muscle memory. Next part enters **layer 2**—Ethernet, VLANs, loops, aggregation, and failure drills.

Networking Foundations

What Is Networking?

What Is Networking?

A network moves **data** between **endpoints** over **links**, using **protocols**—agreed rules for format, addressing, and behavior. Everything later in this book (VLANs, OSPF, VXLAN) is a specialization of that idea.

Learning goals

- Name the main building blocks of a network and what problem each solves
- Distinguish local delivery (same L2 domain) from routed delivery (across networks)
- Explain why we use layered protocols instead of one monolithic format
- Tie lab tools (Containerlab, `ip`, captures) to those ideas

Building blocks

Piece	Role	Lab example
Endpoint / host	Source or sink of traffic	Alpine container, your laptop
Interface	Attachment to a link	<code>eth0</code> , <code>eth1</code>
Link	Path between interfaces	Cable, veth pair, Wi-Fi
Switch / bridge	Forward within an L2 domain by MAC	Linux bridge, virtual switch
Router	Forward between L3 networks by IP	FRR node, Linux with forwarding
Protocol	Rules and message formats	Ethernet, IPv4, TCP, DNS

What problem networking solves

Without a network, processes on different machines cannot cooperate. Networking provides:

1. **Reachability** — a path exists (or is reported as unreachable)

2. **Naming / addressing** — who is who (MAC, IP, names via DNS)
3. **Multiplexing** — many conversations share links (ports, VLANs, queues)
4. **Reliability options** — best-effort (UDP/IP) vs recovered streams (TCP)
5. **Policy** — who may talk to whom, with what priority

Switching vs routing (first cut)

Same subnet / VLAN

Host A switch Host B
Frame uses MAC

Different subnets

Host A → gateway → ... → Host B
Packet uses IP; each hop rewrites L2

- **Switching (L2):** deliver a frame inside one broadcast domain; learn MACs; flood unknown unicasts.
- **Routing (L3):** deliver a packet across networks; each router decrements TTL and chooses a next hop.

You will refine both in the Layer 2 and Layer 3 parts. Foundations give vocabulary and host-level truth.

Protocols as contracts

A protocol answers:

- What bits mean (headers, fields)
- Who may send what next (state machines)
- How errors are signaled (ICMP, TCP RST, DHCP NAK)

Vendors invent CLIs; **RFCs and open implementations** define the contracts this book cares about.

Why labs beat slideware

Networks fail in partial, asymmetric, and time-dependent ways. A Containerlab topology lets you:

- Change **one** variable (shut a link, wrong mask, bad gateway)
- Observe **tables** (ARP, routes, MAC) and **packets** (tcpdump)
- Reset and try again

Observation toolkit (preview)

Tool	Use
<code>ip addr / ip route / ip neigh</code>	Host addressing and neighbors
<code>ping / traceroute / mtr</code>	Reachability and path hints
<code>ss / netstat</code>	Sockets and listeners
<code>tcpdump / tshark</code>	On-the-wire proof
Containerlab	Multi-node topologies as code

Checkpoint

Before deeper chapters, you should be able to point at a simple two-host lab and say: *these are endpoints, this is a link, delivery is L2 or L3 for this destination, and here is how I would prove it.*

Next

- [TCP/IP and the Layered Stack](#)

TCP/IP and the Layered Stack

TCP/IP and the Layered Stack

Networks are built as a **stack of layers**. Each layer solves one class of problem and offers a service to the layer above. This book uses the **TCP/IP** (Internet) model in day-to-day work; the **OSI** model remains a useful legend for vendor docs and troubleshooting language.

Learning goals

- Map problems and tools to the correct TCP/IP layer
- Translate common OSI layer numbers into TCP/IP practice
- Explain demultiplexing (how a host hands data up the stack)
- Avoid “wrong layer” diagnoses

TCP/IP model (working map)

Layer	Examples	Responsibility
Application	HTTP, DNS, SSH, DHCP, BGP (as app of TCP/UDP)	Meaning for programs; often client/server
Transport	TCP, UDP	Process-to-process; ports; reliability options
Internet	IPv4, IPv6, ICMP	Host-to-host logical addressing; routing; error signaling
Link / network access	Ethernet, Wi-Fi, PPP	Adjacent-node framing; media access

Some texts split “host-to-network” further; for labs, **Ethernet** + **ARP/ND** is the daily link reality.

OSI as a legend (not a religion)

OSI #	Name	Rough TCP/IP home
7	Application	Application
6	Presentation	Often folded into app (TLS, encodings)

OSI #	Name	Rough TCP/IP home
5	Session	Often folded into app/TCP
4	Transport	TCP/UDP
3	Network	IP / ICMP
2	Data link	Ethernet frames, switching
1	Physical	Cables, optics, signaling (usually out of scope here)

Practical rule: know *where the address lives* and *which header you are looking at in a capture*, not seven acronyms for a quiz.

What each layer “owns”

Link

- Local delivery between neighbors
- MAC addresses, Ethertype, FCS
- Switches forward here without caring about IP

Internet (IP)

- Global-ish logical addressing
- Best-effort packet forwarding
- TTL/hop limit; fragmentation (IPv4) or reliance on PMTU (IPv6)

Transport

- Which **process** on the host
- Ports; TCP connection state vs UDP datagrams

Application

- Protocol semantics (DNS query/response, HTTP methods)
- Often implements its own retries on top of UDP

Demultiplexing (up the stack)

On receive:

1. NIC / driver accepts a frame (MAC filter)
2. Ethertype → IP vs ARP vs other
3. IP protocol field → TCP / UDP / ICMP / ...
4. Port numbers → socket / application

```
Frame [Eth | IP | TCP | HTTP ]
      ^   ^   ^   ^
      |   |   |   |
      |   |   |   application
      |   |   transport demux (ports)
      |   internet demux (protocol #)
      link demux (Ethertype)
```

Layered troubleshooting

Symptom	Look first at
No link light / veth missing	Link / lab cabling
ARP incomplete	Link + IP on-link config
Ping fails, ARP OK	Routing, firewall, ICMP policy
TCP SYN no reply	ACL, listening socket, path, RST
TLS fails after TCP	Application / certificates

Encapsulation (preview)

Each layer **adds a header** when sending and **strips it** when receiving. Details: [Packets](#), [Frames](#), [Encapsulation](#).

Checkpoint

Given a failure description, name the **layer** you will inspect first and **one command or table** that proves or disproves that theory.

Next

- [Packets](#), [Frames](#), [Encapsulation](#)

Packets, Frames, Encapsulation

Packets, Frames, Encapsulation

Data is **wrapped** in headers as it descends the stack and **unwrapped** on the way up. Clear names prevent confusion in captures and design talks.

Learning goals

- Use correct terms: data, segment/datagram, packet, frame
- Sketch encapsulation for a simple TCP and UDP flow
- Read a tcpdump line and identify outer headers
- Explain how tunnels add outer encapsulations

Vocabulary

Term	Layer (typical)	Meaning
Data / payload	Application	What the app cares about
Segment	TCP	TCP header + payload
Datagram	UDP (and sometimes IP)	UDP header + payload
Packet	IP	IP header + payload
Frame	Link	Ethernet (or other) header + payload + FCS

People say “packet” loosely; in precise talk, **frame** is L2 and **packet** is L3.

Encapsulation path (send)

1. Application writes bytes
2. Transport adds **src/dst ports** (TCP adds seq/ack/flags)
3. IP adds **src/dst IP**, TTL, protocol
4. Ethernet adds **src/dst MAC**, Ethertype

5. Bits go on the wire (or veth)

```
Application data
+-----+
| TCP/UDP header |
+-----+
| IP header       |
+-----+
| Ethernet header |
+-----+
```

Decapsulation path (receive)

Reverse order: FCS check → MAC → Ethertype → IP → protocol → ports → app.

Maximum sizes and MTU

- **MTU**: max IP packet size a link will accept (commonly 1500 on Ethernet)
- Larger packets must fragment (IPv4) or fail with Packet Too Big (IPv6 + PMTUD)
- Extra encapsulation (VLAN tag, VXLAN, IPsec) **reduces** effective payload size

Details and labs: Layer 3 ICMP/PMTU chapter later; foundations need the idea.

Captures: reading the onion

```
# On a Linux lab node
tcpdump -ni eth1 -vv
tcpdump -ni eth1 icmp
tcpdump -ni eth1 'tcp port 22'
```

What you see first is the **outer** header. Inner headers appear after decap or when capturing on the right interface.

Tunnels and overlays (preview)

Overlays wrap an **inner** packet in an **outer** IP/UDP (or GRE) packet. Underlay routers only need to forward the outer packet. Full treatment: Overlays part.

```
[ Outer Eth | Outer IP | Outer UDP | VXLAN | Inner Eth | Inner IP | ... ]
```

Lab sketch (two nodes)

1. Deploy two Linux nodes on one L2 segment
2. Capture while `ping` runs — ICMP inside IP inside Ethernet
3. Capture `nc` or `curl` — TCP three-way handshake fields

Checkpoint

Draw encapsulation for **DNS over UDP** and **SSH over TCP** from host A to B on the same Ethernet. Label every header.

Next

- [Link Layer and Ethernet](#)

Link Layer and Ethernet

Link Layer and Ethernet

The **link layer** moves frames between neighbors on a shared medium or point-to-point link. In almost every lab in this book, that means **Ethernet** (physical or virtual).

Learning goals

- Describe Ethernet frame fields used in daily troubleshooting
- Explain unicast, multicast, broadcast at L2
- Contrast hubs (obsolete), switches, and bridges
- Preview MAC learning (full drills in Layer 2 part)

Why link layer matters

IP cannot deliver alone: on multi-access Ethernet, the next hop is identified by **MAC**. Wrong VLAN, wrong MAC, or broken L2 path makes “routing” look broken when the problem is local.

Ethernet frame (simplified)

Field	Role
Dest MAC	Who should accept the frame
Src MAC	Who sent it (learning key on switches)
Ethertype / length	Payload type (e.g. 0x0800 IPv4, 0x86DD IPv6, 0x0806 ARP)
Payload	IP packet, ARP, etc.
FCS	Error detection

802.1Q VLAN tags insert **TPID** + **TCI** (including VLAN ID) between SA and Ethertype on trunks—details in VLANs chapter.

MAC addresses

- 48-bit; written **aa:bb:cc:dd:ee:ff**
- Universally administered vs locally administered
- **Broadcast** **ff:ff:ff:ff:ff:ff**
- Multicast: group bit set

```
ip -br link
ip link show eth1
```

Broadcast domain vs collision domain

- **Collision domain:** historically shared half-duplex media; switched full-duplex Ethernet largely eliminates classic collisions
- **Broadcast domain:** extent of L2 broadcast/unknown flood—usually one VLAN

Routers **stop** L2 broadcasts; switches do not.

Switch vs router (again)

Device	Forwarding key	Changes MAC?	Changes IP?
L2 switch	Dest MAC + FDB	No (transparent)	No
Router	Dest IP + FIB	Yes (new Ethernet hop)	No (until NAT)

MAC learning (preview)

1. Learn **src MAC** → **port** on ingress
2. If dest MAC known, forward to that port
3. If unknown unicast, **flood** in the VLAN
4. Broadcast/multicast: flood per rules

Deep labs: [Ethernet & MAC Learning](#).

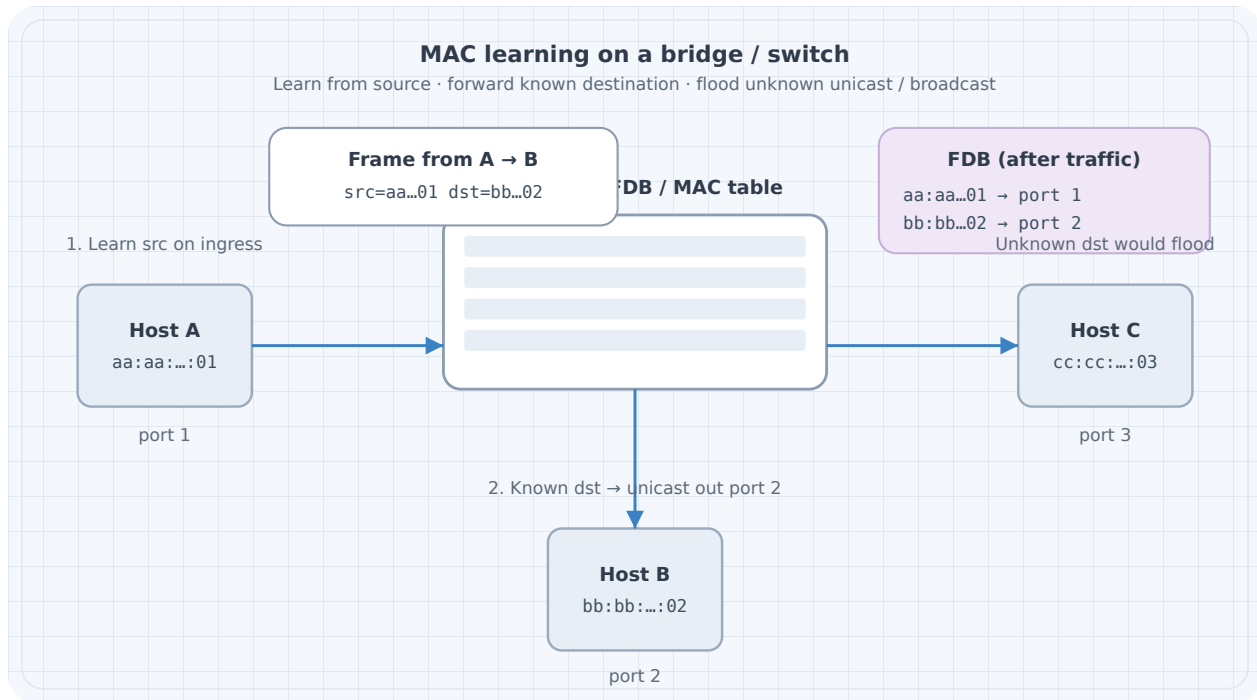


Figure 1: MAC learning

Virtual Ethernet in labs

Containerlab connects nodes with **veth pairs** and bridges—same Ethernet model, software implementation:

```
ip link
bridge link # if bridge-utils / iproute2 bridge
```

Common L2 failures (foundations list)

Symptom	Typical cause
Incomplete ARP	Wrong subnet, wrong VLAN, silent host
Flapping MAC	Loops or dual-homing without design
No frame at all	Down interface, wrong cable/veth, admin down

Checkpoint

Explain what a switch does when it receives a frame with an **unknown** destination MAC, and what it does with the **source** MAC.

Next

- [IPv4 Addressing and Subnetting](#)

IPv4 Addressing and Subnetting

IPv4 Addressing and Subnetting

IPv4 addressing is the daily language of lab design: **prefixes**, **masks**, **subnets**, and **who is on-link**. Master this before multi-router labs.

Learning goals

- Read and write CIDR prefixes (**a.b.c.d/nn**) fluently
- Convert between prefix length and dotted masks
- Design non-overlapping lab subnets and point-to-point links
- Identify special-use and documentation ranges
- Configure addresses on Linux and verify with **ip**

IPv4 structure

- **32 bits**, usually dotted decimal (four 0–255 octets)
- **Prefix length** /N = number of network bits; host bits = 32 - N
- **Network address**: host bits zero
- **Broadcast address** (classical Ethernet/IPv4): host bits one (except special cases like /31)

Prefix	Mask	Classical usable hosts	Common use
/32	255.255.255.255	1	Host route, loopback
/31	255.255.255.254	2	Point-to-point (RFC 3021)
/30	255.255.255.252	2	Classic P2P
/29	255.255.255.248	6	Small segment
/24	255.255.255.0	254	Common LAN
/16	255.255.0.0	many	Summaries / big labs
/8	255.0.0.0	huge	Rare in labs

CIDR and masks

```
192.168.10.0/24
network: 192.168.10.0
mask:    255.255.255.0
hosts:   192.168.10.1 - 192.168.10.254
broadcast: 192.168.10.255
```

```
# Linux: add address
ip addr add 192.168.10.10/24 dev eth1
ip -br addr
```

On-link decision

If the destination is inside a **connected** prefix, the host resolves L2 (ARP) on that interface. Otherwise it looks up a **route** (often default via gateway).

```
ip route get 192.168.10.50
ip route get 8.8.8.8
```

Private and documentation ranges

Range	Role
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private (RFC 1918)—default lab space
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Documentation (TEST-NET)—safe in examples
127.0.0.0/8	Loopback
169.254.0.0/16	Link-local (APIPA)
224.0.0.0/4	Multicast

Lab hygiene: pick a documented plan (10.20.0.0/16 etc.) and never invent overlapping subnets “for now.”

Subnetting for multi-tier labs

Example allocation from 10.20.0.0/16:

Use	Block
Access / users A	10.20.1.0/24
Access / users B	10.20.2.0/24

Use	Block
Servers	10.20.10.0/24
P2P core	10.20.255.0/24 carved as /31 or /30
Loopbacks	10.20.254.0/24 as /32s

Rules:

1. No accidental overlap
2. Leave growth room
3. Write the plan **before** deploy
4. Align future summaries with real topology

Point-to-point links

Style	Example	Notes
/31	10.20.255.0/31 → .0 and .1	Preferred in modern designs
/30	10.20.255.0/30 → .1 and .2 usable	Classic textbooks

Summarization (intro)

Advertising 10.20.0.0/22 for four /24s only works if you **own** the whole /22. Bad summaries **blackhole** traffic for space you do not have.

Action	Risk
Summary too broad	Blackholes
No aggregation ever	Table growth later (IGP/BGP scale)

Layer 3 part continues summarization drills; foundations need the danger signal.

VLSM

Variable Length Subnet Masking: different prefix lengths in one design (/24 LAN + /31 core). Required for efficient labs—do not force everything to /24.

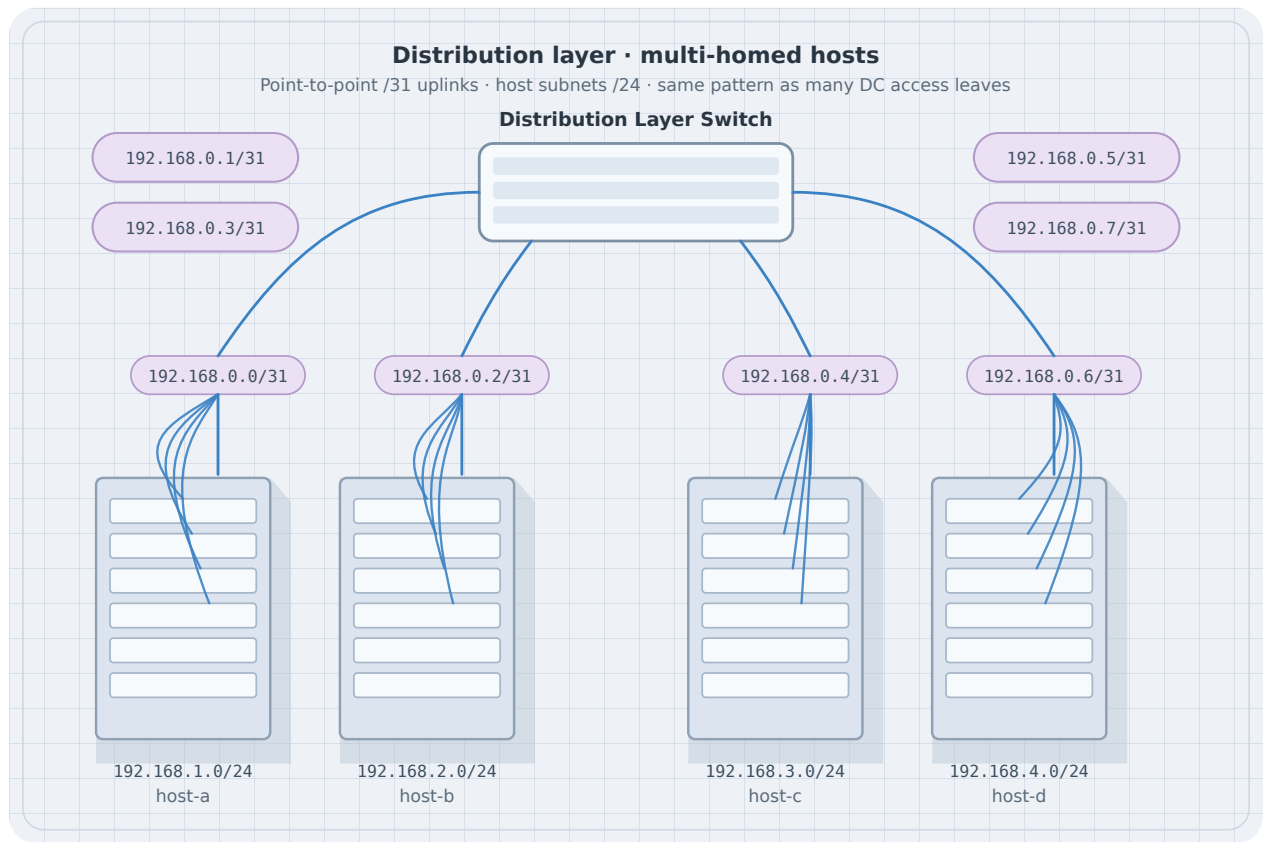


Figure 1: Dist switch multi-home

Host configuration checklist

1. Address + prefix on the correct interface
2. Up state (`ip link set eth1 up`)
3. Gateway if leaving the subnet
4. DNS only if apps need names (separate from pure L3 reachability)

```
ip addr add 10.20.1.10/24 dev eth1
ip link set eth1 up
ip route add default via 10.20.1.1
ip -br a
ip route
```

Worked subnet: /26 from a /24

Parent: 192.168.10.0/24 (256 addresses). Split into four /26s (64 addresses each):

Block	Network	Usable hosts	Broadcast
0	192.168.10.0/26	.1–.62	.63
1	192.168.10.64/26	.65–.126	.127
2	192.168.10.128/26	.129–.190	.191
3	192.168.10.192/26	.193–.254	.255

Host bits: /26 → 6 host bits → $2^6 = 64$ addresses → 62 usable (reserve network + broadcast) on Ethernet-style LANs. Point-to-point often uses /31 (RFC 3021) or /30.

Longest-prefix match: if a router has both 10.0.0.0/16 and 10.1.2.0/24, a packet to 10.1.2.5 uses the /24. More specific wins—plan summarization so you do not punch accidental holes.

Private vs public (RFC 1918)

Range	Typical use
10.0.0.0/8	Large private / DC
172.16.0.0/12	Mid private
192.168.0.0/16	Home/lab LANs
100.64.0.0/10	CGNAT (ISP shared) — not “your” private LAN plan

Lab rule: pick one private block for the whole topology and **document** the plan before cabling Containerlab.

Paper drills

1. Network, first host, last host, broadcast for 172.16.5.0/26
2. How many /24s fit in 10.0.0.0/16?
3. Can 10.0.1.0/24 and 10.0.1.128/25 coexist without overlap?
4. From 10.50.0.0/16, carve three /24s and one /28 for a mgmt VLAN without overlap

Checkpoint

Design addresses for: two LANs (/24 each), one router-router link (/31), two router loopbacks (/32). No overlaps; list every interface IP.

Next

- [IPv6 Essentials](#)

IPv6 Essentials

IPv6 Essentials

IPv6 is not “IPv4 with longer addresses only.” Foundations need **address types**, **prefix notation**, **link-local**, and enough config to run dual-stack labs.

Learning goals

- Read compressed IPv6 addresses and /**prefix** lengths
- Distinguish link-local, unique local, global unicast
- Configure a static IPv6 address on Linux
- Know what Neighbor Discovery replaces (ARP, parts of ICMP discovery)

Why IPv6 in this book

- Dual-stack labs mirror real networks
- ND and SLAAC appear in operations
- Many overlays and DC fabrics are IPv6-capable or IPv6-underlay

You do not need to memorize every transition technology here.

Address format

- **128 bits**, eight hextets, e.g. 2001:db8:1:2:3:4:5:6
- Compress longest zero run with :: (once)
- Prefix: 2001:db8:1::/64 (common LAN length is /**64**)

Documentation prefix: **2001:db8::/32** (use in examples).

Address kinds (must know)

Kind	Pattern / notes	Role
Link-local	fe80::/10	On-link only; required on every interface
Unique local (ULA)	fc00::/7 (commonly fd00::/8)	Private-like lab space
Global unicast	2000::/3 (in practice)	Internet-routable space
Multicast	ff00::/8	One-to-many
Loopback	::1	Local host
Unspecified	::	“Any” / not set

```
ip -6 addr
ip -6 route
```

Interface identifiers and /64

LAN segments almost always use /64. Host bits can come from:

- Manual config
- SLAAC (router advertisements)
- DHCPv6 (where used)

EUI-64 and privacy extensions exist; labs often set **static** addresses for clarity.

Linux static example

```
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip -6 route add default via 2001:db8:1::1 dev eth1
ping -6 2001:db8:1::1
```

Neighbor Discovery (preview)

IPv6 uses **ND** (ICMPv6) instead of ARP:

- Neighbor Solicitation / Advertisement
- Router Solicitation / Advertisement

Details: [ARP, ND, and Local Resolution](#).

Dual-stack thinking

Stack	Failure isolation
IPv4 works, IPv6 fails	v6 route, RA, ND, firewall ip6tables/nft
Prefer AAAA vs A	Happy Eyeballs / app order

Test **both** when dual-stack is in scope:

```
ping -4 <host>
ping -6 <host>
```

Checkpoint

Write a dual-stack plan for one LAN: IPv4 /24, IPv6 /64 from 2001:db8:10::/64, gateway ::1 / .1, two hosts.

Next

- [ARP, ND, and Local Resolution](#)

ARP, ND, and Local Resolution

ARP, ND, and Local Resolution

On multi-access links, IP needs a **local** next-hop mapping: IPv4 uses **ARP**; IPv6 uses **Neighbor Discovery (ND)**. If resolution fails, routing never gets a chance.

Learning goals

- Explain when ARP/ND runs (on-link destinations and gateways)
- Read and clear neighbor tables on Linux
- Spot incomplete entries and wrong-subnet mistakes
- Capture ARP or NS/NA in a lab

When local resolution happens

1. Host decides destination is **on-link** (connected route), or
2. Host needs the **gateway's MAC** for an off-link destination

Then: resolve next-hop IP → MAC; build Ethernet header; send.

ARP (IPv4)

Message	Role
Who-has (request)	Broadcast: “Who has IP X? Tell Y”
Is-at (reply)	Unicast: “X is at MAC M”

```
ip neigh show
ip neigh show dev eth1
ping -c1 192.168.10.1
ip neigh show
# force re-resolve
ip neigh flush dev eth1
```

Incomplete / failed ARP

Cause	What you see
Target down	Incomplete, then failed
Wrong VLAN / segment	No reply
Wrong mask (thinking on-link when not)	ARP for remote IPs on LAN
Firewall drop	Silence

Neighbor Discovery (IPv6)

Message (ICMPv6)	Role
Neighbor Solicitation	Resolve or probe neighbor
Neighbor Advertisement	Provide or confirm MAC
Router Solicitation / Advertisement	Find routers; SLAAC info

```
ip -6 neigh show
ping -6 2001:db8:1::1
```

Gratuitous ARP and updates

Hosts may announce their own IP/MAC (gratuitous ARP) on address change. Useful for failover designs later (first-hop redundancy).

Proxy ARP (awareness)

A router answering ARP for non-local addresses can hide topology mistakes—or create blackholes. Prefer **correct masks and routes** over proxy ARP as a habit.

Lab sketch

1. Two hosts + gateway on /24
2. `tcpdump -ni eth1 arp` while first ping runs
3. Wrong mask on host: watch ARP for remote destinations

Checkpoint

Host 10.0.0.10/24, gateway 10.0.0.1. Destination 10.0.1.10. Which IP is ARP'd for? Why?

Next

- [ICMP and Path Diagnostics](#)

ICMP and Path Diagnostics

ICMP and Path Diagnostics

ICMP (and ICMPv6) signal errors and support diagnostics. `ping` and `traceroute` are applications of those messages—not separate magic protocols.

Learning goals

- Map common ICMP types to meaning
- Use ping/traceroute with clear interpretation limits
- Understand TTL expiry and “why traceroute hops appear”
- Preview Path MTU Discovery

ICMPv4 types (operator set)

Type	Name	Typical meaning
0 / 8	Echo reply / request	Ping
3	Destination unreachable	No route, port unreachable, admin prohibited, ...
11	Time exceeded	TTL expired (traceroute hops)

Codes refine type 3 (network unreachable, host unreachable, etc.).

ICMPv6

Similar roles: echo, destination unreachable, **packet too big**, time exceeded, ND messages (see previous chapter).

```
ping -c3 192.0.2.1
ping -6 -c3 2001:db8::1
```

Interpreting ping

Result	Possible meaning
Reply	Forward path + return path + ICMP allowed
Timeout	Drop, filter, wrong route, silent host
Unreachable	Explicit ICMP error from some node

Ping success **application success**. TCP port filters can still block SSH/HTTP.

Traceroute mechanics (conceptual)

1. Send probes with TTL=1, 2, 3, ...
2. Each hop that decrements TTL to 0 should send **time exceeded**
3. Destination eventually responds (port unreachable / echo)

```
traceroute 192.0.2.1
# or
mtr -r -c 5 192.0.2.1
```

Limits: load balancers, ICMP rate limits, firewalls that hide hops, asymmetric return paths.

Path MTU Discovery (preview)

- Senders prefer **not** to fragment (IPv6: no in-path fragmentation)
- Hop returns **Packet Too Big** / frag-needed if DF set and MTU smaller
- Broken PMTUD → hangs for large transfers while small pings work

Deep labs: Layer 3 ICMP & Path MTU chapter.

Filters and policy

Many networks rate-limit or block ICMP. That improves some attacks and **hurts** diagnostics and PMTUD. Lab hygiene: know your nftables/iptables ICMP rules.

```
# Example observability only-do not paste blindly into production
nft list ruleset
iptables -L -n
```

Lab sketch

1. Three-node line (A—B—C)
2. Ping A→C; traceroute A→C
3. ACL drop ICMP on B: compare symptoms

Checkpoint

Why can `ping` work while `curl http://...` fails? Why can `traceroute` show * * * on middle hops while the destination still answers?

Next

- [Transport: TCP and UDP](#)

Transport: TCP and UDP

Transport: TCP and UDP

The **transport layer** delivers data to the correct **process** on a host using **ports**, with different reliability and connection models.

Learning goals

- Explain 5-tuples and port roles
- Contrast TCP and UDP services
- Read basic TCP handshake and teardown signals
- Use `ss` to inspect listeners and connections

Ports and sockets

- Port: 16-bit number (0–65535)
- **Well-known** ports (e.g. 22 SSH, 53 DNS, 80 HTTP) vs **ephemeral** client ports
- Socket: local address + port (+ protocol), often with peer

5-tuple: protocol, src IP, src port, dst IP, dst port — identifies a conversation.

```
ss -tulpn
ss -tan
ss -uan
```

UDP

Property	UDP
Connection	None
Reliability	None (app may add)
Ordering	Not guaranteed
Header	Lightweight
Use cases	DNS, DHCP, many tunnels, telemetry, real-time

```
# one-shot listener / client ideas
nc -u -l -p 9999
nc -u <peer> 9999
```

TCP

Property	TCP
Connection	Yes (stateful)
Reliability	Ack, retransmit
Ordering	Byte stream reassembly
Flow / congestion control	Yes
Use cases	SSH, HTTP, BGP, config APIs

Three-way handshake (must know)

1. Client → Server: **SYN**
2. Server → Client: **SYN-ACK**
3. Client → Server: **ACK**

Then data can flow. Teardown uses **FIN/ACK** (or **RST** for abort).

```
tcpdump -ni eth1 'tcp port 22'
```

States (operator view)

State	Meaning
LISTEN	Waiting for connections
SYN-SENT / SYN-RECV	Handshake in progress
ESTABLISHED	Data transfer
TIME-WAIT	Waiting to be sure old segments die
CLOSE-WAIT / FIN-WAIT	Teardown in progress

TCP vs UDP selection

Need	Prefer
Reliable byte stream	TCP

Need	Prefer
Simple request/response, app handles loss	UDP
Multicast / lightweight tunnel outer	Often UDP
Head-of-line sensitivity	Design carefully (HTTP/2 vs QUIC later topics)

NAT and transports (preview)

NATS track **transport** state (especially TCP). UDP mappings time out differently. Edge/NAT chapter expands this.

Security basics

- **Do not** expose admin TCP ports to untrusted networks without policy
- Spoofing and scan noise are normal on open nets
- TLS usually rides **on TCP** (or QUIC on UDP)—app layer crypto

TCP state machine (operator view)

```
CLOSED → SYN_SENT → ESTABLISHED → FIN_WAIT_* → TIME_WAIT → CLOSED
(listen) LISTEN → SYN_RCVD → ESTABLISHED → CLOSE_WAIT → LAST_ACK → CLOSED
```

State you see (<code>ss</code> / <code>netstat</code>)	Meaning
LISTEN	Server bound, waiting for SYN
ESTAB / ESTABLISHED	Data can flow
TIME-WAIT	Local side closed; holding tuple so delayed segments die (normal after busy services)
CLOSE-WAIT	Peer closed; your app must close its side (often a bug if stuck)
SYN-SENT	Client waiting for SYN-ACK (firewall/blackhole if stuck)

Flow control vs congestion control: window advertisements limit *receiver* buffer pressure; congestion control (slow start, Reno/CUBIC, ...) limits *network* load. Both shrink effective throughput; do not “tune sysctls” before measuring.

UDP: no handshake, no stream—each datagram is independent. Good for DNS, DHCP, QUIC’s outer path, and lab pings of custom apps. Reliability, if needed, is **your** application’s job.

Three-way handshake and teardown (why captures look busy)

1. Client → Server: SYN (seq=c)
2. Server → Client: SYN-ACK (seq=s, ack=c+1)
3. Client → Server: ACK (ack=s+1)

Teardown is often four segments (FIN/ACK each way) or RST on abort. In labs, filter `tcp.flags.syn==1` or `tcp.flags.fin==1` or `tcp.flags.reset==1` in Wireshark/tshark to see only control.

Lab sketch

1. `ss -tulpn` on a node before/after starting SSH
2. Capture handshake for SSH (`tcpdump -ni any port 22`)
3. `nc` UDP echo between two lab hosts
4. Force a stuck CLOSE-WAIT by killing only one side of a netcat TCP session; observe with `ss -tan`
5. Note TIME-WAIT count after many short connections (`ss -tan state time-wait | wc -l`)

Checkpoint

A client opens HTTPS to a server. Name protocol numbers/fields for: Ethernet type, IP protocol, TCP ports (client ephemeral + 443), and where TLS sits.

Next

- [Common Application Protocols](#)

Common Application Protocols

Common Application Protocols

Application protocols give **meaning** to payloads. Operators still need a working vocabulary: DNS, DHCP, HTTP, SSH, and TLS appear in almost every network design.

Learning goals

- Place common apps on TCP/UDP and well-known ports
- Describe DNS resolution flow at a high level
- Describe DHCP DORA and where relays fit
- Separate HTTP from TLS and from TCP failures
- Know what SSH needs from the network

Map (operator cheat sheet)

Protocol	Transport	Port(s)	Role
DNS	UDP/TCP	53	Name → address (and more)
DHCP	UDP	67/68	IPv4 host config
DHCPv6	UDP	546/547	IPv6 host config
HTTP	TCP	80	Web (cleartext)
HTTPS	TCP	443	HTTP over TLS
SSH	TCP	22	Secure shell / SFTP
NTP	UDP	123	Time
BGP	TCP	179	Routing (control plane app)
SNMP	UDP	161/162	Legacy management

DNS

Roles

Role	Function
Stub resolver	On the host; asks a recursive resolver
Recursive resolver	Walks the tree for the answer
Authoritative server	Holds zone data

Query types (common)

Type	Meaning
A	IPv4 address
AAAA	IPv6 address
CNAME	Alias
MX / TXT / SRV	Mail, text, services

```
# Linux lab host examples
getent hosts example.com
# if dig/nslookup installed:
# dig +short A example.com
# dig +short AAAA example.com
```

Failure patterns

Symptom	Check
Names fail, IPs work	DNS config, UDP/53 filters, resolver reachability
Slow apps	Timeouts to dead resolvers
Wrong address	Cache, split DNS, hosts file

DHCP (IPv4)

DORA: Discover → Offer → Request → Ack

- Client uses broadcasts initially (needs L2 reachability to server or **relay**)
- Server assigns IP, mask, gateway, DNS, lease time

```
Client --L2 domain-- DHCP server
Client --L2-- Relay --L3-- DHCP server
```

Edge designs with helpers/relays: later Edge & services part.

HTTP and HTTPS

HTTP: App TCP:80 Server
HTTPS: App TCP:443 TLS HTTP Server

Failure	Layer to inspect
TCP SYN timeout	Path, ACL, listen socket
TCP up, TLS error	Certificates, SNI, protocol mismatch
TLS up, HTTP 5xx	Application

```
curl -v http://192.0.2.10/  
curl -vk https://192.0.2.10/
```

SSH

- TCP/22 (by default)
- Needs bidirectional path and allowed security policy
- Used heavily for lab and production management

```
ssh user@192.0.2.10  
ss -tlnp | grep 22
```

TLS in one paragraph

TLS provides encryption and authentication **above** TCP (classically). Network engineers care when: MTU/PMTUD breaks large records, middleboxes interfere, or certificates expire. Crypto suite deep-dives are optional here.

NTP and time

Broken time breaks auth (Kerberos, cert validation) and log correlation. Lab VMs often need chrony/ntp or host sync.

Lab sketch

1. DNS: break `/etc/resolv.conf`; compare `ping` by name vs by IP
2. HTTP: capture TCP/80 three-way + first GET
3. SSH: confirm LISTEN on 22; ACL drop and observe client timeout

Checkpoint

User says “the network is down” but `ping` by IP works and `curl` by name fails. List the top three protocol checks in order.

Next

- [Host Forwarding and Routing Intro](#)

Host Forwarding and Routing Intro

Host Forwarding and Routing Intro

Every host is a mini-router for its **own** traffic: connected routes, default gateway, and policy later. This chapter is host-centric; multi-router IGP/BGP comes in later parts.

Learning goals

- Read a Linux routing table
- Explain longest-prefix match (LPM)
- Configure a default route and a static host route
- Distinguish RIB vs what `ip route get` uses
- Enable IP forwarding on a Linux router node (preview)

Connected vs remote

Destination	Host behavior
In a connected prefix	ARP/ND on that interface; no gateway
Not connected	Lookup route → next hop → ARP/ND for next hop

Linux route table essentials

```
ip route
ip -6 route
ip route get 1.1.1.1
ip route get 192.168.10.50
```

Common entry types:

Entry	Meaning
default via G dev IF	Default gateway
P.P.P.P/nn dev IF	Connected or direct
H via G	Host route (/32)

Entry	Meaning
metric / proto	Preference and source (dhcp, static, ...)

Longest prefix match

More specific prefixes win:

```
0.0.0.0/0      via 10.0.0.1
10.0.0.0/8     via 10.0.0.2
10.1.1.0/24    dev eth2
```

Destination 10.1.1.5 → **eth2** connected.

Destination 10.2.0.1 → via 10.0.0.2.

Destination 8.8.8.8 → default via 10.0.0.1.

Default gateway

Most endpoints need one:

```
ip route add default via 10.20.1.1 dev eth1
```

Without it, on-link works and off-link fails—classic “I can ping the gateway but not the internet.”

Static routes (host or simple router)

```
ip route add 10.30.0.0/16 via 10.20.1.2
ip route add 192.0.2.10/32 via 10.20.1.3
```

Multi-router static designs and floating statics: Layer 3 static routing labs.

IP forwarding (Linux as router)

```
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
sysctl -w net.ipv6.conf.all.forwarding=1
```

With forwarding on, Linux can move packets between interfaces per the FIB—foundation of FRR data plane coexistence in labs.

Policy routing (awareness)

Multiple tables and `ip rule` exist for VRFs and advanced designs. Foundations: know they exist; do not start there.

Firewall vs routing

A **correct route** with a **drop rule** still fails. Always separate:

1. Is there a route? (`ip route get`)
2. Does L2 resolve? (`ip neigh`)
3. Does policy allow? (nft/iptables, security groups)

Lab sketch

1. Two subnets, one Linux router
2. Hosts with defaults pointing at the router
3. `ip route get` on host and router for east-west traffic
4. Break default on host; compare symptoms

Checkpoint

Host 10.1.1.10/24, gateway 10.1.1.1. Route table has only the connected /24. What works? What fails? What single command fixes off-subnet reachability?

Next

- [Control, Data, and Management Planes](#)

Control, Data, and Management Planes

Control, Data, and Management Planes

Separate **how the network decides**, **how it forwards**, and **how you operate it**. This split keeps troubleshooting honest from first lab to fabric design.

Learning goals

- Define data, control, and management planes with examples
- Classify failures into the right plane
- Pick evidence (tables vs protocols vs access path) per plane
- Connect planes to tools used in this book

Data plane

Forwards user and transit packets using programmed tables.

Examples	Evidence
Ethernet FDB / MAC table	<code>bridge fdb</code> , switch MAC table
IP FIB / routes used for forward	<code>ip route get</code> , CEF/FIB show
ACL counters / nft hits	Counters increment on matches
Queues / drops	Interface drop stats

Failures: blackhole route, wrong next hop, MTU drop, ACL drop, congestion.

Control plane

Builds and maintains the information data plane uses.

Examples	Evidence
ARP/ND	<code>ip neigh</code>
OSPF/BGP adjacencies	<code>protocol show</code> commands
MAC learning	dynamic FDB entries
Route computation	RIB entries, SPF/BGP path choice

Failures: adjacency down, authentication mismatch, policy filter, CPU overload, CoPP drops.

Management plane

How humans and automation configure and observe devices.

Examples	Evidence
SSH, HTTPS API, NETCONF	Reachability to mgmt VRF/IP
Telemetry, syslog, SNMP	Streams arrive at collectors
Console / OOB	Still works when in-band dies

Failures: ACL on vty, expired credentials, mgmt routing broken, “network down” that is only mgmt path down.

Three planes of a network device

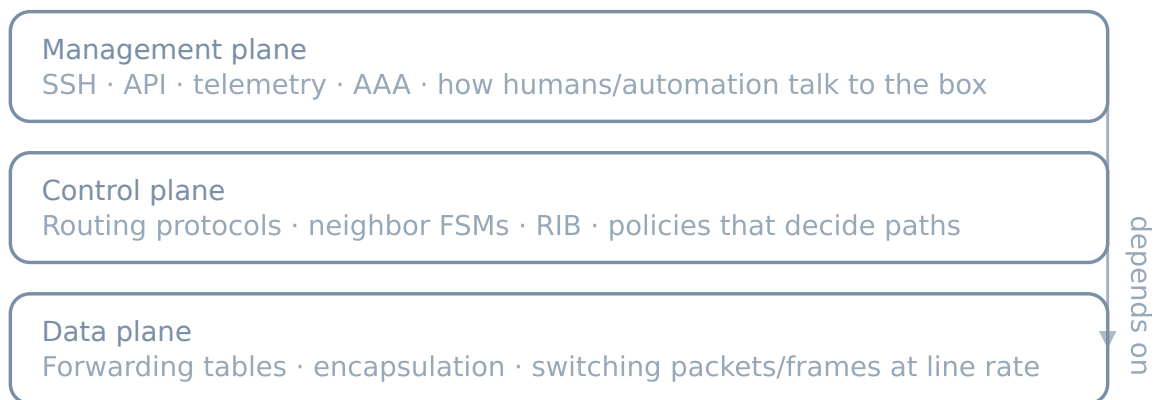


Figure 1: Planes

Interaction

```
Management → configures → Control
Control     → programs   → Data
Data        → forwards   user packets
```

A BGP session (control) programs the FIB (data). You establish BGP over TCP (data plane for the control session—meta but real).

Troubleshooting order (habit)

1. **Management:** Can I access the device?
2. **Data:** Does `ping/curl` of interest work? What does `ip route get` say?
3. **Control:** Are neighbors up? Are routes present in the RIB?
4. Reconcile: RIB has route but FIB/policy drops?

Lab habit

When something fails, write one sentence: *“I think this is a **data** / **control** / **management** plane fault because ...; I will prove it with ...”*

Checkpoint

SSH to a router fails, but the router still forwards traffic between two lab hosts. Which plane is most likely broken for you, and which is still working?

Next

Depth continues in [Ethernet & MAC Learning](#). Host and Containerlab skills remain in [Lab platform](#).

Layer 2

Ethernet & MAC Learning

Ethernet & MAC Learning

Layer 2 is where frames move inside a broadcast domain. Before VLANs and spanning tree, master **Ethernet addressing, flooding, and MAC learning**—the behaviors every switch and bridge still implements.

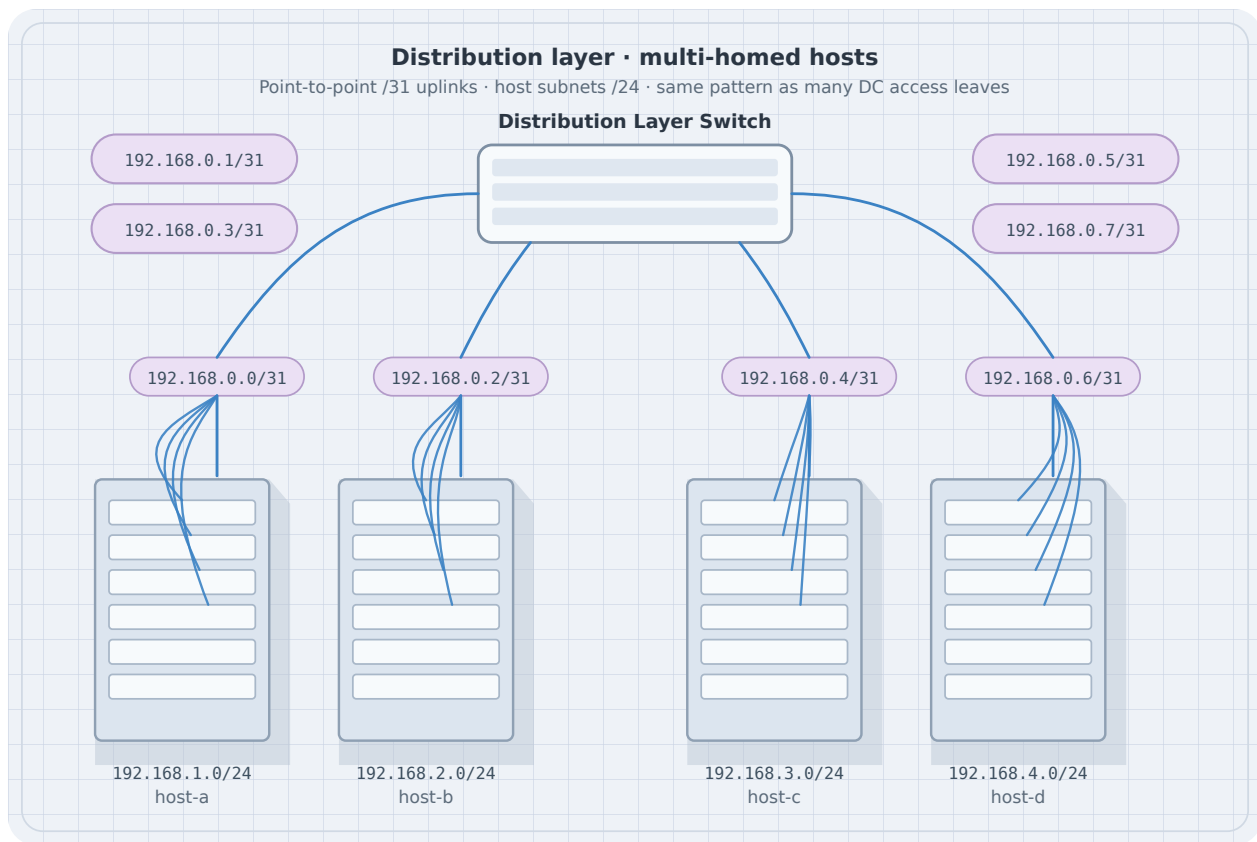


Figure 1: Distribution switch multi-home topology

Learning goals

By the end of this chapter you can:

- Explain source learning and unknown unicast flooding
- Read and interpret a MAC/FDB table

- Predict ARP’s role binding IP to MAC on-link
- Build a small bridged lab with Linux or Containerlab
- Diagnose “same subnet but no ping” with neigh + FDB + capture

Concepts

Frame essentials

An Ethernet frame carries:

- Destination MAC
- Source MAC
- Ethertype (e.g. IPv4 0x0800, ARP 0x0806, IPv6 0x86DD, 802.1Q tag)
- Payload
- FCS (hardware)

Switches forward using **destination MAC** and the VLAN context (next chapter). They **learn** from **source MAC**.

Learning and flooding

Event	Switch action
Frame arrives with source MAC S on port P	Learn $S \rightarrow P$ (for that VLAN)
Destination MAC known	Forward out that port only (not ingress)
Destination unknown unicast	Flood within VLAN
Broadcast (ff:ff:ff:ff:ff:ff)	Flood within VLAN
Multicast	Flood or constrain (IGMP snooping later world)

Stale entries age out (aging timer). Moves update the port mapping—important when you relocate a VM/container.

MAC addresses

- 48-bit, usually written aa:bb:cc:dd:ee:ff
- Locally administered vs universal bits exist; labs often show container random MACs

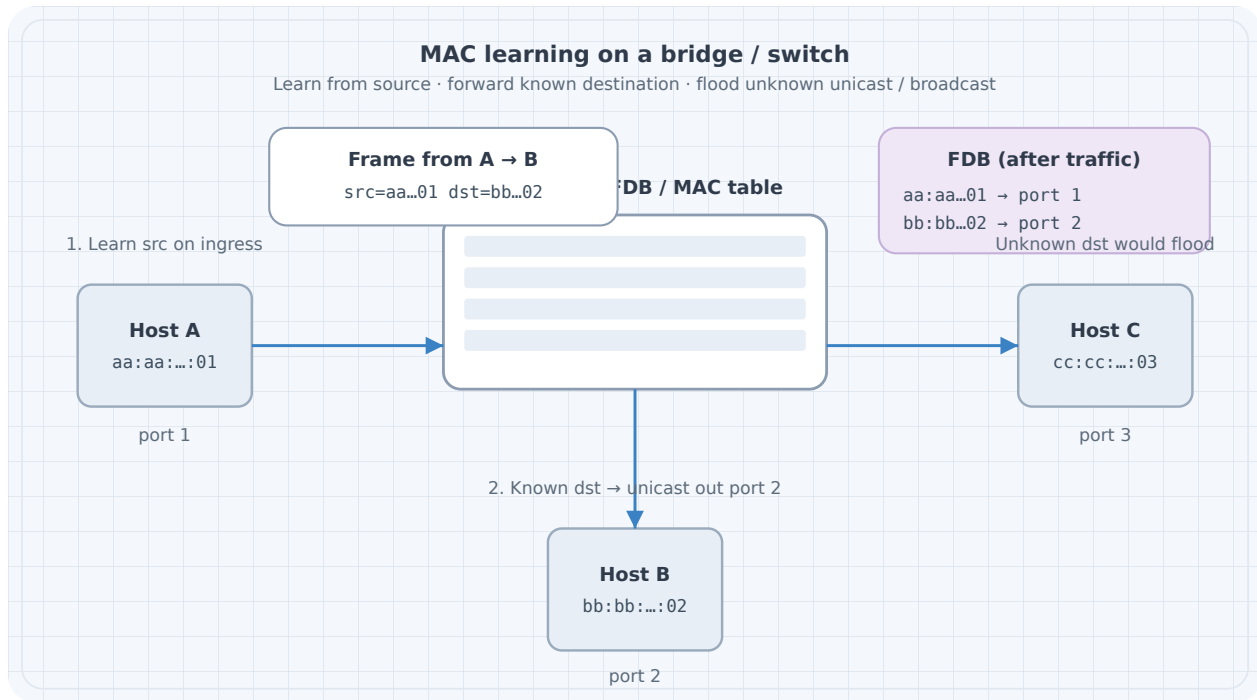


Figure 2: MAC learning: learn source, forward or flood destination

- Do not design security that only trusts MAC equality (easy to spoof)

```
ip link show eth1
# link/ether xx:xx:xx:xx:xx:xx
```

ARP: glue for IPv4 on Ethernet

On-link IPv4 needs ARP:

1. Host knows dest IP is on-link (same subnet / connected route)
2. Who has IP X? Tell Y — broadcast
3. Reply unicast with MAC
4. Neigh cache stores IP MAC

```
ip neigh show
ip neigh flush dev eth1 # lab use carefully
```

IPv6 uses Neighbor Discovery (ICMPv6) instead of ARP—same role, different messages.

Linux bridge FDB

```
# Create bridge and show FDB after traffic
bridge fdb show br br0
bridge link show
```

Dynamic entries learn from traffic; static entries can be added for experiments.

Lab A: two hosts, one bridge (hand or Containerlab)

Containerlab topology

```
name: l2-mac

topology:
  nodes:
    br1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 bridge
        - ip link add name br0 type bridge
        - ip link set br0 up
        # attach eth1/eth2 after links exist - see note below
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.2/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "br1:eth1"]
    - endpoints: ["h2:eth1", "br1:eth2"]
```

Operational note: Alpine bridge setup via `exec` may race; a mounted `entrypoint.sh` is more reliable:


```
#!/bin/sh
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add name br0 type bridge 2>/dev/null || true
ip link set br0 up
ip link set eth1 master br0
ip link set eth2 master br0
ip link set eth1 up
ip link set eth2 up
sleep infinity
```

Bind and set as command for br1. Hosts keep simple addressing execs.

Predict

- First ping causes ARP broadcast flooded to the other host
- FDB on br0 learns h1 and h2 MACs on respective ports
- Subsequent unicast ICMP is switched, not broadcast

Observe

```
docker exec clab-l2-mac-h1 ping -c 3 10.10.0.2
docker exec clab-l2-mac-br1 bridge fdb show br br0
docker exec clab-l2-mac-h1 ip neigh
docker exec clab-l2-mac-h1 tcpdump -ni eth1 -e -c 20 arp or icmp
```

Fix inject

```
# Wrong mask on h2 - still "same numbers" but off-link behavior
docker exec clab-l2-mac-h2 ip addr flush dev eth1
docker exec clab-l2-mac-h2 ip addr add 10.10.0.2/32 dev eth1
docker exec clab-l2-mac-h2 ip link set eth1 up
docker exec clab-l2-mac-h1 ping -c 2 10.10.0.2
```

Restore /24. Journal the symptom.

Harden

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-l2-mac-h1 ping -c 2 -W 1 10.10.0.2
docker exec clab-l2-mac-br1 bridge fdb show br br0 | grep -q .
echo OK
```

Lab B: three hosts — flooding visibility

Add h3 on 10.10.0.3/24 linked to br1:eth3.

Predict: Unknown unicast to a silent host floods; after traffic, FDB constrains.

Observe: tcpdump on h3 while h1 pings h2—after learning, h3 should **not** see unicast ICMP (ideal case). During ARP storms or failures, you may still see broadcasts.

```
docker exec clab-l2-mac-h3 tcpdump -ni eth1 -c 30 &
docker exec clab-l2-mac-h1 ping -c 5 10.10.0.2
```

MAC moves and flaps

```
# Simulate "move": shut one path (in multi-path labs) or change container
# Observe FDB port update and short loss
bridge fdb show br br0
```

Duplicate MACs on two ports (misconfig or bridge loops) cause flapping—spanning tree chapter addresses loops.

Common L2 failures (pre-VLAN)

Symptom	Likely cause
No ARP reply	Far host down, wrong cable/port, filter
ARP ok, no ping	Local firewall, wrong IP on host
Intermittent	MAC flap, duplex issues (physical), loop
Works after first flood only	Misread; check FDB aging and silent nodes

Promiscuous mode and labs

Bridges and captures may require promisc on interfaces:

```
ip link set eth1 promisc on
tcpdump -ni eth1
```

Containerlab links typically allow node-local captures on data interfaces.

Security digression (short)

- Port security / sticky MAC exist on enterprise gear—concept: limit learned MANs
- DHCP snooping / dynamic ARP inspection—later ops world
- For this book: understand **why** spoofing works at L2 so you do not mythologize the LAN

Predict worksheet (paper)

Domain with switch S and hosts A,B,C. A sends to B's IP first time.

1. Is the first frame after ARP unicast or might B's MAC still be unknown?
2. Does C receive the ARP request?
3. Does C receive the ICMP echo request once FDB learned?

Answers: (1) after ARP, A knows B's MAC—frame is unicast; switch floods only if FDB miss on B. (2) yes, ARP request is broadcast. (3) no, not if learning completed and no loop/mirror.

FRR / NOS note

Hardware switches expose `show mac address-table` equivalents. Linux `bridge fdb` is the same idea. When you use SR Linux or others later, translate:

Intent	Linux bridge	Typical NOS language
Show learned MACs	<code>bridge fdb show</code>	<code>show mac table</code>
Clear dynamic	<code>bridge fdb del ...</code>	<code>clear mac</code>

Summary

- Switches learn source MACs, forward known unicasts, flood unknowns/broadcasts
- ARP/ND bind IP to MAC on-link

- FDB + neigh + tcpdump diagnose most “L2 should work” failures
- Build bridge labs with Containerlab Linux nodes before complex VLAN designs

Next: **VLANs and trunks**—splitting broadcast domains without new cables.

VLANs & Trunks

VLANs & Trunks

VLANs split one physical switching fabric into multiple **broadcast domains**. Trunks carry many VLANs between switches with 802.1Q tags. Mis-matched tags and native VLANs are classic outage factories—this chapter makes them deliberate.

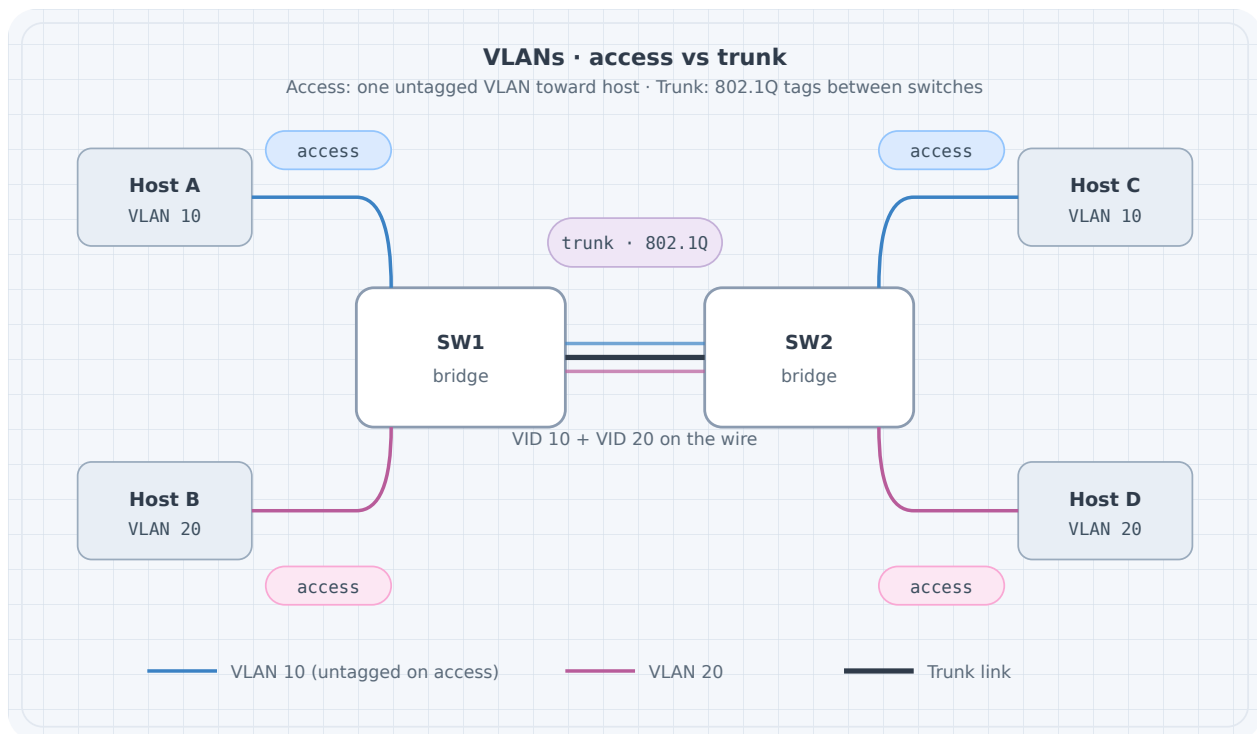


Figure 1: Access ports versus 802.1Q trunk

Learning goals

By the end of this chapter you can:

- Explain access vs trunk ports
- Design a multi-VLAN lab with consistent IDs
- Predict tag behavior on the wire
- Spot native VLAN mismatch symptoms

- Verify isolation between VLANs without a router

Concepts

Broadcast domains

Without VLANs, one bridge = one flood domain. VLAN-aware bridging tags frames with a VLAN ID (VID) and keeps separate FDBs per VLAN (implementation details vary, model holds).

Port mode	Behavior
Access	One VLAN; frames untagged on the wire toward the host
Trunk	Multiple VLANs; tags on the wire (except optional native)
Native VLAN	Untagged traffic on a trunk is mapped to this VLAN

802.1Q tag

Inserts TPID 0x8100 + PCP/DEI + VID. Capture shows tagged frames between switches:

```
tcpdump -ni eth1 -e vlan
# or
tcpdump -ni eth1 -e | head
```

Router roles

Inter-VLAN traffic needs an L3 device (router-on-a-stick, SVI, separate routed ports). Same switch + different VLANs **no IP connectivity** by design.

Addressing plan example

VLAN	Name	Subnet	Access ports
10	users	10.10.10.0/24	h1
20	iot	10.10.20.0/24	h2
99	native/mgmt lab	10.10.99.0/24	optional

Trunk between **sw1** and **sw2** allows 10,20 (and 99 if used).

Lab topology (conceptual)

```
h1 --(access VLAN 10)-- sw1 ==trunk== sw2 --(access VLAN 10)-- h3
h2 --(access VLAN 20)-- sw1 ==trunk== sw2 --(access VLAN 20)-- h4
```

Containerlab sketch using Linux VLAN-aware bridge

Full production-grade switch images are optional; Linux can teach tags.

```
name: l2-vlan

topology:
  nodes:
    sw1:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/sw1.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    sw2:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/sw2.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.20.1/24 dev eth1
        - ip link set eth1 up
    h3:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.3/24 dev eth1
```



```

        - ip link set eth1 up
h4:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 10.10.20.4/24 dev eth1
    - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "sw1:eth1"]
    - endpoints: ["h2:eth1", "sw1:eth2"]
    - endpoints: ["h3:eth1", "sw2:eth1"]
    - endpoints: ["h4:eth1", "sw2:eth2"]
    - endpoints: ["sw1:eth3", "sw2:eth3"]

```

config/sw1.sh (VLAN filtering bridge)

```

#!/bin/sh
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add br0 type bridge vlan_filtering 1
ip link set br0 up

# access ports
for i in 1 2; do ip link set eth$i up; ip link set eth$i master br0; done
ip link set eth3 up
ip link set eth3 master br0

# clear default VLAN 1 membership as needed; set access VLANs
bridge vlan add vid 10 dev eth1 pvid untagged
bridge vlan del vid 1 dev eth1 2>/dev/null || true
bridge vlan add vid 20 dev eth2 pvid untagged
bridge vlan del vid 1 dev eth2 2>/dev/null || true

# trunk eth3: tagged 10,20
bridge vlan add vid 10 dev eth3
bridge vlan add vid 20 dev eth3
bridge vlan del vid 1 dev eth3 2>/dev/null || true

# bridge self VLANs
bridge vlan add vid 10 dev br0 self
bridge vlan add vid 20 dev br0 self

sleep infinity

```

Mirror on **sw2** with eth1→VLAN10, eth2→VLAN20, eth3 trunk.

Kernel/**bridge** command details vary slightly by version—if a flag fails, check **man bridge** on the image and adjust. The **model** (pvid untagged access, tagged trunk) is the learning target.

Predict → observe → fix

Predict

- h1 h3 (VLAN 10) ping works
- h2 h4 (VLAN 20) ping works
- h1 h2 fails (no L3 inter-VLAN)
- Capture on trunk shows VLAN tags for inter-switch frames

Observe

```
docker exec clab-l2-vlan-h1 ping -c 2 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 10.10.20.4
docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.1 # expect fail
docker exec clab-l2-vlan-sw1 bridge vlan show
docker exec clab-l2-vlan-sw1 tcpdump -ni eth3 -e -c 20
```

Failure drill: native / PVID mismatch

On sw1 trunk, map untagged to VLAN 10; on sw2 map untagged to VLAN 20. Send untagged (or mis-access) traffic.

Symptoms often seen:

- Silent discard
- Hosts “leak” into wrong segment
- Intermittent weirdness depending on tagging

Harden: Explicitly tag all lab VLANs on trunks; avoid relying on native VLAN for user data. Document native VLAN ID identically on both ends if you must use it.

Failure drill: missing allowed VLAN

Remove VLAN 20 from sw2 trunk membership.

Predict: VLAN 10 still works; VLAN 20 across switches dies.

```
# on sw2, delete vid 20 from eth3, retest h2->h4
```

Router-on-a-stick preview

```
sw trunk --- r1 eth1.10 + eth1.20

ip link add link eth1 name eth1.10 type vlan id 10
ip link add link eth1 name eth1.20 type vlan id 20
ip addr add 10.10.10.254/24 dev eth1.10
ip addr add 10.10.20.254/24 dev eth1.20
ip link set eth1.10 up
ip link set eth1.20 up
sysctl -w net.ipv4.ip_forward=1
```

Hosts use .254 as gateway. Full L3 chapters expand routing; here you only prove **VLAN separation needs L3 to merge**.

Verification checklist

```
# Per switch
bridge vlan show
bridge fdb show

# Per host
ip -br a
ip neigh
ping -c 2 <same-vlan-far-host>
ping -c 1 -W 1 <other-vlan-host> # should fail without router
```

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-l2-vlan-h1 ping -c 2 -W 1 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 -W 1 10.10.20.4
if docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.1; then
```

```

    echo "FAIL: inter-VLAN should be blocked" >&2
    exit 1
fi
echo OK

```

Negative checks are gold.

Design tips

Tip	Why
Plan VLAN IDs globally in the lab	Avoid “VLAN 10 means different things”
Do not reuse VLAN 1 for user data	Habit for real gear hygiene
Match trunk allow lists both ends	One-way or partial connectivity
Document native VLAN	Mismatch class of bugs
Separate voice/data IDs even in labs	Practice real patterns

Common mistakes

Mistake	Result
Access port in wrong VLAN	ARP never reaches partner
Trunk missing VID	Works locally, fails across switches
Same subnet on two VLANs	Broken design + asymmetric mess
Expecting VLANs to route	Need L3

Summary

- VLANs partition broadcast domains; trunks carry tagged sets between switches
- Access = untagged one VID; trunk = tagged many VIDs
- Align allow lists and native VLAN on both ends
- Prove isolation with intentional negative pings
- Inter-VLAN = router/SVI job

Next: **loop prevention**—what happens when redundant L2 paths exist without a control protocol.

Loop Prevention

Loop Prevention

Redundant L2 links are good for resilience and catastrophic without loop control. Broadcasts and unknown unicasts circulate forever; MAC tables flap; the LAN melts. This chapter covers the **spanning tree family** as a model (not a vendor certification deep dive) and how to see loops in the lab.

Learning goals

By the end of this chapter you can:

- Explain why Ethernet needs loop prevention
- Describe root bridge election and port roles at a conceptual level
- Predict a blocked port in a simple triangle of switches
- Build a loop, observe the blast, then enable prevention
- Prefer design alternatives when L2 redundancy is the wrong tool

Why loops kill L2

Ethernet has no TTL on frames. In a loop:

1. Broadcast / unknown unicast is flooded
2. Copies multiply around the cycle
3. CPUs and links saturate
4. Learning oscillates (MAC seen on alternating ports)

IP routers do not save you inside a pure L2 domain.

Spanning tree behavior

Classic STP/RSTP/MSTP ideas:

Idea	Meaning
Root bridge	Reference switch (lowest priority/BID wins)
Root port	Each non-root switch's best path toward root
Designated port	Best port on a segment toward downstream
Blocked / alternate	Redundant path held in reserve
BPDU	Control frames that build the tree

Modern networks often use **RSTP** or vendor rapid variants; data centers may avoid large L2 domains entirely (L3 fabric). Still, you must recognize STP language on campus gear and many NOS features.

Priority and ties

Lower bridge priority wins. Ties break by MAC. In labs, set priorities explicitly so root placement is intentional:

```
sw1 priority better (numerically lower) → root
sw2/sw3 farther
```

Port states (classic story)

Blocking → Listening → Learning → Forwarding (RSTP simplifies roles/states). During convergence, expect transient blackholes—measure them in drills.

Design alternatives (model maturity)

Approach	Loop story
STP/RSTP on bridged triangle	Protocol blocks one link
L3 access / routed triangle	No L2 loop; ECMP at L3
MC-LAG / bundling	Multi-chassis tricks (complex)
Single homing	No redundancy, no loop

For many greenfield labs after this chapter, **prefer L3 redundancy**. Learn STP so you can operate brownfield and understand failure domains.

Lab: three switches, intentional loop

```
sw1
 /  \
sw2---sw3
```

Hosts on sw2 and sw3.

Topology YAML (Linux bridges)

```
name: l2-loop

topology:
  nodes:
    sw1:
      kind: linux
      image: alpine:3.20
      binds: ["/config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    sw2:
      kind: linux
      image: alpine:3.20
      binds: ["/config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    sw3:
      kind: linux
      image: alpine:3.20
      binds: ["/config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.2/24 dev eth1 && ip link set eth1 up
    h3:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.3/24 dev eth1 && ip link set eth1 up

  links:
    - endpoints: ["sw1:eth1", "sw2:eth1"]
    - endpoints: ["sw1:eth2", "sw3:eth1"]
```



```

- endpoints: ["sw2:eth2", "sw3:eth2"]
- endpoints: ["h2:eth1", "sw2:eth3"]
- endpoints: ["h3:eth1", "sw3:eth3"]

```

Dangerous mode: dumb bridges without STP

```

#!/bin/sh
# config/sw-loop.sh - NO STP (for short controlled drill only!)
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add br0 type bridge
ip link set br0 up
for i in 1 2 3; do
    ip link set eth$i up 2>/dev/null || true
    ip link set eth$i master br0 2>/dev/null || true
done
sleep infinity

```

Warning: Run the loop experiment **briefly** on an isolated lab host. Broadcast storms can stress CPU.

Observe storm (careful)

```

# terminal 1
docker exec clab-l2-loop-h2 ping -f 10.10.0.3 &
# or send broadcasts
docker exec clab-l2-loop-sw1 ip -s link
# counters explode

```

Destroy promptly:

```

sudo containerlab destroy -t l2-loop.clab.yml --cleanup

```

Safer mode: enable STP on the bridge

Linux bridge STP:

```

ip link add br0 type bridge
ip link set br0 type bridge stp_state 1
ip link set br0 up
# attach ports...

```

```
# inspect
docker exec clab-l2-loop-sw1 cat /sys/class/net/br0/bridge/stp_state
# port states under /sys/class/net/br0/brif/*/state
```

Predict: One of the triangle links is blocked; ping between h2 and h3 still works via remaining tree; storm does not persist.

Observe:

```
docker exec clab-l2-loop-h2 ping -c 5 10.10.0.3
# check which port is blocking on each sw
for s in sw1 sw2 sw3; do
    echo "== $s =="
    docker exec clab-l2-loop-$s sh -c 'for p in /sys/class/net/br0/brif/*/state; do echo $p $(
done
```

Root placement drill

Influence root by bridge priority (when using STP implementations that expose it):

```
# example sysfs / tools depend on stack; concept: make sw1 root
ip link set br0 type bridge priority 0x1000
```

On NOS images (SR Linux, etc.), use vendor STP priority knobs—same election idea.

Predict: With sw1 root, blocked port is likely on the sw2–sw3 link (common triangle outcome with symmetric costs).

Verify: Diagram the tree; mark root ports and blocked port.

Failure drill: link down on active tree edge

1. Baseline ping
2. Shut a forwarding uplink
3. Measure time until ping recovers
4. Confirm previously blocked port becomes forwarding

```
docker exec clab-l2-loop-sw1 ip link set eth1 down
# time recovery
docker exec clab-l2-loop-h2 ping -c 20 10.10.0.3
docker exec clab-l2-loop-sw1 ip link set eth1 up
```

Journal convergence feel (RSTP vs classic differs a lot).

BPDUs and protection concepts (awareness)

Feature idea	Intent
BPDU guard	Err-disable access port if BPDU seen
Root guard	Prevent downstream from becoming root
Loop guard	Protect against unidirectional link issues

You may not implement all in Alpine labs; know the **jobs** so production configs make sense.

When not to extend L2

Smell	Prefer
L2 stretched across sites	L3 + overlay if needed
Huge broadcast domain	Segment VLANs + route
STP diameter anxiety	Routed access / leaf-spine

Spanning tree is a safety net, not an excuse for continent-sized broadcast domains.

Predict worksheet

Triangle swA–swB–swC, all costs equal, swA root.

1. Which link is most likely blocked?
2. Host on swB to host on swC: path?
3. If swA dies, what must happen?

Answers: (1) often B–C; (2) B–A–C; (3) re-election, former blocked may forward, connectivity among survivors if links remain.

Verification checklist for STP labs

- Never leave an intentional storm lab running unattended
- Record before/after port states
- Compare L2 redundancy recovery to L3 ECMP recovery later—feel the difference

Summary

- L2 loops multiply floods; no frame TTL
- Spanning tree elects a root and blocks redundant ports
- Practice triangle labs with STP on; only briefly demo storms
- Root placement is a design choice
- Mature designs often push redundancy to L3

Next: **link aggregation**—bundling links for bandwidth and failover without creating an independent loop story (when configured correctly).

Link Aggregation

Link Aggregation

Link aggregation (LAG / bonding / etherchannel-class ideas) combines multiple physical links into one logical link: more bandwidth, faster failover, and fewer STP blocked paths between the same pair of devices—when both ends agree.

Learning goals

By the end of this chapter you can:

- Explain when LAG helps vs when it hides problems
- Contrast static bundle vs LACP
- Configure a Linux bond in a lab
- Predict traffic behavior and failover
- Verify member health and hashing limitations

Concepts

One logical link

```
      +---- eth1 ----+
sw1 -+                +- sw2
      +---- eth2 ----+
           \____ bond0 / lag0 ____/
```

Upper protocols (STP, L3 adjacency) see **one** link if bundling is correct.

Static vs LACP

Mode	Pros	Cons
Static / on	Simple	Mis-wiring may blackhole; no negotiation

LACP	Detects partner; protects against some mistakes	Both ends must support/enable
-------------	---	-------------------------------

Labs may start static; production almost always prefers LACP.

Hashing

Flows distribute across members by a hash (MAC/IP/L4 ports depending on config). **One TCP flow** typically stays on one member—do not expect a single iperf stream to fill $N \times$ bandwidth.

Split brain / misconfig

Classic fail: one side bundles, the other has independent ports—loops or blackholes. Always verify both ends.

Linux bonding quickstart

Kernel bonding modes (selection depends on goal):

Mode	Name	Notes
0	balance-rr	Round-robin
1	active-backup	Simple failover
4	802.3ad	LACP

```
# Example active-backup (conceptual host commands)
modprobe bonding
ip link add bond0 type bond mode active-backup miimon 100
ip link set eth1 down
ip link set eth2 down
ip link set eth1 master bond0
ip link set eth2 master bond0
ip addr add 10.0.0.1/24 dev bond0
ip link set bond0 up
ip link set eth1 up
ip link set eth2 up
cat /proc/net/bonding/bond0
```

In containers, bonding support depends on privileges and kernel modules on the **host**. Containerlab labs sometimes simulate aggregation at the NOS layer instead. If bonding is blocked in your environment, run this chapter on a Linux VM/nested host and keep Containerlab for endpoints.

Containerlab topology (two paths, bond on routers)

```
name: l2-lag

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/r1-bond.sh:/startup.sh
      cmd: /bin/sh /startup.sh
      # may need privileged: true depending on clab version/kind options
    r2:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/r2-bond.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["h1:eth1", "r1:eth3"]
    - endpoints: ["h2:eth1", "r2:eth3"]
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r1:eth2", "r2:eth2"]
```


config/r1-bond.sh

```
#!/bin/sh
set -e
apk add --no-cache iproute2 iputils >/dev/null
# fallback if bonding unavailable: sequential routes demo only
if ip link add bond0 type bond mode 802.3ad 2>/dev/null; then
    ip link set eth1 down
    ip link set eth2 down
    ip link set eth1 master bond0
    ip link set eth2 master bond0
    ip addr add 10.0.0.1/24 dev bond0
    ip link set bond0 up
    ip link set eth1 up
    ip link set eth2 up
else
    echo "bonding unavailable; configure two links manually for failover drill" >&2
    ip addr add 10.0.0.1/24 dev eth1
    ip link set eth1 up
    ip link set eth2 up
fi
ip addr add 192.168.1.1/24 dev eth3
ip link set eth3 up
sysctl -w net.ipv4.ip_forward=1
ip route add 192.168.2.0/24 via 10.0.0.2
sleep infinity
```

Symmetric on r2 with 10.0.0.2, LAN 192.168.2.1/24, route to 192.168.1.0/24 via 10.0.0.1.

If LACP mode fails without a partner daemon, use mode **active-backup** for reliable lab failover learning.

Predict → observe → fix

Predict

- h1→h2 ping works through logical path
- Shutting **one** member keeps session alive (active-backup or LACP with remaining member)
- Single flow throughput one member speed

Observe

```
docker exec clab-l2-lag-h1 ping -c 5 192.168.2.10
docker exec clab-l2-lag-r1 cat /proc/net/bonding/bond0 2>/dev/null || true
docker exec clab-l2-lag-r1 ip -br link
```

Failover drill

```
docker exec clab-l2-lag-r1 ip link set eth1 down
docker exec clab-l2-lag-h1 ping -c 10 192.168.2.10
docker exec clab-l2-lag-r1 ip link set eth1 up
```

Measure loss count. Journal recovery time.

Mismatch drill

On r2, do **not** enslave eth2; leave as separate interface with another IP.

Predict: loops, flaps, or partial connectivity—document what **your** kernel does; fix by matching configs.

Hashing awareness drill

```
# multiple parallel streams may use more members than one stream
# on hosts with iperf3
iperf3 -s # h2
iperf3 -c 192.168.2.10 -P 8
```

Compare -P 1 vs -P 8.

STP interaction

Between two switches, a multi-link LAG is one logical link STP sees one edge. Independent unbundled links STP blocks all but one (or you loop). **Bundle before you celebrate “redundant cables.”**

Check	Intent
-------	--------

Operational checks (NOS-neutral)

Check	Intent
Members all up	Physical/L1
LACP PDUs both ways	Negotiation
Same speed/duplex	Bundle eligibility
Counters increment on members	Actual use
Control plane adjacency single	Logical link view

Common mistakes

Mistake	Result
One end LACP, other static only	Stuck negotiation / down
Different VLANs on members	Inconsistent bundle
Expect one flow = N x bandwidth	Disappointed benchmarks
Forget host firewall on bond MAC	Odd ARP

Summary

- LAG = one logical link from multiple physical members
- Prefer LACP in real life; static only with tight control
- Hashing limits single-flow throughput
- Verify both ends; failover-test by shutting members
- Unbundled parallel L2 links reintroduce STP/loop problems

Next: **L2 failure drills**—deliberate breakage across MAC, VLAN, loop, and bundle scenarios with evidence.

L2 Failure Drills

L2 Failure Drills

This chapter is a **practice arena**. You combine Ethernet, VLANs, loop prevention, and aggregation under intentional failure. Competence is measured by evidence and recovery, not by green first pings.

Learning goals

By the end of this chapter you can:

- Run a structured failure catalog against an L2 lab
- Collect the minimum evidence set for each class of break
- Recover cleanly and re-verify
- Write short incident notes a peer can follow
- Know when an “L2 issue” is actually L3 or host firewall

Baseline lab

Use a multi-switch VLAN lab (from the VLANs chapter) with:

- Two switches, trunk between them
- VLAN 10 and VLAN 20
- Hosts h1/h3 in VLAN 10, h2/h4 in VLAN 20
- Optional inter-VLAN router for a subset of drills

Document addressing in **addressing.md**. Ensure **./verify.sh** is green before any inject.

Evidence kit

```
# Host
ip -br a
ip neigh
ping -c 3 -W 1 <target>
tcpdump -ni eth1 -e -c 30 arp or icmp

# Switch / bridge
bridge vlan show
bridge fdb show
ip -s link
# STP port states if enabled
```

Save outputs under `journal.md` with timestamps.

Drill catalog

For each drill: **predict** → **inject** → **observe** → **restore** → **harden**.

Drill 1 — Access port wrong VLAN

Inject: Move h1's switch port from VLAN 10 to VLAN 20 (PVID change).

Predict: h1 cannot ARP h3; may see wrong subnet peers if any; `verify.sh` fails.

Observe: `bridge vlan show`, `tcpdump` for ARP who-has never answered by h3.

Restore: Correct PVID/untagged VLAN.

Harden: Label ports in diagrams; negative test in `verify` (optional).

Drill 2 — Trunk missing VLAN

Inject: Remove VLAN 10 from trunk allow list on one switch only.

Predict: Local VLAN 10 still works on that switch; cross-switch VLAN 10 dies; VLAN 20 may still work.

Observe: Asymmetric “works on same switch only.”

Restore: Re-add VID on both ends.

Harden: Checklist “both ends allow list.”

Drill 3 — Native VLAN mismatch

Inject: Native/PVID untagged mapping differs on trunk ends.

Predict: Untagged frames land in different broadcast domains; confusing intermittent leaks.

Observe: Captures show untagged where you expected tags; wrong FDB.

Restore: Align native VLAN; prefer tagged-only user VLANs.

Harden: Explicit lab policy: native VLAN unused for data.

Drill 4 — Cable / interface down

Inject:

```
docker exec clab-l2-vlan-sw1 ip link set eth3 down
```

Predict: Cross-switch traffic dies; FDB ages; local switching may live.

Observe: ping loss; interface counters; STP reconvergence if redundant path exists.

Restore: `ip link set eth3 up`; wait for learning/STP.

Harden: If no redundant path, document single point of failure.

Drill 5 — Broadcast storm (controlled)

Inject: Temporary bridging loop **without** STP on an isolated lab (very short).

Predict: CPU spikes, loss of management responsiveness, exploding counters.

Observe: `ip -s link`, host load.

Restore: `containerlab destroy --cleanup` immediately if unstable.

Harden: STP on; never lab this on a shared prod-like bridge.

Drill 6 — STP blocked port confusion

Inject: With STP triangle, shut the active root port path.

Predict: Alternate port activates; brief loss; new tree.

Observe: Port state sysfs or NOS show; ping -c 50 loss burst.

Restore: Bring link up; tree may revert depending on costs.

Harden: Note expected reconvergence budget.

Drill 7 — MAC move

Inject: Move host container link from sw1 to sw2 (redploy link or migrate veth).

Predict: FDB updates; short blackhole until relearn; ARP may refresh.

Observe: bridge fdb show before/after; neigh on peers.

Restore: Stable attachment.

Harden: In virtualization, understand port channels / bond moves.

Drill 8 — Duplicate IP

Inject: Configure h4 temporarily with h3's IP on VLAN 10 (careful).

Predict: Flapping ARP, intermittent delivery, wrong neigh.

Observe: ip neigh oscillating; tcpdump conflicting ARP replies.

Restore: Unique IPs.

Harden: Addressing plan ownership; IPAM habit.

Drill 9 — Duplicate MAC (advanced)

Inject: Spoof MAC on a second host (lab only).

```
ip link set eth1 down
ip link set eth1 address aa:bb:cc:dd:ee:ff
ip link set eth1 up
```

Predict: FDB flap; unstable delivery.

Restore: Original MAC; destroy/recreate if messy.

Harden: Understand L2 trust boundaries.

Drill 10 — LAG member down

Inject: On a bond/LAG lab, shut one member.

Predict: Traffic continues if min-links satisfied; single-flow rate unchanged.

Observe: `/proc/net/bonding/bond0`; ping continuity.

Restore: Member up.

Harden: Alert on member-down even if bond stays up.

Drill 11 — Inter-VLAN path mistaken for L2

Inject: Delete router inter-VLAN route or shut subinterface.

Predict: Same-VLAN ok; cross-VLAN fails—looks like “VLAN broken” if you misread.

Observe: traceroute never leaves gateway; ARP to gateway fails or gateway unreachable.

Restore: Router config.

Harden: Always check same-VLAN baseline before blaming trunk.

Drill 12 — Host firewall

Inject: On Linux host, drop ICMP.

```
# if nft/iptables available
iptables -A INPUT -p icmp -j DROP
```

Predict: Others cannot ping in; egress may still work.

Observe: Asymmetric ping stories.

Restore: Flush rule.

Harden: Include host endpoint in the failure picture.

Incident note template

```
## Incident: trunk missing VLAN 10 on sw2
Date:
Lab: l2-vlan

### User symptom
h1 cannot reach h3; h1 reaches local same-switch hosts.

### Prediction before fix
Trunk allow list asymmetry.

### Evidence
- bridge vlan show sw1/sw2 (paste)
- tcpdump trunk: no VLAN 10 tags toward sw2
- ping results

### Root cause
vid 10 deleted from sw2 eth3

### Fix
bridge vlan add vid 10 dev eth3

### Prevent
verify.sh checks cross-switch VLAN 10 and 20
README allow-list both ends
```

Scorecard

Run at least **six** drills across different classes. Check:

Class	Drills done
VLAN membership	
Trunk tagging	
Link down	
STP / loop	
Aggregation	
Host/IP mistakes	

Combined verify.sh ideas

```
#!/usr/bin/env bash
set -euo pipefail
# positive
docker exec clab-l2-vlan-h1 ping -c 2 -W 1 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 -W 1 10.10.20.4
# negative isolation
if docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.4 2>/dev/null; then
    echo "unexpected inter-VLAN without router" >&2
    exit 1
fi
echo OK
```

After each restore, re-run.

Checkpoint: L2 competence

You pass the part when you can:

1. Deploy multi-switch VLANs from code
2. Produce evidence for tag/allow-list bugs
3. Explain and demo loop risk
4. Fail over a link or LAG member deliberately
5. Write an incident note with root cause and prevention

Summary

- Failure drills convert L2 theory into ops judgment

- Always baseline with `verify.sh`
- Predict before inject; restore and re-verify after
- Separate L2, L3, and host causes with evidence
- Incident notes are part of the curriculum artifact

Next part: **layer 3**—addressing, routing fundamentals, static labs, ICMP and path MTU.

Layer 3

IP Addressing

IP Addressing

Forwarding starts with **addressing**: structure, masks, subnets, summarization, and enough IPv6 to run dual-stack labs. This chapter is calculation plus lab verification—not exam trick puzzles for their own sake.

Learning goals

By the end of this chapter you can:

- Convert between prefix length and masks fluently
- Design subnets for a multi-tier lab without overlaps
- Summarize contiguous blocks and predict blackholes from bad summaries
- Configure IPv4 and basic IPv6 on Linux
- Use documentation ranges and lab hygiene ranges deliberately

IPv4 structure

An IPv4 address is 32 bits, usually dotted decimal. A **prefix address/len** identifies network bits vs host bits.

Prefix	Mask	Hosts (approx usable)
/32	255.255.255.255	1 (host route)
/31	255.255.255.254	2 (P2P often)
/30	255.255.255.252	2 usable classic
/24	255.255.255.0	254 usable classic
/16	255.255.0.0	large
/8	255.0.0.0	huge

```
# Linux
ip addr add 192.0.2.10/24 dev eth1
ip -br a
ip route
```

On-link decision

If destination is inside a connected prefix, the host uses ARP/ND on that interface. Otherwise it sends to a gateway (route lookup).

```
ip route get 192.0.2.50
ip route get 8.8.8.8
```

Subnetting for labs

Example campus-ish allocation from 10.20.0.0/16:

Use	Block
Users site A	10.20.1.0/24
Users site B	10.20.2.0/24
Servers	10.20.10.0/24
P2P core	10.20.255.0/24 as /30s
Loopbacks	10.20.254.0/24 as /32s

Rules:

1. No overlapping prefixes on different segments unless deliberate anycast
2. Leave growth room
3. Write the plan before deploy
4. Align summaries with actual aggregation topology

Summarization

If you own 10.20.1.0/24 and 10.20.2.0/24, a summary 10.20.0.0/22 may or may not be correct depending on other contents of that /22.

Action	Risk
Summary advertises space you do not have	Blackhole for missing parts
Too-specific everywhere	Table bloat (later IGP/BGP scale)
Forgetting more-specific local routes	Traffic steers wrong

Paper drill

You have:

- 10.0.0.0/25
- 10.0.0.128/25

Correct tight summary? 10.0.0.0/24.

You have only 10.0.0.0/25 but advertise 10.0.0.0/24 upstream without a sink. What happens to 10.0.0.200? **Blackhole** toward you.

Special-use ranges (know them)

Range	Role
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private (RFC 1918)
127.0.0.0/8	Loopback
169.254.0.0/16	Link-local
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Documentation (RFC 5737)
224.0.0.0/4	Multicast

IPv6 essentials for dual-stack labs

IPv6 address is 128 bits; common lab prefix /64 on LANs.

```
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip -6 route
ip -6 neigh
ping -6 -c 2 2001:db8:1::1
```

Prefix idea	Example
Documentation	2001:db8::/32
Unique local	fd00::/8 (with local generation rules)
Link-local	fe80::/10 auto on interfaces

ND replaces ARP. Router Advertisements may autoconfigure hosts—labs often use statics for control.

Dual-stack habit

Give nodes both families when learning:

```
ip addr add 10.1.0.10/24 dev eth1
ip -6 addr add 2001:db8:1::10/64 dev eth1
```

Test both paths; do not assume v4 success means v6 works.

Linux configuration patterns

Persistent lab style

In Containerlab, prefer startup scripts or image config over manual `ip addr` that vanishes.

```
#!/bin/sh
ip addr flush dev eth1
ip addr add 10.1.0.10/24 dev eth1
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip route add default via 10.1.0.1
ip -6 route add default via 2001:db8:1::1
```

Multiple addresses

```
ip addr add 10.1.0.10/24 dev eth1
ip addr add 10.1.0.11/24 dev eth1
```

Secondary addresses appear in ARP; know who answers.

Lab: addressing plan + dual-stack hosts

```
name: l3-addr

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - sysctl -w net.ipv4.ip_forward=1
```

```

- sysctl -w net.ipv6.conf.all.forwarding=1
- ip addr add 10.1.0.1/24 dev eth1
- ip addr add 10.2.0.1/24 dev eth2
- ip -6 addr add 2001:db8:1::1/64 dev eth1
- ip -6 addr add 2001:db8:2::1/64 dev eth2
- ip link set eth1 up
- ip link set eth2 up
h1:
kind: linux
image: alpine:3.20
exec:
- apk add --no-cache iproute2 iputils
- ip addr add 10.1.0.10/24 dev eth1
- ip -6 addr add 2001:db8:1::10/64 dev eth1
- ip link set eth1 up
- ip route add default via 10.1.0.1
- ip -6 route add default via 2001:db8:1::1
h2:
kind: linux
image: alpine:3.20
exec:
- apk add --no-cache iproute2 iputils
- ip addr add 10.2.0.10/24 dev eth1
- ip -6 addr add 2001:db8:2::10/64 dev eth1
- ip link set eth1 up
- ip route add default via 10.2.0.1
- ip -6 route add default via 2001:db8:2::1
links:
- endpoints: ["h1:eth1", "r1:eth1"]
- endpoints: ["h2:eth1", "r1:eth2"]

```

Predict

- h1 pings h2 v4 and v6 through r1
- Wrong mask on h1 (e.g. /32 only) breaks on-link perception of gateway

Observe

```

docker exec clab-l3-addr-h1 ping -c 2 10.2.0.10
docker exec clab-l3-addr-h1 ping -6 -c 2 2001:db8:2::10
docker exec clab-l3-addr-h1 ip route get 10.2.0.10
docker exec clab-l3-addr-r1 ip -br a

```

Failure: overlapping subnets

Put h2 also in 10.1.0.0/24 while connected to r1 eth2 differently—create intentional overlap and journal chaos (ARP confusion, asymmetric paths). Restore clean plan.

Failure: bad summary (static preview)

On a third router path (optional), advertise/install 10.0.0.0/8 via a dead next hop while more specifics missing—observe blackhole for unused space.

Calculation practice (quick)

1. How many /24s in a /16? → 256
2. First/last usable in 10.0.0.0/30 classic? → 10.0.0.1 and 10.0.0.2
3. Is 10.0.0.64/26 inside 10.0.0.0/24? → yes
4. Can you summarize 10.1.0.0/24 and 10.2.0.0/24 as a single /23? → no (not contiguous bit boundary as one /23 covering both cleanly without others)

Work these on paper until automatic.

Host vs router addressing habits

Role	Typical
Host	One address + default route
Router	Addresses on each L3 interface + routing table
Loopback	Stable /32 or /128 for protocols later

```
ip addr add 1.1.1.1/32 dev lo
```

Verification checklist

```
ip -br a
ip route
ip -6 route
ip route get <dst>
ping -c 2 <dst>
ping -6 -c 2 <dst6>
```

Summary

- Prefixes define on-link vs routed behavior
- Plan non-overlapping lab allocations; document them
- Summaries must match reality or they blackhole
- Dual-stack labs need explicit v6 config and tests
- `ip route get` is your friend

Next: **routing fundamentals**—RIB/FIB, longest match, next hops, and host vs router behavior including ARP/ND.

Routing Fundamentals

Routing Fundamentals

Routing is **longest-prefix match** to a next hop or interface, plus the glue that resolves the next hop to L2. This chapter ties RIB/FIB, metrics, and host vs router behavior into one model you will reuse for statics and dynamic protocols.

Learning goals

By the end of this chapter you can:

- Explain longest prefix match with competing routes
- Distinguish connected, static, and protocol routes
- Resolve next hops (ARP/ND) and see failures when resolution dies
- Use `ip route get` as a decision debugger
- Describe host default-route behavior vs multi-interface routers

Concepts

Forwarding equation

For each packet:

1. Look up destination in FIB
2. Select winning route (longest prefix, then tie-breakers)
3. Find egress interface and L2 rewrite target
4. Decrement TTL / hop limit; drop if zero

Store	Role
-------	------

RIB vs FIB

Store	Role
RIB	Routing information: all candidates, protocol owners
FIB	What data plane uses for forward

On Linux, `ip route` shows kernel FIB. FRR holds protocol RIB and installs selected routes into the kernel via zebra.

```
ip route
vtysh -c 'show ip route'    # FRR RIB view
```

Longest prefix match

Routes:

- 0.0.0.0/0 via gwA
- 10.0.0.0/8 via gwB
- 10.1.2.0/24 via gwC

Destination 10.1.2.5 → **gwC**. Destination 10.9.0.1 → **gwB**. Destination 1.2.3.4 → **gwA**.

```
ip route add 10.0.0.0/8 via 192.0.2.1
ip route add 10.1.2.0/24 via 192.0.2.2
ip route get 10.1.2.5
```

Metrics and distance (ideas)

Multiple protocols may offer the same prefix. Selection uses administrative preference (distance) then metric. Linux has metrics on routes; FRR has AD per protocol. Exact numbers differ—**model** is ranked sources.

Connected routes

Configuring 10.1.0.1/24 on eth1 installs connected route 10.1.0.0/24 dev eth1. Connected usually beats static/default for that space.

Host vs router

Host	Router
Often one default route	Many specific routes
Forwards? usually no	<code>ip_forward=1</code>
ARP for on-link and gateway	ARP for each next hop per interface
Care about DNS, apps	Care about RIB scale, policy

Gateway ARP

When host sends off-link:

- Destination IP stays the remote
- Ethernet dest is **gateway MAC**

```
# host
ip route get 8.8.8.8
ip neigh show
tcpdump -ni eth1 -e host 8.8.8.8
```

Next-hop resolution failures

Symptom: route exists, ping fails.

```
ip route get 10.2.0.10
# shows via 10.0.0.2 dev eth1
ip neigh show dev eth1
# if FAILED/INCOMPLETE for 10.0.0.2 → L2 problem or peer down
```

Control vs data: Static route “up” in config does not guarantee next hop resolves.

Asymmetric routing

Forward path $R1 \rightarrow R2 \rightarrow R3$, return $R3 \rightarrow R4 \rightarrow R1$. Stateless ping may work; stateful firewalls and some reverse-path filters break.

Always verify both directions.

```
# from each side
ping -c 2 <other>
traceroute -n <other>
```

Lab: competing prefixes

```
name: l3-lpm

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.0.1
        - ip route add 10.1.2.0/24 via 192.168.0.2
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.1/24 dev eth1
        - ip link set eth1 up
    r2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.2/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    # NOTE: for a real multi-router demo, use a bridge multipoint or
    # connect h1-r1 and h1-r2 carefully; simplest teaching variant below.
```

Cleaner multipoint teaching lab: one LAN bridge with h1, gw1, gw2:

```
name: l3-lpm

topology:
  nodes:
    br0:
      kind: linux
      image: alpine:3.20
      binds: ["/config/br.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
```

```

h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.0.10/24 dev eth1 && ip link set eth1 up
    - ip route add default via 192.168.0.1
    - ip route add 10.1.2.0/24 via 192.168.0.2
gw1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.0.1/24 dev eth1 && ip link set eth1 up
gw2:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.0.2/24 dev eth1 && ip link set eth1 up

links:
  - endpoints: ["h1:eth1", "br0:eth1"]
  - endpoints: ["gw1:eth1", "br0:eth2"]
  - endpoints: ["gw2:eth1", "br0:eth3"]

```

Predict / observe

```

docker exec clab-l3-lpm-h1 ip route get 10.1.2.5
# expect via 192.168.0.2
docker exec clab-l3-lpm-h1 ip route get 10.9.0.1
# expect via 192.168.0.1 (default) if no better route

```

Lab: router with two interfaces

```

# r1
sysctl -w net.ipv4.ip_forward=1
ip addr add 10.1.0.1/24 dev eth1
ip addr add 10.2.0.1/24 dev eth2
# hosts with defaults via r1

```

Predict: Connected routes cover both LANs; hosts need default only; r1 needs no static for inter-LAN.

Observe:

```
ip route
ip route get 10.2.0.10 from 10.1.0.10 iif eth1
```

ARP/ND deep enough for routing

IPv4	IPv6
ARP request/reply	NS/NA
ip neigh	ip -6 neigh
Broadcast	Multicast solicited-node

```
ip monitor neigh
```

Flush for drills:

```
ip neigh flush dev eth1
```

TTL and traceroute

Each hop decrements TTL. Traceroute infers path from ICMP time exceeded.

```
traceroute -n 10.2.0.10
# or
mtr -rwzc 20 10.2.0.10
```

Filters that drop ICMP break traceroute but not always data—note for ops.

Policy preview

Policy routing (`ip rule`) can select alternate tables. Out of scope for depth here; know it exists when “`ip route get`” surprises you with marks/fwmarks.

```
ip rule list
```

Predict → observe → fix drills

Drill: missing return route

Two routers between hosts; only one direction has statics.

Symptom: requests arrive, replies die.

Fix: install return route.

Harden: verify.sh pings both ways.

Drill: wrong gateway MAC path

Shut gateway interface; host neigh becomes FAILED.

```
ip neigh show
ping -c 2 <offlink>
```

Drill: more-specific shadow

Add junk more-specific blackhole:

```
ip route add 10.2.0.10/32 via 192.0.2.254
# 192.0.2.254 dead
ping 10.2.0.10 # fail despite broader working route
ip route del 10.2.0.10/32
```

Cheatsheet

```
ip route
ip route get 1.2.3.4
ip route add 10.0.0.0/8 via 192.168.0.1 metric 100
ip route del 10.0.0.0/8 via 192.168.0.1
ip neigh show
sysctl net.ipv4.ip_forward
```

Summary

- Longest prefix match drives selection
- Connected, static, and dynamic routes feed RIB→FIB

- Next hop must resolve at L2
- Hosts mostly default; routers interconnect prefixes
- Bidirectional reachability is the real goal

Next: **static routing labs**—triangles, floating statics, and deliberate failover.

Static Routing Labs

Static Routing Labs

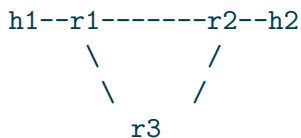
Static routes are the control plane you can see with your eyeballs. Master them in Containerlab with FRR or Linux before OSPF/BGP hide the decisions inside protocols.

Learning goals

By the end of this chapter you can:

- Build a multi-router static topology as code
- Install and verify routes in Linux and FRR
- Implement floating static failover with metrics
- Break and restore return paths deliberately
- Write verify scripts that catch one-way routing

Topology: triangle with hosts



Link	Subnet
r1-r2	10.0.12.0/24
r2-r3	10.0.23.0/24
r3-r1	10.0.13.0/24
h1 LAN	192.168.1.0/24 (r1 gateway)
h2 LAN	192.168.2.0/24 (r2 gateway)
r3 LAN optional	192.168.3.0/24

Loopbacks: r1 1.1.1.1/32, r2 2.2.2.2/32, r3 3.3.3.3/32.

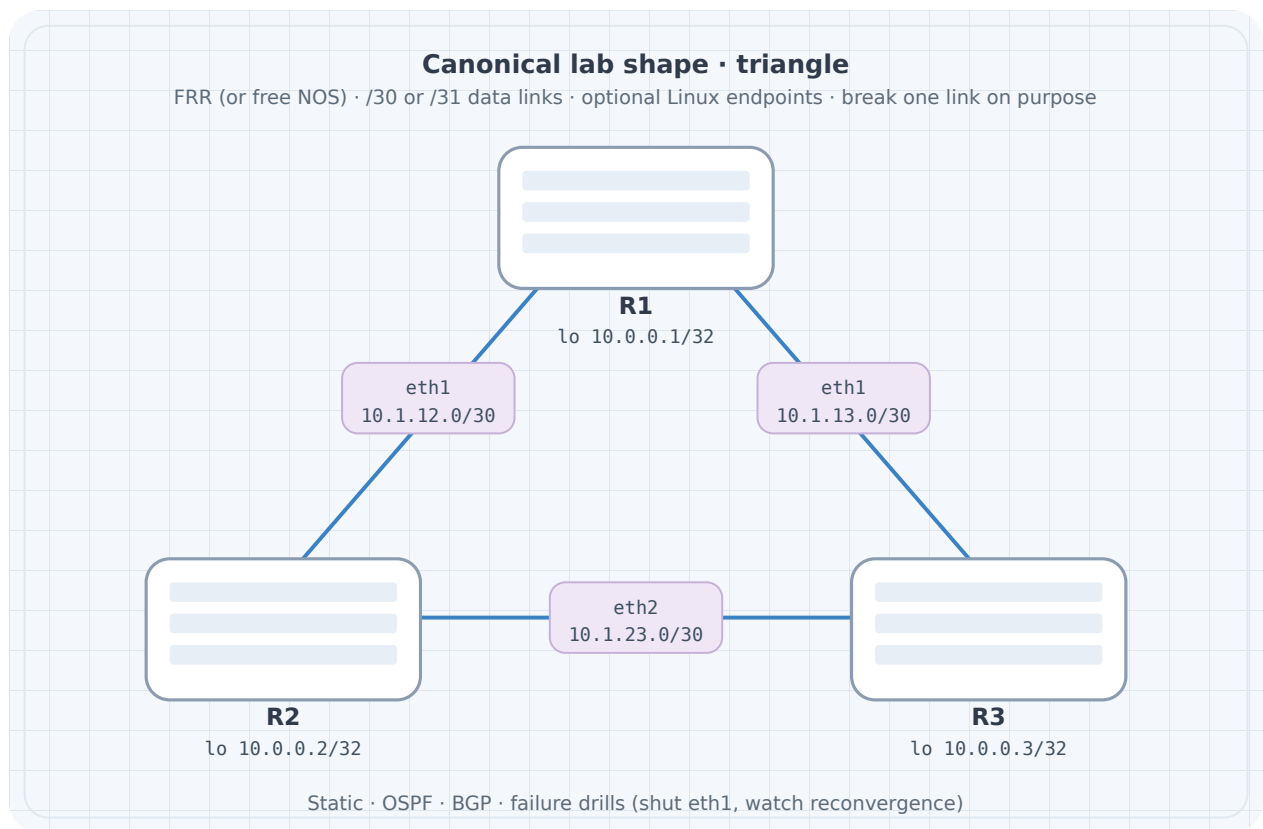


Figure 1: Triangle

Containerlab topology

```
name: static-tri

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["h1:eth1", "r1:eth3"]
    - endpoints: ["h2:eth1", "r2:eth3"]
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]
```

FRR config sketches

Enable zebra in daemons. Example `frr.conf` for `r1`:

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface lo
 ip address 1.1.1.1/32
!
interface eth1
 ip address 10.0.12.1/24
!
interface eth2
 ip address 10.0.13.1/24
!
interface eth3
 ip address 192.168.1.1/24
!
ip route 192.168.2.0/24 10.0.12.2
ip route 2.2.2.2/32 10.0.12.2
ip route 3.3.3.3/32 10.0.13.2
!
line vty
```

r2 mirrors with LAN 192.168.2.1/24, route to 192.168.1.0/24 via 10.0.12.1, etc.

r3 needs routes to both LANs if used as alternate path:

```
ip route 192.168.1.0/24 10.0.13.1
ip route 192.168.2.0/24 10.0.23.1
```

Adjust interface IPs for `r3` on 10.0.13.2 and 10.0.23.2.

Exact FRR container startup (vtysh config load) depends on image—follow image docs for `/etc/frr` layout (`frr.conf`, `daemons`, permissions).

Linux-only alternative

If FRR image friction appears, pure Alpine routers work:

```
sysctl -w net.ipv4.ip_forward=1
ip addr add ...
ip route add 192.168.2.0/24 via 10.0.12.2
```

Deploy and baseline

```
sudo containerlab deploy -t static-tri.clab.yml
docker exec clab-static-tri-r1 vtysh -c 'show ip route'
docker exec clab-static-tri-h1 ping -c 3 192.168.2.10
docker exec clab-static-tri-h1 traceroute -n 192.168.2.10
```

Predict

- Path h1→h2 is h1-r1-r2-h2 (direct static)
- Loopback 2.2.2.2 reachable from r1

Observe

```
docker exec clab-static-tri-r1 ip route get 192.168.2.10
docker exec clab-static-tri-r2 ip route get 192.168.1.10
```

Floating statics

On r1, prefer r2 path, backup via r3:

```
ip route 192.168.2.0/24 10.0.12.2
ip route 192.168.2.0/24 10.0.13.2 200
```

In Linux:

```
ip route add 192.168.2.0/24 via 10.0.12.2 metric 10
ip route add 192.168.2.0/24 via 10.0.13.2 metric 200
```

Caveat: Floating statics failover when the **route is withdrawn**—often when the interface or next hop tracking says down. A silent next hop that stays “up” but blackholes may **not** fail over without tracking/BFD (advanced). In labs, shut the interface to force failover.

Failover drill

```
docker exec clab-static-tri-h1 ping -c 100 192.168.2.10 &
docker exec clab-static-tri-r1 ip link set eth1 down
# observe loss then recovery via r3 if floating installed end-to-end
docker exec clab-static-tri-r1 ip route get 192.168.2.10
docker exec clab-static-tri-r1 ip link set eth1 up
```

Ensure r3 and r2 have consistent return paths for the backup trajectory.

Return path drill (mandatory)

Inject on r2:

```
# remove route to h1 LAN
vtysh -c 'configure terminal' -c 'no ip route 192.168.1.0/24 10.0.12.1' -c 'end'
# or linux: ip route del 192.168.1.0/24
```

Predict: One-way behavior; h1→h2 may fail depending on path; debugging must check reverse.

Restore from config files + reload, not memory folklore.

Blackhole and reject

```
ip route add blackhole 10.66.0.0/16
# or
ip route add unreachable 10.66.0.0/16
```

Useful when summarizing: explicit sink for unallocated space you advertise.

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }

docker exec clab-static-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-static-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"
docker exec clab-static-tri-r1 vtysh -c 'show ip route' | grep -q 192.168.2.0 || fail "r1 route"

echo OK
```

Addressing.md excerpt

```
| Node | Iface | IP |
|-----|-----|-----|
| r1 | eth1 | 10.0.12.1/24 |
| r2 | eth1 | 10.0.12.2/24 |
| r1 | eth2 | 10.0.13.1/24 |
```

```
| r3 | eth2 | 10.0.13.2/24 |
| r2 | eth2 | 10.0.23.1/24 |
| r3 | eth1 | 10.0.23.2/24 |
| r1 | eth3 | 192.168.1.1/24 |
| r2 | eth3 | 192.168.2.1/24 |
| h1 | eth1 | 192.168.1.10/24 |
| h2 | eth1 | 192.168.2.10/24 |
```

Default routes at edge

Simulating “internet”:

```
# on r1
ip route 0.0.0.0/0 10.0.12.2
```

Only if r2 has a path onward. Avoid default loops (r1 defaults to r2, r2 defaults to r1).

IPv6 statics (optional parallel)

```
ipv6 route 2001:db8:2::/64 2001:db8:12::2

ip -6 route add 2001:db8:2::/64 via 2001:db8:12::2
```

Dual-stack the triangle when comfortable.

Common mistakes

Mistake	Symptom
Missing return route	One-way ping
Wrong next hop IP	Route present, unresolved neigh
Floating without interface down	No failover on silent blackhole
Overlapping LAN IPs	Chaos
Forgetting ip_forward	Router becomes host

Summary

- Static labs teach bidirectional design and verification

- Triangle + floating statics build failover intuition
- Interface shutdown is the honest failover inject
- FRR and Linux both work; keep configs in git
- verify.sh must test **both** directions

Next: **ICMP and path MTU**—when small pings lie and large packets die.

ICMP & Path MTU

ICMP & Path MTU

Small pings succeed while large transfers hang: classic **MTU / PMTUD** failure. This chapter treats ICMP as a control helper and shows you how to find black holes with deliberate packet sizes.

Learning goals

By the end of this chapter you can:

- Explain key ICMP types used in operations
- Test reachability with controlled sizes and DF bit
- Predict PMTUD behavior when ICMP “fragmentation needed” is filtered
- Clamp MTU on a lab link and measure the symptom
- Document a harden checklist for MTU in topologies

ICMP essentials (IPv4)

Type	Role
Echo request/reply	ping
Destination unreachable	no route, port unreachable, frag needed
Time exceeded	TTL expired (traceroute)
Redirect	host guidance (often ignored/noisy)

```
ping -c 3 10.0.0.1
ping -c 3 -s 1400 10.0.0.1
ping -M do -s 1472 10.0.0.1    # DF set; 1472 + 28 = 1500
```

`-M do` (Linux) sets Don't Fragment. Adjust payload so total IP packet hits MTU boundary.

IPv6 uses ICMPv6 extensively (ND, PMTUD); similar lab methods with `ping -6`.

Path MTU discovery (idea)

1. Sender assumes path MTU (often interface MTU)
2. Sends DF packets
3. Hop with smaller MTU sends ICMP frag needed + next-hop MTU
4. Sender caches smaller PMTU

If step 3 is **filtered**, large packets blackhole; small pings work.

Lab: MTU mismatch

```
h1 -- r1 ===== r2 -- h2
      MTU 1400 on r1-r2 link
```

Topology

```
name: mtu-lab

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iptables
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 192.168.1.1/24 dev eth1
        - ip addr add 10.0.0.1/24 dev eth2
        - ip link set eth1 up
        - ip link set eth2 up
        - ip link set eth2 mtu 1400
        - ip route add 192.168.2.0/24 via 10.0.0.2
    r2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iptables
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 192.168.2.1/24 dev eth1
        - ip addr add 10.0.0.2/24 dev eth2
        - ip link set eth1 up
```

```

- ip link set eth2 up
- ip link set eth2 mtu 1400
- ip route add 192.168.1.0/24 via 10.0.0.1
h1:
kind: linux
image: alpine:3.20
exec:
- apk add --no-cache iproute2 iputils
- ip addr add 192.168.1.10/24 dev eth1
- ip link set eth1 up
- ip route add default via 192.168.1.1
h2:
kind: linux
image: alpine:3.20
exec:
- apk add --no-cache iproute2 iputils
- ip addr add 192.168.2.10/24 dev eth1
- ip link set eth1 up
- ip route add default via 192.168.2.1

links:
- endpoints: ["h1:eth1", "r1:eth1"]
- endpoints: ["h2:eth1", "r2:eth1"]
- endpoints: ["r1:eth2", "r2:eth2"]

```

Predict

- `ping -c 2 192.168.2.10` small default works
- Large DF ping across path may fail or trigger PMTUD depending on sizes
- Capture may show ICMP unreachable frag needed if not filtered

Observe

```

docker exec clab-mtu-lab-h1 ping -c 2 192.168.2.10
docker exec clab-mtu-lab-h1 ping -M do -s 1472 -c 2 192.168.2.10
docker exec clab-mtu-lab-h1 ping -M do -s 1200 -c 2 192.168.2.10
docker exec clab-mtu-lab-r1 ip link show eth2

```

Inject: drop ICMP unreachable on r1

```
docker exec clab-mtu-lab-r1 iptables -A FORWARD -p icmp --icmp-type fragmentation-needed -j D
# also output path variants may need INPUT/OUTPUT depending on generation point
docker exec clab-mtu-lab-r1 iptables -A OUTPUT -p icmp --icmp-type fragmentation-needed -j D
```

Retest large DF pings / large TCP if iperf available.

Predict: hard blackhole for large DF packets.

Restore

```
docker exec clab-mtu-lab-r1 iptables -F
```

Harden

- Match MTUs both ends of a link
- Do not filter all ICMP on transit routers
- For TCP, MSS clamping on edges sometimes used as workaround—prefer fixing MTU/ICMP

```
# example TCP MSS clamp (awareness; use carefully)
iptables -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1360
```

Traceroute and ICMP filters

```
traceroute -n 192.168.2.10
```

If time-exceeded filtered, traceroute incomplete while data works. Journal both ICMP classes separately: **PMTUD** vs **traceroute**.

Capture workflow

```
docker exec clab-mtu-lab-r1 tcpdump -ni eth2 -c 40 icmp or '(ip[6:2] & 0x1fff) != 0'
docker exec clab-mtu-lab-h1 ping -M do -s 1472 -c 3 192.168.2.10
```

Look for:

- Echo requests size

- ICMP type 3 code 4 (frag needed)
- Whether sender reduces size after

iperf3 large transfers

```
# on h2
apk add iperf3
iperf3 -s
# on h1
iperf3 -c 192.168.2.10
```

Compare before/after MTU clamp and ICMP drop. TCP may stall if PMTUD blackhole.

IPv6 notes

IPv6 routers do not fragment in the same way; PMTUD is even more critical. Minimum link MTU is 1280.

```
ping -6 -M do -s 1400 <dst>
ip -6 route get <dst>
```

Operational checklist

Check	Command / action
Interface MTU	<code>ip link</code>
End-to-end small ping	<code>ping -c 3</code>
DF large ping	<code>ping -M do -s ...</code>
ICMP allowed	policy review
TCP test	<code>iperf3</code> / <code>curl</code> large
Tunnel overhead	account for encapsulation

Tunnels (later chapters) reduce effective MTU—plan overhead.

Predict worksheet

Path MTU 1500 except one hop 1400; ICMP filtered.

1. Ping default size 56 payload — ?

2. DF packet size 1500 — ?

3. TCP bulk transfer — ?

Answers: (1) usually ok; (2) blackhole; (3) often hang or extreme slowness depending on stack workarounds.

verify.sh additions

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-mtu-lab-h1 ping -c 2 -W 1 192.168.2.10
# expect success for moderate DF size under 1400 path
docker exec clab-mtu-lab-h1 ping -M do -s 1200 -c 2 -W 1 192.168.2.10
echo OK
```

Optional: assert large 1472 DF fails when MTU 1400—can be a negative test in failure mode only.

Summary

- ICMP is part of healthy networks (PMTUD, traceroute), not pure “attack surface” to zero
- Size-controlled pings expose MTU issues small pings hide
- Filtering frag-needed creates blackholes
- Match link MTUs; account for encapsulation overhead
- Always pair ping with a bulk TCP test when performance is in question

Next part: **interior routing**—OSPF and BGP basics plus policy/filters on free FRR images.

Interior Routing

OSPF Basics

OSPF Basics

OSPF is a **link-state IGP**: routers flood topology information within an area, run SPF, and install routes. This chapter builds a single-area triangle on FRR in Containerlab—the highest-ROI dynamic routing lab in the book.

Learning goals

By the end of this chapter you can:

- Contrast link-state vs distance-vector at a model level
- Bring up OSPF adjacencies on FRR and read neighbor states
- Advertise loopbacks and LANs into OSPF
- Predict reconvergence when a link costs out or dies
- Verify RIB/FIB and traceroute paths

Concepts

Distance-vector vs link-state

	Distance-vector (idea)	Link-state (OSPF)
What is shared	Vectors to prefixes	Topology (LSAs)
Loop risk	Needs careful rules	SPF on common map
Visibility	Partial	Area-wide topology view
Example	Classic RIP	OSPF, IS-IS

OSPF adjacency state machine (short)

Down → Init → 2-Way → ExStart → Exchange → Loading → **Full**

For learning: **Full** with expected neighbors on each transit link. Stuck states often mean MTU mismatch, authentication, layer-2, or network type mismatch.

Areas

Single-area (0) is enough here. Multi-area adds ABR summarization later in your journey; do not rush.

Cost

OSPF picks lowest cost path. Cost often derives from reference bandwidth / interface bandwidth. In FRR you can set `ip ospf cost` on interfaces for drills.

Network types (awareness)

Point-to-point vs broadcast affects adjacency count and DR/BDR. P2P links in labs are simple; Ethernet multipoint may elect DR.

Addressing plan

Node	lo	eth1 (to peer)	eth2	LAN eth3
r1	1.1.1.1/32	10.0.12.1/24 (r2)	10.0.13.1/24 (r3)	192.168.1.1/24
r2	2.2.2.2/32	10.0.12.2/24	10.0.23.1/24	192.168.2.1/24
r3	3.3.3.3/32	10.0.23.2/24	10.0.13.2/24	optional

Hosts: 192.168.1.10, 192.168.2.10 with defaults to local r.

Topology YAML

```
name: ospf-tri

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
```

```

kind: linux
image: quay.io/frrouting/frr:10.2.1
binds:
  - ./config/r3:/etc/frr
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
    - ip route add default via 192.168.1.1
h2:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
    - ip route add default via 192.168.2.1

links:
  - endpoints: ["r1:eth1", "r2:eth1"]
  - endpoints: ["r2:eth2", "r3:eth1"]
  - endpoints: ["r3:eth2", "r1:eth2"]
  - endpoints: ["h1:eth1", "r1:eth3"]
  - endpoints: ["h2:eth1", "r2:eth3"]

```

FRR daemons

daemons file must enable:

```

zebra=yes
ospfd=yes

```

FRR OSPF config (r1 example)

```

frr version 10.2.1
frr defaults traditional
hostname r1
!
interface lo
 ip address 1.1.1.1/32
 ip ospf area 0
!

```

```

interface eth1
  ip address 10.0.12.1/24
  ip ospf area 0
!
interface eth2
  ip address 10.0.13.1/24
  ip ospf area 0
!
interface eth3
  ip address 192.168.1.1/24
  ip ospf area 0
!
router ospf
  ospf router-id 1.1.1.1
!
line vty

```

Advertise by putting interfaces in area 0 (common FRR style). Equivalent network statements exist; be consistent across nodes.

r2/r3: unique router-ids 2.2.2.2, 3.3.3.3; matching interface IPs; LANs in OSPF so remote LANs appear in RIB.

Deploy and verify

```

sudo containerlab deploy -t ospf-tri.clab.yml

docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf route'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf database'

docker exec clab-ospf-tri-h1 ping -c 3 192.168.2.10
docker exec clab-ospf-tri-h1 traceroute -n 192.168.2.10

```

Predict

- Each router: 2 neighbors Full (triangle)
- RIB contains remote LAN and loopbacks via OSPF
- Path prefers lower cost sides

Neighbor not Full?

Check:

```
ip link
ip addr
# MTU match
vtysh -c 'show ip ospf interface'
# timers, area mismatch, passive interfaces accidental
```

Cost change drill

```
# on r1 eth1 increase cost so traffic prefers other way
docker exec -it clab-ospf-tri-r1 vtysh
configure terminal
interface eth1
  ip ospf cost 1000
end
write
# promote back to frr.conf in git after experiment

docker exec clab-ospf-tri-r1 vtysh -c 'show ip route 192.168.2.0/24'
docker exec clab-ospf-tri-h1 traceroute -n 192.168.2.10
```

Predict: path shifts if alternate exists with lower total cost.

Link failure drill

```
docker exec clab-ospf-tri-r1 ip link set eth1 down
sleep 5
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-ospf-tri-h1 ping -c 20 192.168.2.10
docker exec clab-ospf-tri-r1 ip link set eth1 up
```

Journal reconvergence behavior and whether ping loss matches expectations.

Passive interfaces / LAN notes

Sometimes you put a LAN in OSPF for advertisement but want **no** adjacency on user ports:

```
router ospf
  passive-interface eth3
```

Loopbacks are often passive. Transit links remain non-passive.

Verification checklist

Check	Command
Neighbors	<code>show ip ospf neighbor</code>
Database	<code>show ip ospf database</code>
OSPF RIB	<code>show ip ospf route</code>
Kernel/FIB	<code>show ip route / ip route</code>
Data plane	<code>ping / traceroute</code>

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }

docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor' | grep -q Full \
  || fail "no Full neighbor on r1"
docker exec clab-ospf-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-ospf-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"
echo OK
```

Common mistakes

Mistake	Symptom
ospfd not enabled	no neighbors
Mismatched areas	stuck
Different IP subnets on link	no adjacency
MTU mismatch	ExStart/Exchange issues
router-id collisions	confusing DB
Forgetting LAN in OSPF	hosts missing remote routes

Beyond this chapter

- Multi-area + summarization at ABR
- Stub/NSSA ideas
- OSPFv3 for IPv6
- Authentication

Master single-area Full adjacencies and failover first.

Summary

- OSPF floods link state, computes SPF, installs routes
- FRR triangle lab is the default IGP workout
- Always verify neighbors Full **and** FIB **and** ping
- Cost and link-down drills teach reconvergence
- Keep configs in binds; promote experimental vtysh changes to git

Next: **BGP basics**—sessions, path selection process, and a dual-router lab without vendor exam noise.

BGP Basics

BGP Basics

BGP is the inter-domain workhorse and also appears inside modern DC fabrics. This chapter stays vendor-agnostic and free-image-only: **eBGP and iBGP ideas**, sessions, advertisements, and path selection at a practitioner level—implemented with FRR.

Learning goals

By the end of this chapter you can:

- Explain ASNs, eBGP vs iBGP roles
- Bring up a BGP session on FRR to Established
- Advertise a prefix and see it on the peer
- Apply the path selection process at a high level
- Break a session and observe withdrawal

Concepts

Autonomous systems

An **AS** is a policy domain with an ASN. Labs use private ASNs:

Range	Notes
64512–65534	16-bit private
4200000000+	32-bit private space (use docs for exact ranges)

Example: AS65001 and AS65002 as “two sites” or “customer and ISP.”

eBGP	iBGP
------	------

eBGP vs iBGP

	eBGP	iBGP
Between	Different ASNs	Same ASN
Typical TTL	1 (direct)	multi-hop ok with care
Default next-hop	Peer address	Needs next-hop-self often for reachability
Scale	Edge policy	Route reflection / confed for large iBGP

Session state

Idle → Connect → Active → OpenSent → OpenConfirm → **Established**

Only Established exchanges routes.

Path selection overview

BGP is policy-heavy. Simplified order of preference ideas:

1. Prefer higher **local preference** (inbound influence)
2. Prefer shorter **AS path**
3. Origin, MED, eBGP over iBGP, IGP metric to next hop, router-id, etc.

You will not memorize every tie-breaker today—**know that policy beats pure shortest path.**

NLRI and attributes

Routes carry attributes (AS_PATH, NEXT_HOP, LOCAL_PREF, MED, communities...). Filters rewrite attributes; that is policy.

Lab A: two AS eBGP

```
h1--r1 (AS65001) ----- r2 (AS65002)--h2
      10.0.0.1/30    10.0.0.2/30
```

Advertise LANs (or loopbacks) across eBGP.

Topology

```
name: bgp-ebgp

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["h1:eth1", "r1:eth2"]
    - endpoints: ["h2:eth1", "r2:eth2"]
```

daemons

```
zebra=yes
bgpd=yes
```

r1 frr.conf

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.0.0.1/30
!
interface eth2
 ip address 192.168.1.1/24
!
interface lo
 ip address 1.1.1.1/32
!
router bgp 65001
 bgp router-id 1.1.1.1
 no bgp ebgp-requires-policy
 neighbor 10.0.0.2 remote-as 65002
!
 address-family ipv4 unicast
  network 192.168.1.0/24
  network 1.1.1.1/32
  neighbor 10.0.0.2 activate
 exit-address-family
!
line vty
```

no bgp ebgp-requires-policy eases labs on modern FRR that otherwise require explicit export policy. In real life, **prefer explicit policies** (next chapter).

r2

ASN 65002, IP 10.0.0.2/30, LAN 192.168.2.0/24, networks advertised symmetrically, neighbor 10.0.0.1 remote-as 65001.

Verify session

```
sudo containerlab deploy -t bgp-ebgp.clab.yml
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp ipv4 unicast'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show ip route'
docker exec clab-bgp-ebgp-h1 ping -c 3 192.168.2.10
```

Predict

- `show bgp summary` state Established, prefixes received > 0
- Kernel has remote LAN via BGP next hop 10.0.0.2
- Ping works both ways

Lab B: iBGP sketch (optional same day)

Two routers same ASN, loopbacks as session endpoints, IGP or statics for loopback reachability first.

```
router bgp 65001
 neighbor 2.2.2.2 remote-as 65001
 neighbor 2.2.2.2 update-source lo
 neighbor 2.2.2.2 next-hop-self
```

Rule: iBGP needs **underlying reachability** to the session address. OSPF underlay + iBGP overlay is a common pattern (underlay vs overlay planes!).

Session failure drill

```
docker exec clab-bgp-ebgp-r1 ip link set eth1 down
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show ip route 192.168.1.0'
# routes withdraw after hold timers - observe timing
docker exec clab-bgp-ebgp-r1 ip link set eth1 up
```

Predict: session drops; prefixes withdrawn; ping fails until re-established.

Optional: `neighbor x shutdown` in vtysh for cleaner control-plane inject.

Next-hop reachability drill

Install BGP route whose next hop is unreachable (misconfigured).

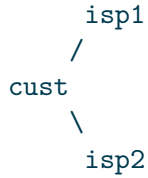
Symptom: BGP table shows path but not installed in FIB / not usable.

```
vtysh -c 'show bgp ipv4 unicast'
vtysh -c 'show ip route'
```

Compare “known in BGP” vs “installed.”

Dual-home teaser

Customer with two eBGP uplinks steers with local-pref / AS-path prepend—full treatment lives in policy chapter. Topology idea:



verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }

docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary' | grep -q Established \
|| fail "session not Established"
docker exec clab-bgp-ebgp-h1 ping -c 2 -W 1 192.168.2.10 || fail "ping"
docker exec clab-bgp-ebgp-h2 ping -c 2 -W 1 192.168.1.10 || fail "reverse ping"
echo OK
```

Safety habits

- Do not experiment with BGP on real Internet edges without change control
- Private ASNs in labs only unless you know what you are doing
- Explicit export policy in anything beyond throwaway labs
- Cap lab prefixes; never leak real full tables into tiny FRR VMs by accident

Common mistakes

Mistake	Result
wrong remote-as	session fails
no underlay to iBGP peer	idle/active
missing activate / AF	Established but no routes
ebgp-requires-policy deny	no prefixes without route-maps
next hop unreachable	BGP path not used

Summary

- BGP sessions must be Established before routes flow
- eBGP links ASNs; iBGP needs underlay reachability
- Path selection is policy-centric, not pure SPF
- FRR lab: advertise LANs, ping through, shut session
- Prefer explicit policies as you leave pure lab mode

Next: **policy and filters**—prefix lists, route-maps-as-idea, and preventing loops/leaks.

Policy & Filters

Policy & Filters

Routing without policy is a demo; routing with policy is operations. This chapter covers **prefix filters, distribute ideas, route-maps as a pattern, and loop-prevention instincts** using FRR—still free and vendor-agnostic.

Learning goals

By the end of this chapter you can:

- Write prefix-lists that match intended space only
- Apply import/export filters on BGP neighbors
- Use route-map style logic (match → set → permit/deny)
- Predict leaks and loops from redistribution mistakes
- Build a lab that rejects unauthorized prefixes

Why policy exists

Goal	Mechanism ideas
Do not accept bogons / your own space from customers	Inbound prefix filter
Do not advertise internal-only nets to Internet	Outbound filter
Prefer one uplink	local-pref on import
De-prefer backup path	AS-path prepend on export
Redistribute without feedback loops	Tags, filters, distance

Prefix lists (FRR)

```
ip prefix-list OWN-NETS seq 10 permit 192.168.1.0/24
ip prefix-list OWN-NETS seq 20 permit 1.1.1.1/32
ip prefix-list OWN-NETS seq 30 deny any
```

```
ip prefix-list CUST-ALLOW seq 10 permit 198.51.100.0/24
ip prefix-list CUST-ALLOW seq 20 deny any
```

Order matters: first match wins. Always know your implicit/default at the end.

Route-maps as idea

Regardless of vendor syntax, the pattern is:

```
if match prefix-list X:
    set local-preference 200
    permit
else deny
```

FRR example:

```
route-map FROM-CUST permit 10
    match ip address prefix-list CUST-ALLOW
    set local-preference 200
route-map FROM-CUST deny 99
!
route-map TO-CUST permit 10
    match ip address prefix-list OWN-NETS
```

Attach:

```
router bgp 65002
    neighbor 10.0.0.1 route-map FROM-CUST in
    neighbor 10.0.0.1 route-map TO-CUST out
```

Lab: filter unauthorized prefixes

Extend eBGP lab: r2 (ISP) should **only** accept 192.168.1.0/24 from r1 (customer), not a hijack prefix.

Customer r1 advertises extra junk

```
address-family ipv4 unicast
  network 192.168.1.0/24
  network 203.0.113.0/24
```

ISP r2 policy

```
ip prefix-list CUST-R1 seq 5 permit 192.168.1.0/24
ip prefix-list CUST-R1 seq 10 deny any
!
route-map FROM-R1 permit 10
  match ip address prefix-list CUST-R1
route-map FROM-R1 deny 20
!
router bgp 65002
  neighbor 10.0.0.1 route-map FROM-R1 in
```

Predict

- ISP installs 192.168.1.0/24
- 203.0.113.0/24 is rejected
- `show bgp` on r2 lacks the junk

Observe

```
docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show ip route 203.0.113.0'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp ipv4 unicast neighbors 10.0.0.2 advertised-r
```

Harden

Return to explicit export on customer too—only advertise owned space.

```
route-map EXPORT-OWN permit 10
  match ip address prefix-list OWN-NETS
!
neighbor 10.0.0.2 route-map EXPORT-OWN out
```

Remove `no bgp ebgp-requires-policy` when policies are in place if you want FRR to enforce the discipline.

Local-pref steer drill

Two ISPs: isp1 and isp2; customer prefers isp1 for inbound to customer... actually:

- **Local pref** (on customer) influences **outbound** exit from customer AS
- **AS prepend** / **MED** / **communities** influence how others send **inbound**

Outbound preference (customer)

```
route-map PREFER-ISP1 permit 10
  set local-preference 300
route-map PREFER-ISP2 permit 10
  set local-preference 100
!
neighbor <isp1> route-map PREFER-ISP1 in
neighbor <isp2> route-map PREFER-ISP2 in
```

Predict: traffic leaves via isp1 when both paths exist.

```
vtysh -c 'show ip route 0.0.0.0'
vtysh -c 'show bgp ipv4 unicast'
```

Redistribution loop awareness

Redistributing OSPF BGP (or static OSPF) without filters can:

- Re-inject prefixes with changed metrics
- Create loops or suboptimal paths
- Explode tables

Rules of thumb:

1. Redistribute **on purpose**, on as few points as possible
2. Tag routes; filter on tags
3. Never blindly mutual redistribute two IGPs
4. Prefer BGP for policy domains, IGP for underlay topology

Example static→OSPF with filter (sketch)

```
ip prefix-list STATICS seq 5 permit 192.168.9.0/24
route-map REDIST-STATIC permit 10
  match ip address prefix-list STATICS
!
router ospf
  redistribute static route-map REDIST-STATIC
```

ACL-style packet filters vs routing policy

Do not confuse:

Routing policy	Packet filter
Which routes enter RIB BGP/OSPF route-maps Control plane	Which packets pass nftables/iptables/ACLs Data plane

Both appear in hardening. A permit route does not mean packets are allowed through a firewall mid-path.

Distribute-list / filter-list ideas

Older or alternate forms filter by ACL or AS-path lists:

```
bgp as-path access-list 1 permit _65001$
neighbor x filter-list 1 in
```

Same intent: **accept only what policy allows**.

Failure drills

Drill 1 — over-broad permit any

Export **permit any** to a lab “ISP.”

Observe: internal lab nets leave the AS.

Fix: prefix-list OWN only.

Harden: default-deny export mindset.

Drill 2 — accidental deny all

Route-map deny first line matches everything.

Symptom: Established but zero prefixes.

Debug: `show bgp neighbors ... routes / policy counters` if available; simplify map.

Drill 3 — next-hop-self missing (iBGP)

Learned eBGP route reflected iBGP with remote next hop unreachable.

Symptom: BGP has path; FIB lacks usable route.

Fix: `next-hop-self` or IGP reachability to next hop.

verify.sh policy checks

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }

docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast' | grep -q '192.168.1.0/24' \
    || fail "missing customer prefix"
if docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast' | grep -q '203.0.113.0/24';
    fail "hijack prefix should be filtered"
fi
echo OK
```

Policy design worksheet

Before applying:

```
## Neighbor: r1-r2 eBGP
Import accept: ...
Import deny: ...
Export advertise: ...
Export never: ...
Preference intent: ...
Failure mode if map wrong: ...
```

Summary

- Policy decides which routes and with which attributes
- Prefix-lists + route-maps are the portable pattern
- Default-deny posture on Internet edges
- Redistribution needs filters/tags or it will bite
- Verify with BGP table **and** negative tests for rejected prefixes

Next part: **ops & automation**—observability, lab-as-code automation, and a capstone outline that pulls the book together.

Ops and Automation

Observability for Networks

Observability for Networks

If you cannot see it, you cannot operate it. This chapter turns lab verification habits into a minimal **observability stack**: interface signals, routing state, captures, logs, and a path toward metrics/telemetry—without requiring paid NMS.

Learning goals

By the end of this chapter you can:

- Build a layered observability checklist for any lab or small network
- Use Linux and FRR show/counters for incident triage
- Capture and filter packets productively
- Sketch metrics that matter (link, session, error)
- Write a short runbook entry from an observed failure

Observability layers

Layer	Questions	Tools
L1/L2 link	Up? errors? flaps?	<code>ip -s link</code> , ethtool, MAC/FDB
L3 reachability	Path? loss?	ping, mtr, traceroute
Control plane	Neighbors/sessions?	vttysh shows, logs
Data plane path	Which hop?	route get, capture
Services	App timeouts?	ss, curl, synthetic checks
Continuity	When did it start?	logs, metrics, journals

Baseline: what “healthy” looks like

Before incidents, save baselines:

```
mkdir -p baseline
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor' > baseline/r1-ospf-nbr.txt
```

Three planes of a network device

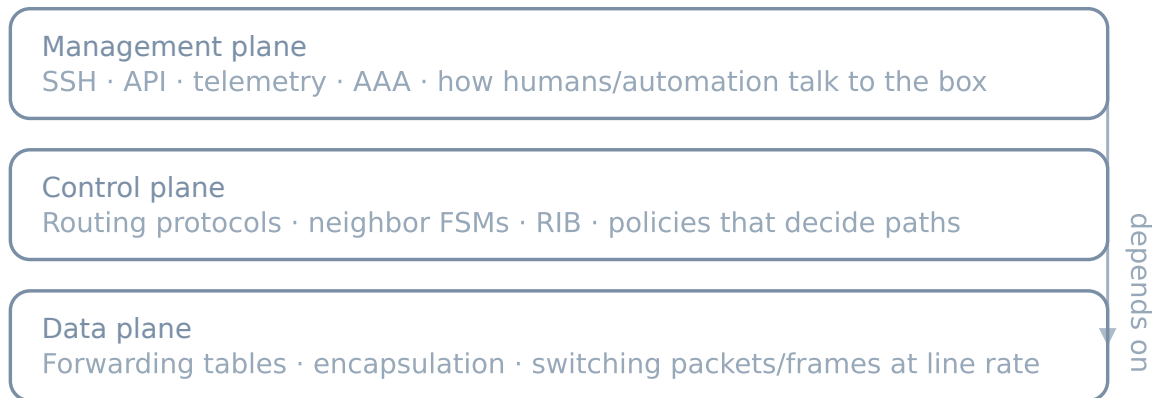


Figure 1: Planes

```
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route' > baseline/r1-route.txt
docker exec clab-ospf-tri-r1 ip -s link > baseline/r1-link.txt
```

Diff during incidents:

```
diff -u baseline/r1-ospf-nbr.txt <(docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neigh
```

Interface and error signals

```
ip -s -s link show eth1
# Look at RX/TX errors, drops, carrier changes
```

Drill: bounce a link; confirm counters and oper state track reality.

```
ip monitor link
```

Control-plane signals

OSPF

```
vtysh -c 'show ip ospf neighbor'
vtysh -c 'show ip ospf interface'
vtysh -c 'show log' # if logging configured
```

BGP

```
vttysh -c 'show bgp summary'
vttysh -c 'show bgp neighbors 10.0.0.2'
vttysh -c 'show bgp neighbors 10.0.0.2 routes'
```

Watch for: session flaps, prefix count drops to zero, unexpected AS paths.

Path quality

```
ping -c 50 -i 0.2 192.168.2.10
mtr -rwzc 100 192.168.2.10
traceroute -n 192.168.2.10
```

Interpret:

Pattern	Suspect
Loss from first hop	Local segment
Loss starting mid-path	Transit / congestion / ACL
High latency one hop jump	That hop or link serialization
Jittery wireless-like	Lab CPU contention sometimes!

On shared lab hosts, **CPU starvation** fakes network loss—check `docker stats`.

Packet capture practice

```
tcpdump -ni eth1 -w /tmp/inc.pcap not port 22
# reproduce
tcpdump -r /tmp/inc.pcap -nn
tshark -r /tmp/inc.pcap -Y 'icmp || bgp || arp' -V | less
```

Capture points

Question	Where to capture
Did host send?	Host interface
Did router rewrite L2?	Router egress
Did peer receive?	Peer ingress
Policy drop?	Both sides + counters

Logging

FRR

Enable meaningful logging to file or syslog inside the node (image-dependent):

```
log file /var/log/frr/frr.log
log timestamp precision 3
```

```
docker exec clab-ospf-tri-r1 tail -f /var/log/frr/frr.log
```

Kernel

```
dmesg -T | tail
# martians, neighbor table full, etc.
```

Metrics sketch (Prometheus-shaped thinking)

Even if you do not run Prometheus yet, define signals:

Metric idea	Type	Labels
link_up	gauge	node, iface
interface_errors_total	counter	node, iface, dir
bgp_session_up	gauge	node, peer
bgp_prefixes_received	gauge	node, peer
ospf_neighbors_full	gauge	node
ping_loss_ratio	gauge	src, dst

Synthetic ping exporter + node exporter covers a surprising amount for home/lab NetOps.

Minimal blackbox idea

```
#!/usr/bin/env bash
# probe.sh - exit 1 on failure for CI/cron
ping -c 3 -W 1 192.168.2.10 >/dev/null
```

Cron or systemd timer + alert on fail is valid ops.

Telemetry awareness (2026)

Modern NOS (e.g. SR Linux community) emphasize streaming telemetry (gNMI). FRR/Linux labs may stick to show + exporters. Skill transfer:

- **Subscribe** to interface + protocol state
- Prefer push metrics over SSH scraping when scale grows
- Still keep packet capture for truth

Incident timeline template

```
## Incident
Start (UTC):
Detect signal:
Impact:
## Timeline
- T0:
- T+2m:
## Hypotheses
1.
2.
## Checks performed
## Root cause
## Fix
## Follow-ups
- monitor:
- prevent:
```

Lab drill: observe a BGP flap

1. Baseline `show bgp summary`
2. `neighbor shutdown` or link down
3. Record timestamps of state change and route withdrawal
4. Restore; measure time to Established and ping recovery
5. Write incident note with detection idea (“alert if `bgp_session_up==0` for 60s”)

Lab drill: silent MTU blackhole

From the MTU chapter: large ping fails, small works.

Observability angle: which metric would catch this? (not `link_down`). Need synthetic large-packet probe or TCP service check.

Runbook snippet library

Keep short runbooks next to labs:

```
## Runbook: OSPF neighbors down
1. Check link state ip link
2. Check addressing same subnet
3. show ip ospf interface MTU
4. capture hello packets
5. check area / auth / passive
```

Privacy and safety

- Captures may contain payloads—store carefully
- Do not tcpdump production spans without authorization
- Redact journals before sharing

Summary

- Observability is layered: link → path → control → packet → service
- Baselines make incidents measurable
- Counters + mtr + captures beat vibes
- Define metrics before you buy platforms
- Runbooks convert personal heroics into repeatable ops

Next: **automation** / **lab-as-code**—pushing configs, smoke deploys, and CI-shaped checks.

Automation & Lab-as-Code

Automation & Lab-as-Code

A lab you cannot rebuild is a rumor. This chapter automates the Containerlab lifecycle, config push patterns, and smoke verification so your networking skill compounds in git.

Learning goals

By the end of this chapter you can:

- Wrap deploy/destroy/verify in repeatable scripts
- Organize inventories for multi-node labs
- Push or render configs with light Ansible or shell
- Sketch CI that smoke-tests a topology
- Treat secrets and image pins correctly

Principles

1. **Topology and config in git**
2. **Idempotent as practical** — re-run without fear
3. **Verify automatically** — exit codes matter
4. **No click-ops source of truth**
5. **Small steps** — shell first, Ansible when it pays off

Directory layout (automation-ready)

```
labs/ospf-tri/  
  topology.clab.yml  
  addressing.md  
  README.md
```

```
Makefile
verify.sh
up.sh
down.sh
config/
  r1/frr.conf
  r1/daemons
  r2/...
ansible/
  inventory.yml
  playbooks/deploy-config.yml
```

Makefile pattern

```
TOPO := topology.clab.yml

.PHONY: up down verify smoke

up:
  sudo containerlab deploy -t $(TOPO)

down:
  sudo containerlab destroy -t $(TOPO) --cleanup

verify:
  ./verify.sh

smoke: up verify
  @echo smoke ok

make smoke
make down
```

up.sh / down.sh

```
#!/usr/bin/env bash
# up.sh
set -euo pipefail
cd "$(dirname "$0")"
sudo containerlab deploy -t topology.clab.yml
./verify.sh
```

```
#!/usr/bin/env bash
# down.sh
set -euo pipefail
cd "$(dirname "$0")"
sudo containerlab destroy -t topology.clab.yml --cleanup
```

Config strategies

Strategy	Pros	Cons
Bind-mount <code>/etc/frr</code>	Simple, git-native	Image load quirks
<code>containerlab startup-config</code>	Kind-supported	Kind-specific
Ansible after deploy	Flexible templating	Extra tool
Render templates → bind	Pure git artifacts	Build step

Prefer bind + pure files for FRR until pain appears.

Safe interactive explore → promote

```
docker exec -it clab-ospf-tri-r1 vtysh
# make change, show run
# copy running config back into config/r1/frr.conf
# redeploy clean to prove
```

Light Ansible (optional)

ansible/inventory.yml:

```
all:
  children:
    routers:
      hosts:
        r1:
          ansible_host: clab-ospf-tri-r1
          ansible_connection: docker
        r2:
          ansible_host: clab-ospf-tri-r2
          ansible_connection: docker
        r3:
          ansible_host: clab-ospf-tri-r3
          ansible_connection: docker
```

Playbook sketch:

```
- name: Push FRR config files
  hosts: routers
  gather_facts: false
  tasks:
    - name: Copy frr.conf
      copy:
        src: "{{ playbook_dir }}/../../config/{{ inventory_hostname }}/frr.conf"
        dest: /etc/frr/frr.conf
    - name: Reload FRR
      shell: vtysh -c 'write' || true
      # better: proper frr reload unit - image dependent
```

```
ansible-playbook -i ansible/inventory.yml ansible/playbooks/deploy-config.yml
```

Python with scrapli/netmiko is equally valid; pick one tool family and go deep later.

Templating addressing

addressing.yml:

```
links:
  r1_r2: { r1: 10.0.12.1/24, r2: 10.0.12.2/24 }
lans:
  h1: { gw: 192.168.1.1/24, host: 192.168.1.10/24 }
```

Render `frr.conf.j2` → `config/r1/frr.conf` with a small Python/Jinja script or Ansible template module. **Generated files can be committed** for diff visibility or generated in CI—pick one workflow and document it.

verify.sh mature pattern

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }

need_full() {
  local node=$1
  docker exec "$node" vtysh -c 'show ip ospf neighbor' | grep -q Full \
    || fail "$node missing Full neighbor"
}

need_full clab-ospf-tri-r1
```

```

need_full clab-ospf-tri-r2
need_full clab-ospf-tri-r3

docker exec clab-ospf-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-ospf-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"

echo "OK $(date -Is)"

```

CI smoke (GitHub Actions sketch)

Conceptual job (runner must support Docker + Containerlab—often self-hosted):

```

# conceptual - adapt to your runners
jobs:
  lab-smoke:
    runs-on: self-hosted-linux
    steps:
      - uses: actions/checkout@v4
      - name: deploy+verify
        working-directory: books/Networking/labs/ospf-tri
        run: |
          sudo containerlab deploy -t topology.clab.yml
          ./verify.sh
      - name: cleanup
        if: always()
        working-directory: books/Networking/labs/ospf-tri
        run: sudo containerlab destroy -t topology.clab.yml --cleanup

```

This book's monorepo CI renders Quarto; **lab smoke is optional local/self-hosted**. Do not assume GitHub-hosted runners allow nested Containerlab without setup.

Image pins in automation

```

# .env or Makefile
FRR_IMAGE=quay.io/frrouting/frr:10.2.1
ALPINE_IMAGE=alpine:3.20

```

Topology can be generated or documented to match. Record digests for serious CI.

Idempotency drills

1. make smoke twice without down — should not corrupt

2. `make down && make smoke` — clean path
3. Break config, `make smoke` fails `exit 0`
4. Fix config, smoke passes

Secret hygiene

- No production passwords in lab repos
- Lab default creds documented as lab-only
- Use `.gitignore` for private env files

Graph and inventory export

```
sudo containerlab inspect -t topology.clab.yml --format json > inspect.json
sudo containerlab graph -t topology.clab.yml || true
```

Feed inspect outputs into your own tools later.

Predict → observe → fix

Predict: A peer clones the lab folder, runs `make smoke`, gets OK without chatting you.

Observe: Remove tribal knowledge; only README + scripts.

Fix: Missing apk packages, wrong container names, unpinned images.

Harden: Add `make smoke` badge or CI on self-hosted when ready.

Anti-patterns

Anti-pattern	Replace with
Only screenshot proof	<code>verify.sh</code>
Snowflake node edits	<code>bind</code> + <code>git</code>
<code>latest</code> tags	<code>pins</code>
Huge mono playbook day one	<code>Makefile smoke</code>
CI without cleanup	<code>always destroy</code>

Summary

- Lab-as-code = topology + config + verify in git
- Start with Makefile/scripts; add Ansible when templating hurts
- Exit codes turn labs into testable systems
- CI is optional but shaping; cleanup always
- Promote working state from nodes back into the repo

Next: **capstone outline**—projects that prove the whole journey.

Capstone Outline

Capstone Outline

This chapter is a **map of finishing projects**, not a single mandatory mega-lab. Pick at least one capstone and run it to the book’s definition of done: design, code, break, prove, operate.

Learning goals

By the end of this chapter you can:

- Choose a capstone that matches your goals
- Scope addressing, topology, and success metrics
- Apply models + verification + automation end-to-end
- Produce a portfolio-quality lab folder
- Identify natural next topics beyond this spine

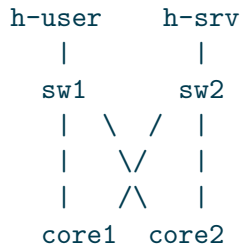
Definition of done (all capstones)

- ☐ Written design (1–2 pages): goals, non-goals, addressing, failure modes
- ☐ `topology.clab.yml` + configs in git
- ☐ `addressing.md` + diagram
- ☐ `verify.sh` green on clean deploy
- ☐ 3 failure drills with incident notes
- ☐ One automation wrapper (`make smoke` or equivalent)
- ☐ Short retrospective: what model changed in your head

Capstone C1 — Campus-style dual core (open images)

Intent

Two access switches (or Linux bridges), dual core routers, dual-homed access, VLANs for users/servers, OSPF underlay between cores, static or OSPF toward access.



Skills exercised

- VLANs/trunks
- L2/L3 boundary
- OSPF or static redundancy
- Failure: core link down, trunk VLAN missing

Success metrics

- Hosts in VLAN 10 communicate across access
- Inter-VLAN via cores
- One core or one uplink loss recovers within your documented budget

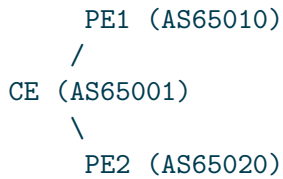
Suggested stack

FRR cores + Alpine hosts + Linux bridge “switches” or SR Linux if you want NOS practice.

Capstone C2 — Dual-homed site to provider edge

Intent

Customer edge (CE) with two eBGP uplinks to two “ISP” PE routers; prefer primary; failover to backup; filter what you advertise/accept.



Skills exercised

- eBGP sessions
- Prefix filters / route-maps
- Local-pref / prepend
- Session loss drill

Success metrics

- Only owned prefixes exported
- ISP rejects hijack test prefix
- Traffic shifts on primary PE failure

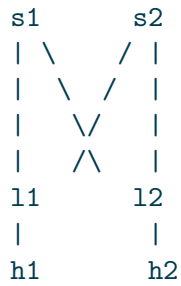
verify extras

Negative test: unauthorized prefix must not appear on PE.

Capstone C3 — Mini fabric snack

Intent

2 spine + 2 leaf (or 2×2) IP fabric with OSPF or eBGP underlay; endpoints on leaves; break a leaf uplink.



Skills exercised

- Underlay design
- ECMP behavior
- Scale of configs (templating pays off)
- Observability under failure

Stretch (optional)

Add a simple tunnel/overlay between leaves **after** underlay is solid—never before.

Capstone C4 — Incident week

Intent

Take any working lab (C1–C3 or static triangle). Each day for five days inject a **hidden** fault (or swap with a peer) and practice detect → diagnose → fix → prevent.

Day	Example inject
1	Wrong VLAN on one access port
2	Missing return route
3	MTU clamp + ICMP filter
4	OSPF cost surprise / link flap
5	BGP filter too strict

Skills exercised

- Observability under time pressure
- Incident notes
- Avoiding random change storms

Success metrics

- Each day: root cause written with evidence
 - verify.sh restored green
 - One new automated check added by end of week
-

How to pick

Goal	Pick
Campus / enterprise ops	C1
Edge / multihoming	C2
DC-ish fabric thinking	C3
Diagnostic mastery	C4 (optionally + another)

Time box: **1–2 weeks evenings** per capstone at 5–8h/week pace.

Project template (copy)

```
capstones/c2-dual-home/  
  DESIGN.md  
  README.md  
  topology.clab.yml  
  addressing.md  
  verify.sh  
  Makefile  
  config/  
  journals/  
    drill-1.md  
    drill-2.md  
    drill-3.md
```

```
diagrams/  
RETRO.md
```

DESIGN.md skeleton

```
# Design: Dual-homed CE  
  
## Goals  
## Non-goals  
## Topology  
## Addressing  
## Control plane  
## Policy  
## Failure modes  
## Observability  
## Test plan
```

RETRO.md prompts

- Which plane failed most often in your drills?
- What did you overcomplicate?
- Which verify checks caught real bugs?
- What would you template next?

Integration with the book spine

Book part	Capstone use
Models	Language in DESIGN.md
Lab craft	Containerlab hygiene
L2	C1 trunks/VLANs
L3	All (addressing, static edges)
Interior routing	C1 OSPF, C2 BGP, C3 underlay
Ops automation	Makefile, verify, journals

Beyond this book (next horizons)

When capstones feel comfortable:

- Multi-area OSPF / IS-IS
- EVPN/VXLAN on free images where feasible
- QoS classification labs
- Stronger automation (CI self-hosted, config generate)
- Streaming telemetry on SR Linux
- Traffic engineering and more advanced BGP

Stay open-tools-first unless your job provides licensed platforms.

Final checkpoint: “expert” as defined in the syllabus

You can:

- Design multi-site L2/L3 with clear underlay roles
- Implement in Containerlab with free images
- Diagnose control vs data failures with tables + captures
- Apply routing policy without creating loops
- Automate lab lifecycle and basic validation
- Operate with observability and recovery habits

If any bullet is weak, return to that part’s drills—not to random new features.

Summary

- Capstones prove integration, not trivia
- C1 campus, C2 dual-home BGP, C3 fabric snack, C4 incident week
- Same definition of done for all: design, code, verify, break, document
- Automation and negative tests separate portfolio work from toy demos
- The journey map continues; the habits stay

Return to the syllabus for the full long-range map (overlays, QoS, multi-area depth). Your next commit should be a capstone folder, not another unread PDF.

Edge and Services

First-Hop Redundancy

First-Hop Redundancy

Hosts rarely run a routing protocol. They send traffic to a **default gateway**. If that gateway dies, the LAN dies with it—unless you give the LAN a **virtual first hop** that can move between routers. This chapter builds the model and a free-stack lab: Linux + FRR + Containerlab (mid-2026 open path).

Learning goals

By the end of this chapter you can:

- Explain why a single gateway IP is a single point of failure
- Contrast VRRP-class first-hop redundancy with floating statics and ECMP
- Deploy a dual-router VRRP pair on FRR for a shared virtual IP (VIP)
- Predict host behavior when the master fails and when it recovers
- Verify with `ip neigh`, pings, and FRR VRRP state—not hope

Concepts

The problem

```
h1 ---- [LAN] ---- gw 10.0.0.1  (one box)
      |
      Internet-ish
```

Host config: `default via 10.0.0.1`. When that box reboots, ARP still points at a MAC that is gone until the host relearns—or traffic blackholes forever if nothing answers.

First-hop redundancy idea

Two (or more) routers share:

Object	Role
Virtual IP (VIP)	Address hosts use as gateway

Object	Role
Virtual MAC (often)	Stable L2 identity for the VIP
Master / backup roles	Who owns the VIP right now
Hello / advertise	“I am alive and I claim priority”

Hosts keep one gateway IP. The **active forwarder** changes under them.

Protocol family (vendor-agnostic)

Name	Notes
VRRP	Open standard idea; FRR supports VRRP
HSRP / GLBP	Proprietary dialects of the same problem class
CARP	OpenBSD-family cousin

This book uses **VRRP on FRR** because it is free, documented, and works in Containerlab without licenses. Optional: Nokia SR Linux community images have their own FHRP configuration surface—same model, different CLI.

Priority, preempt, timers

- **Priority:** higher wins master election (within rules).
- **Preempt:** when a higher-priority router returns, does it take the VIP back?
- **Advertisement interval:** how fast backups notice silence.
- **Skew / master-down interval:** how long before backup promotes.

What FHRP is *not*

Not this	Because
Full routing protocol	Hosts do not learn remote prefixes via VRRP
Load balancing of arbitrary flows (unless designed)	Classic VRRP is active/standby for a VIP
Multihoming policy	That is BGP/statics + dual default; separate chapter lab

FHRP protects the **LAN’s first hop**. Dual-homing the site edge is complementary (see dual-home lab later in this part).

ARP and the VIP

When master changes, hosts may still have the old MAC in the neighbor cache briefly. Good implementations use a **shared virtual MAC** or gratuitous ARP so clients rebind quickly. In labs, clear neighbor cache deliberately when debugging:

```
# Linux host
ip neigh flush dev eth1
# or
ip neigh del 10.0.0.254 dev eth1
```

Addressing plan

Node	Role	Interface	Address
r1	VRRP priority high	eth1 (LAN)	10.10.10.2/24
r2	VRRP priority low	eth1 (LAN)	10.10.10.3/24
VIP	gateway for hosts	—	10.10.10.254
h1	client	eth1	10.10.10.10/24 via VIP
r1 eth2	uplink	eth2	172.16.1.1/30
r2 eth2	uplink	eth2	172.16.2.1/30
pe	“upstream”	eth1/eth2	172.16.1.2, 172.16.2.2

Upstream **pe** can be a simple FRR or Linux box that advertises a sink for drills (203.0.113.1 loopback).

Topology YAML

```
name: vrrp-edge

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    pe:
```

```

kind: linux
image: quay.io/frrouting/frr:10.2.1
binds:
  - ./config/pe:/etc/frr
h1:
kind: linux
image: alpine:3.20
exec:
  - apk add --no-cache iproute2 iputils
  - ip addr add 10.10.10.10/24 dev eth1
  - ip link set eth1 up
  - ip route add default via 10.10.10.254

links:
  - endpoints: ["h1:eth1", "r1:eth1"]
  - endpoints: ["r1:eth1", "r2:eth1"]    # same LAN: use bridge or multipoint
  - endpoints: ["r1:eth2", "pe:eth1"]
  - endpoints: ["r2:eth2", "pe:eth2"]

```

LAN wiring note: two point-to-point links do not make a shared segment. Prefer a Linux bridge node as “LAN switch,” or Containerlab multipoint / bridge patterns from your lab-craft notes:

```

lan:
kind: linux
image: alpine:3.20
exec:
  - apk add --no-cache iproute2 bridge-utils
  - ip link add name br0 type bridge
  - ip link set br0 up
# links: h1, r1, r2 all into lan with bridge enslavement in exec

```

Simpler teaching pattern: **one host + two routers on a bridge** so ARP and VRRP multicast behave like a campus LAN.

FRR daemons

```

zebra=yes
vrrpd=yes
# staticd or bgpd if you route beyond the VIP
staticd=yes

```

Check mid-2026 FRR image docs for exact daemon names (`vrrpd` may be integrated differently by version—`vttysh show vrrp` is the operator check).

FRR VRRP sketch (r1)

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.10.10.2/24
!
interface eth2
 ip address 172.16.1.1/30
!
interface eth1
 vrrp 1 ip 10.10.10.254
 vrrp 1 priority 200
 vrrp 1 advertisement-interval 1000
!
ip route 0.0.0.0/0 172.16.1.2
!
line vty
```

r2: real IP 10.10.10.3/24, priority 100, same VIP and VRID 1, default via its own uplink.

Syntax varies slightly by FRR minor version—always validate with:

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r1 vtysh -c 'show interface eth1'
```

Linux alternative (keepalived)

Some operators run keepalived on Linux instead of NOS VRRP. Same objects: VIP, priority, advert. In pure Alpine labs you may:

```
# illustrative - package availability varies
# apk add keepalived
# configure vrrp_instance with virtual_ipaddress
```

For this book's default path, **FRR VRRP** keeps one control-plane stack with your OSPF/BGP labs.

Deploy and baseline

```
sudo containerlab deploy -t vrrp-edge.clab.yml

docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 ip route
docker exec clab-vrrp-edge-h1 ip neigh
docker exec clab-vrrp-edge-h1 ping -c 3 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 3 172.16.1.2
```

Predict (baseline)

- r1 is **Master**, r2 is **Backup** (priority 200 > 100)
- h1 ARP for VIP resolves to master's virtual/real MAC pattern for your stack
- Ping VIP succeeds; traffic to upstream uses r1's uplink

Observe

Record: VRRP state, VIP owner interface counters, `ip neigh` on h1, traceroute if upstream is reachable.

Drill 1 — Master failure

```
# Simulate master death (stop data plane or whole node)
docker exec clab-vrrp-edge-r1 ip link set eth1 down
# or: docker stop clab-vrrp-edge-r1

sleep 3
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 ip neigh show 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 10 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 5 203.0.113.1 # if pe has this
```

Predict → observe → fix

Predict	Observe	Fix if wrong
r2 becomes Master after down-timer	<code>show vrrp</code>	timers, VRID mismatch, not same LAN

Predict	Observe	Fix if wrong
Short ping loss then recovery	count drops	ARP stuck—flush neigh; check VIP MAC
Upstream still works via r2	traceroute	r2 default / pe return routes

Restore:

```
docker exec clab-vrrp-edge-r1 ip link set eth1 up
# or docker start + wait FRR ready
sleep 5
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
```

Preempt on: r1 reclaims Master. **Preempt off:** r2 may stay Master—document which you configured.

Drill 2 — Split brain / LAN partition (concept)

If both routers believe they are alone (broken LAN bridge), **two masters** can both answer the VIP → intermittent ARP thrash.

Predict

Duplicate VIP answers; flapping neighbor cache; odd packet loss.

Observe

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
# tcpdump for VRRP (IP protocol 112) on the LAN
docker exec clab-vrrp-edge-h1 sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 proto 112'
```

Fix / harden

- Ensure single L2 domain for the VRID
- BFD or link tracking to uplinks (advanced) so a “master” without upstream demotes
- Monitoring on “two masters seen” is an ops smell

Drill 3 — Priority change live

```
docker exec -it clab-vrrp-edge-r2 vtysh
configure terminal
interface eth1
  vrrp 1 priority 250
end
write
```

Predict

If preempt allowed, r2 becomes Master; h1 may re-ARP; path to upstream shifts to r2's uplink.

Observe

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 traceroute -n 172.16.2.2
```

Revert priorities in git-mounted configs after the experiment.

Return path and asymmetric edges

VIP failover only moves the **outbound first hop**. If upstream routing always returns via r1 while r2 is master, you get asymmetric paths or blackholes. Pair FHRP with:

- Shared upstream design, or
- Dynamic routing so return traffic follows the active edge, or
- State-aware NAT only on the active path (harder—prefer avoid state at edge in labs first)

Verification checklist

Check	Intent
show vrrp	One Master per VRID on the LAN
Host default route	Points at VIP only
ping VIP	L2/L3 ownership of first hop
Kill master	Backup promotes; host recovers
Capture proto 112	Hellos present on LAN
Upstream ping during fail	Data plane + return path

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }

docker exec clab-vrrp-edge-h1 ping -c 2 -W 1 10.10.10.254 || fail "VIP"
# Exactly one master is harder in shell-parse show vrrp or accept soft check:
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp' | grep -qi master \
|| docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp' | grep -qi master \
|| fail "no master"
echo OK
```

Common mistakes

Mistake	Symptom
Routers not same L2 domain	Never form master/backup correctly
Different VRID or VIP	Two independent groups
Host points at real IP not VIP	No redundancy
No return routes via backup	Failover pings die mid-path
Priority tie without tie-break awareness	Unstable election
Forgetting to enable vrrp daemon	Empty <code>show vrrp</code>

Optional SR Linux note (community)

SR Linux community Containerlab kinds expose interface and IRP/FHRP-related configuration in a different tree (YANG/CLI). **Do not memorize CLI here**—map objects: VIP, group id, priority, interface, preempt. Same predict→observe drills.

Summary

- Hosts need a stable **first hop**; VRRP-class protocols move a VIP between routers
- FRR + Containerlab is enough to learn Master/Backup, preempt, and failure timers
- Always verify **host ARP**, **VRRP state**, and **end-to-end path** after failover
- FHRP is not multihoming policy—combine later with dual-home BGP/statics
- Keep configs in binds; promote live priority experiments back to git

Next: **DHCP relay and NAT**—how edge services attach to this LAN model without becoming mysterious middleboxes.

DHCP Relay and NAT

DHCP Relay and NAT

Edges do more than forward packets. They **hand out addresses** (or relay DHCP) and often **translate** private space toward a provider. This chapter keeps both topics vendor-agnostic and lab-first: Linux services + FRR routing in Containerlab (mid-2026 free path).

Learning goals

By the end of this chapter you can:

- Explain DHCP discover/offer/request/ack across a relay
- Configure a simple DHCP server and a relay path in a lab
- Contrast source NAT (masquerade), static NAT, and “why not NAT everything forever”
- Implement Linux `nftables`/`iptables` MASQUERADE on an edge
- Predict breakage when relays, helpers, or return translations fail

Concepts — DHCP

Why not static everything?

Labs love static host IPs. Real access LANs use DHCP for:

Benefit	Trade-off
Central policy (lease, DNS, gateway)	Dependency on server/relay
Fast readdressing	Lease timers and “sticky” clients
Audit of who got what	Need logging and hygiene

Four-way dance (v4)

```
Client          Relay (optional)          Server
|-- Discover -----(unicast to server)-->|
|<-- Offer -----|
|-- Request ----->|
```

|<-- Ack -----|

On a flat LAN, broadcast Discover reaches the server. Across subnets, a **relay agent** (ip helper / DHCP relay) listens on the client VLAN and unicasts to the server, inserting **giaddr** (gateway IP of the client interface) so the server knows which pool to use.

Critical fields

Field / idea	Why it matters
Client MAC / Client-ID	Lease binding
giaddr	Selects pool / subnet
Options: router, DNS, domain	Host becomes usable
Lease time	How long address is “owned”

Relay failure modes

- Wrong giaddr → wrong pool or no offer
- ACL blocking UDP 67/68
- Server unreachable from relay
- Multiple relays without design → duplicate offers chaos

Concepts — NAT family

Kind	Meaning
SNAT / MASQUERADE	Rewrite source as packets leave (common “PAT”)
DNAT	Rewrite destination (port forward / publish service)
1:1 static NAT	Stable mapping for a host
NAT64 / CLAT	v6/v4 transition tools (awareness only here)

NAT is **state** + **rewrite**. It is not a security model by itself. Stateful firewalls may sit with NAT; keep roles clear.

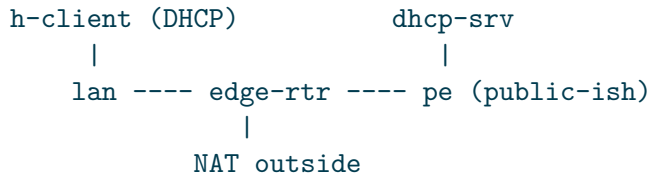
When NAT is appropriate in this book

- Lab edge simulating consumer/ISP CPE
- Hiding RFC1918 behind a single lab “public”

- Temporary access during dual-stack migration

Prefer **real global or ULA design** for multi-site fabrics later—NAT at every hop becomes undebugable.

Addressing plan (combined edge lab)



Net	Use
10.20.30.0/24	Client LAN (DHCP pool)
10.20.30.1	Edge LAN IP + VRRP VIP optional
172.31.0.0/30	Edge-server link for DHCP server placement
198.51.100.0/30	Edge-PE “public” lab space (TEST-NET)
Pool	10.20.30.100–200, GW 10.20.30.1, DNS 10.20.30.1 or public lab DNS

Topology YAML

```

name: dhcp-nat-edge

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      # or FRR image if you co-locate routing; Alpine for pure Linux NAT/relay
      exec:
        - apk add --no-cache iproute2 iptables iputils dhcrelay dnsmasq
    pe:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 198.51.100.2/30 dev eth1
        - ip link set eth1 up
        - ip addr add 203.0.113.1/32 dev lo
        - ip link set lo up
    dhcpsrv:

```

```

kind: linux
image: alpine:3.20
exec:
  - apk add --no-cache iproute2 dnsmasq iputils
  - ip addr add 172.31.0.2/30 dev eth1
  - ip link set eth1 up
client:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 dhclient iputils
lan:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2
    - ip link add br0 type bridge && ip link set br0 up

links:
  - endpoints: ["client:eth1", "lan:eth1"]
  - endpoints: ["edge:eth1", "lan:eth2"]
  - endpoints: ["edge:eth2", "dhcpsrv:eth1"]
  - endpoints: ["edge:eth3", "pe:eth1"]

```

Bridge enslavement (run after deploy or in richer `exec` scripts):

```

# on lan node - enslaves data ports into br0
for i in eth1 eth2; do ip link set $i master br0; ip link set $i up; done

```

Edge addressing and forward

```

# edge LAN toward clients
docker exec clab-dhcp-nat-edge-edge ip addr add 10.20.30.1/24 dev eth1
docker exec clab-dhcp-nat-edge-edge ip link set eth1 up
# toward DHCP server
docker exec clab-dhcp-nat-edge-edge ip addr add 172.31.0.1/30 dev eth2
docker exec clab-dhcp-nat-edge-edge ip link set eth2 up
# toward PE
docker exec clab-dhcp-nat-edge-edge ip addr add 198.51.100.1/30 dev eth3
docker exec clab-dhcp-nat-edge-edge ip link set eth3 up
docker exec clab-dhcp-nat-edge-edge sysctl -w net.ipv4.ip_forward=1
docker exec clab-dhcp-nat-edge-edge ip route add default via 198.51.100.2

```

Bake these into startup scripts for real lab-as-code.

DHCP server (dnsmasq) on dhcpsrv

```
docker exec clab-dhcp-nat-edge-dhcpserver sh -c 'cat >/etc/dnsmasq.conf <<EOF
interface=eth1
dhcp-range=10.20.30.100,10.20.30.200,12h
dhcp-option=option:router,10.20.30.1
dhcp-option=option:dns-server,10.20.30.1
log-dhcp
EOF'
docker exec clab-dhcp-nat-edge-dhcpserver dnsmasq
```

Server is **not** on 10.20.30.0/24—the relay’s giaddr must make the pool selection work. For dnsmasq/simple labs, operators often put the server **on** the client subnet first (no relay). Teach both:

1. **Same subnet:** client broadcast reaches server—no relay.
2. **Off-subnet:** relay required; server must understand the client subnet (shared-network / giaddr-aware config).

Minimal same-subnet lab (sanity first)

Place dnsmasq on edge’s LAN IP or a server on the bridge:

```
dhcp-range=10.20.30.100,10.20.30.200,12h
dhcp-option=option:router,10.20.30.1
```

Relay path (concept + command)

```
# edge: relay client-facing interface toward server
docker exec clab-dhcp-nat-edge-edge dhcrelay -i eth1 172.31.0.2
```

Predict (DHCP)

- Client sends Discover; server logs Offer for 10.20.30.x
- Client installs default via 10.20.30.1
- Lease appears in server logs

Observe

```
docker exec clab-dhcp-nat-edge-client ip link set eth1 up
docker exec clab-dhcp-nat-edge-client dhclient eth1 -v
docker exec clab-dhcp-nat-edge-client ip addr show eth1
docker exec clab-dhcp-nat-edge-client ip route
docker exec clab-dhcp-nat-edge-dhcpshr cat /var/lib/misc/dnsmasq.leases 2>/dev/null || true
```

Fix

Symptom	Checks
No offer	bridge, interface up, firewall, wrong interface in dnsmasq
Offer but wrong GW	dhcp-option router
Relay silent	UDP 67/68, giaddr, server route

NAT — MASQUERADE on edge

Clients use private 10.20.30.0/24; PE only knows 198.51.100.0/30.

```
docker exec clab-dhcp-nat-edge-edge iptables -t nat -A POSTROUTING \
-o eth3 -j MASQUERADE
# nftables equivalent pattern:
# nft add table ip nat
# nft add chain ip nat postrouting { type nat hook postrouting priority 100 \; }
# nft add rule ip nat postrouting oifname "eth3" masquerade
```

PE needs a way to answer (connected route to 198.51.100.1 is enough for return of MASQUERADE traffic).

Predict

- client → 203.0.113.1 works
- On pe, sources appear as 198.51.100.1, not 10.20.30.x
- Without MASQUERADE, pe has no return path to 10.20.30.0/24

Observe

```
docker exec clab-dhcp-nat-edge-client ping -c 3 203.0.113.1
docker exec clab-dhcp-nat-edge-pe sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 -c 5 i
docker exec clab-dhcp-nat-edge-edge iptables -t nat -L -n -v
```

Fix

- `ip_forward=1`
- Correct `-o` egress interface
- Default route on client and edge
- Conntrack full / flush during experiments: `conntrack -F` if tool present

Drill — break DHCP then NAT

1. Stop dnsmasq → new client gets nothing; journal “dependency.”
2. Remove MASQUERADE → pings to PE fail; add static route on PE for 10.20.30.0/24 via edge as a **teaching alternate** (routing vs NAT).
3. DNAT publish (optional): map 198.51.100.1:8080 → client:80 and curl from pe.

```
docker exec clab-dhcp-nat-edge-edge iptables -t nat -A PREROUTING \
-i eth3 -p tcp --dport 8080 -j DNAT --to-destination 10.20.30.100:80
```

Only works if a client actually holds that lease/IP and listens.

FRR on the edge (optional co-location)

Many designs run FRR for BGP/OSPF and Linux for NAT/DHCP. Pattern:

- FRR owns routing (`vttysh`)
- Linux netfilter owns NAT
- DHCP server may be centralized; FRR/host runs relay

Do not assume FRR “is” the NAT engine—check which process owns rewrite.

Verification checklist

Check	Command / intent
Lease	client has /24 + default
Options	DNS/GW match design
Relay	server log shows giaddr path
NAT	capture shows public source
Forward	sysctl ip_forward
Rules	iptables/nft list nat

Common mistakes

Mistake	Symptom
DHCP server wrong subnet logic	No lease or wrong mask
Client static leftover IP	“DHCP broken” false positive
NAT without hairpin plan	Local services odd
Forgetting return routing when NAT removed	One-way pings
Blocking bootp/dhcp in filter	Silent failure

Summary

- DHCP centralizes host addressing; **relay** extends it across L3
- **giaddr** and pool design matter as much as the daemon
- NAT rewrites and tracks state—use deliberately, verify with captures
- Linux edge + Containerlab is enough to practice both without licenses
- Prefer lab scripts that start dnsmasq, relay, and NAT rules idempotently

Next: **name, time, and log hygiene**—the quiet services that make edge failures diagnosable.

Name, Time, and Log Hygiene

Name, Time, and Log Hygiene

Broken DNS, skewed clocks, and silent logs turn simple outages into multi-hour mysteries. This chapter treats **name resolution, time sync, and logging** as first-class edge services—still on free Linux tooling in Containerlab (mid-2026).

Learning goals

By the end of this chapter you can:

- Design a minimal lab DNS path (stub resolver → recursive or authoritative)
- Explain why certificate and log correlation depend on clock sync
- Run chrony/ntp-style time sync in a lab and detect skew
- Centralize or at least structure logs so incidents are replayable
- Apply a hygiene checklist to any edge or fabric lab

Why this chapter exists

Failure	Symptom people mis-blame
DNS	“The network is down” (TCP works, names fail)
Time skew	TLS fails, Kerberos fails, log order nonsense
No logs	Every fix is guesswork; no postmortem

NetOps competence includes **service dependencies of the control and management planes**.

Concepts — naming

Layers of DNS use

```
Application
  → stub resolver (/etc/resolv.conf)
    → recursive resolver (lab or ISP)
      → authoritative servers
```

In labs you often run:

Role	Example tool
Authoritative for <code>lab.example</code>	dnsmasq, unbound, knot (as available)
Recursive	unbound, dnsmasq
Stub on nodes	<code>nameserver</code> in <code>resolv.conf</code>

Records you actually need

Type	Lab use
A/AAAA	Host and VIP names
PTR	Reverse for traceroute readability (optional)
SRV	Only when practicing real apps
CNAME	Alias hygiene—avoid chains

Split horizon awareness

Internal names may not match external. Document which view a resolver serves. Mis-split horizon looks like “works on my jump host.”

Concepts — time

- **UTC everywhere** in configs and logs when possible
- **NTP/chrony/ptp** family: clients step or slew toward sources
- Auth and TLS validate **notBefore/notAfter** against local clock
- Distributed tracing and log merge need comparable timestamps

Skew of minutes is enough to break modern auth. Skew of seconds confuses incident timelines.

Concepts — logs

Plane	What to capture
Device/OS	link up/down, daemon start, OOM
Routing	neighbor flaps, policy denies (where logged)
Security	filter drops if logged
Lab harness	deploy/destroy, verify.sh results

Structured fields beat walls of free text: `timestamp host facility message`.

Addressing plan (hygiene lab)

```
client ---- edge ---- dns-ntp
              \
               pe
```

Node	IP	Role
edge eth1	10.40.0.1/24	client GW
client	10.40.0.10/24	stub resolver → edge or dns
dns-ntp	10.40.0.53/24	dnsmasq + chrony server
edge eth2	198.51.100.1/30	optional upstream

Topology YAML

```
name: hygiene-edge

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iptables iputils bind-tools chrony
    dnsntp:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 dnsmasq chrony iputils bind-tools
    client:
      kind: linux
```



```

    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils bind-tools
  pe:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["client:eth1", "edge:eth1"]
    - endpoints: ["dnsntp:eth1", "edge:eth2"]
    - endpoints: ["edge:eth3", "pe:eth1"]

```

Use a small bridge if you want client and dnsntp on one LAN with edge as L3 gateway only—either design is fine if documented.

Bring-up addressing (example)

```

# edge: two LANs + pe
docker exec clab-hygiene-edge-edge sh -c '
  ip addr add 10.40.0.1/24 dev eth1; ip link set eth1 up
  ip addr add 10.40.1.1/24 dev eth2; ip link set eth2 up
  ip addr add 198.51.100.1/30 dev eth3; ip link set eth3 up
  sysctl -w net.ipv4.ip_forward=1
'

docker exec clab-hygiene-edge-dnsntp sh -c '
  ip addr add 10.40.1.53/24 dev eth1; ip link set eth1 up
  ip route add default via 10.40.1.1
'

docker exec clab-hygiene-edge-client sh -c '
  ip addr add 10.40.0.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.40.0.1
  echo nameserver 10.40.1.53 > /etc/resolv.conf
'

# edge route to dns LAN is connected; client needs path to 10.40.1.53
docker exec clab-hygiene-edge-edge ip route add 10.40.1.0/24 dev eth2 2>/dev/null || true

```

Ensure client can reach 10.40.1.53 (forwarding + reverse path).

DNS lab — dnsmasq authoritative snack

```
docker exec clab-hygiene-edge-dnsntp sh -c 'cat >/etc/dnsmasq.conf <<EOF
interface=eth1
no-resolv
address=/vip.lab.example/10.40.0.254
address=/edge.lab.example/10.40.0.1
address=/client.lab.example/10.40.0.10
log-queries
EOF'
docker exec clab-hygiene-edge-dnsntp dnsmasq
```

Predict

```
docker exec clab-hygiene-edge-client nslookup vip.lab.example 10.40.1.53
# or
docker exec clab-hygiene-edge-client dig +short vip.lab.example @10.40.1.53
```

Returns 10.40.0.254.

Observe / fix

Symptom	Fix
timeout	routing, firewall, dnsmasq not listening
NXDOMAIN	typo in address= or wrong server
uses wrong resolver	resolv.conf

Drill — DNS outage looks like network outage

```
docker exec clab-hygiene-edge-dnsntp pkill dnsmasq || true
docker exec clab-hygiene-edge-client ping -c 2 vip.lab.example # may fail resolve
docker exec clab-hygiene-edge-client ping -c 2 10.40.0.1 # IP still works
```

Predict → observe → fix

- Name fails, IP works → **name plane**, not forwarding
- Restart dnsmasq; retest dig
- Harden: secondary resolver IP in resolv.conf (even if both lab-local)

```
nameserver 10.40.1.53
nameserver 10.40.1.54
```

Time sync — chrony sketch

On dnsntp as a simple lab time source (isolated labs often use local orphan mode or host chrony):

```
docker exec clab-hygiene-edge-dnsntp sh -c 'cat >/etc/chrony/chrony.conf <<EOF
local stratum 8
allow 10.40.0.0/16
makestep 1.0 3
EOF'
docker exec clab-hygiene-edge-dnsntp chronyd
```

Client/edge:

```
docker exec clab-hygiene-edge-client sh -c '
apk add --no-cache chrony
echo "server 10.40.1.53 iburst" > /etc/chrony/chrony.conf
chronyd
'
docker exec clab-hygiene-edge-client chronyc tracking
docker exec clab-hygiene-edge-client chronyc sources -v
```

Package paths vary (/etc/chrony.conf vs /etc/chrony/chrony.conf)—adjust to the image.

Skew drill

```
# induce skew on client (lab only)
docker exec clab-hygiene-edge-client date -s "2020-01-01 00:00:00"
docker exec clab-hygiene-edge-client chronyc -a makestep
docker exec clab-hygiene-edge-client date
docker exec clab-hygiene-edge-dnsntp date
```

Predict

After makestep/slew, client approaches server time; `chronyc tracking` shows low offset.

Observe

Record offset before/after. Correlate with a fake “TLS” check: if you generate certs with short validity in lab tools, wrong dates fail verification.

Logging hygiene

Local journal pattern

```
# example: log dnsmasq and link events to a file on edge
docker exec clab-hygiene-edge-edge sh -c '
  mkdir -p /var/log/lab
  echo "$(date -Is) lab start" >> /var/log/lab/edge.log
'
```

syslog-style forward (concept)

```
device/app → rsyslog/syslog-ng → central lab collector
```

In Containerlab, mounting a host directory for `/var/log/lab` keeps evidence after destroy:

```
edge:
  kind: linux
  image: alpine:3.20
  binds:
    - ./logs/edge:/var/log/lab
```

What to log for network incidents

Event	Why
Link down/up	Correlates with neighbor loss
Daemon restart	Control plane blip
DHCP ACK / VRRP transition	Edge role changes
Filter deny (sampled)	Policy vs outage
verify.sh pass/fail	Automation truth

FRR logging (when routers are FRR)

```
log file /var/log/frr/frr.log
log timestamp precision 3
```

```
docker exec clab-...-r1 vtysh -c 'show log'
# or tail the bound log file
```

Bind `/var/log/frr` into the lab folder for postmortems.

Hygiene checklist (use every serious lab)

- ☐ Addressing plan names every important VIP and service IP
- ☐ DNS A records for VIP, routers, servers (even if only lab zone)
- ☐ Clients have 1 working resolver; document fallback
- ☐ Clocks within seconds of a known source (or explicitly isolated)
- ☐ Logs land on durable storage (bind mount / central)
- ☐ Timezone policy stated (UTC recommended)
- ☐ Incident notes include timestamps from the **same** clock domain

Predict → observe → fix mini-scenarios

Scenario A — “Git clone fails” in a lab VM

- Predict: DNS or proxy
- Observe: `ping 1.1.1.1` vs `ping github.com`
- Fix: `resolv.conf` / reachability to recursive resolver

Scenario B — “Certificate expired” on brand-new lab CA

- Predict: node year is wrong
- Observe: `date -u` on client and server
- Fix: `chrony`; disable fake date experiments

Scenario C — “It failed an hour ago” with no trace

- Predict: no logs retained
- Observe: empty `/var/log/lab`, container destroyed
- Fix: binds + `verify.sh` archival

Optional: name the lab graph

Containerlab graph exports help humans; DNS helps apps. Keep **hostname = inventory name = DNS label** when you can:

```
clab-hygiene-edge-edge    edge.lab.example
```

Common mistakes

Mistake	Symptom
resolv.conf points at host DNS unreachable from namespace	all names fail
No forward path to DNS subnet	intermittent based on which node
Logging only to container overlay	evidence vanishes on destroy
Mixing local timezones in notes	impossible timelines
NTP blocked by ACL you added “for hardening”	chronic skew

Summary

- **Name, time, logs** are edge/management dependencies, not optional polish
- Practice DNS outage vs forwarding outage until the difference is reflex
- Sync clocks before debugging TLS or distributed logs
- Bind-mount lab logs; structure enough fields to postmortem
- Add this hygiene checklist to dual-home and fabric labs later

Next: **edge dual-home lab**—tie FHRP, routing preference, and these services into one site edge workout.

Edge Dual-Home Lab

Edge Dual-Home Lab

This chapter is a **full edge workout**: one customer site, two uplinks to two providers, first-hop redundancy on the LAN, filtered eBGP, and failure drills. Stack: Containerlab + FRR + Alpine (mid-2026 free path). Optional SR Linux as a PE substitute for CLI diversity.

Learning goals

By the end of this chapter you can:

- Design dual-homed CE addressing and AS layout
- Prefer one uplink with local-preference; backup with AS-path prepend
- Combine VRRP VIP for hosts with dual CE or single CE dual-uplink patterns
- Prove failover with ping bursts, BGP tables, and traceroute
- Ship a lab folder that meets the book's definition of done

Design choices (pick one primary)

Pattern A — Single CE, two eBGP uplinks (simpler)

```
    pe1 (AS65010)
    /
ce (AS65001) ---- LAN ---- hosts (GW = ce LAN IP or VIP if HA pair)
    \
    pe2 (AS65020)
```

Pattern B — Dual CE + VRRP (more realistic campus edge)

```
    pe1
    /
ce1 === VRRP VIP === LAN --- hosts
    \  /
    ce2
```


\
pe2

This chapter implements **Pattern B** as the main lab and notes how to simplify to A.

Addressing plan

Object	Value
Site LAN	192.168.50.0/24
VIP gateway	192.168.50.254
ce1 LAN	192.168.50.2/24
ce2 LAN	192.168.50.3/24
host	192.168.50.10/24 via VIP
ce1-pe1	172.16.10.0/30 (ce1 .1, pe1 .2)
ce2-pe2	172.16.20.0/30 (ce2 .1, pe2 .2)
Site prefix	192.168.50.0/24 (advertise)
pe1 lo	1.1.1.1/32 (AS65010)
pe2 lo	2.2.2.2/32 (AS65020)
Hijack test	203.0.113.0/24 must not be accepted from CE

Topology YAML

```
name: dualhome

topology:
  nodes:
    ce1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/ce1:/etc/frr
    ce2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/ce2:/etc/frr
    pe1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/pe1:/etc/frr
    pe2:
      kind: linux
```

```

    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/pe2:/etc/frr
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.50.10/24 dev eth1
    - ip link set eth1 up
    - ip route add default via 192.168.50.254
lan:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2
    - ip link add br0 type bridge
    - ip link set br0 up

links:
  - endpoints: ["h1:eth1", "lan:eth1"]
  - endpoints: ["ce1:eth1", "lan:eth2"]
  - endpoints: ["ce2:eth1", "lan:eth3"]
  - endpoints: ["ce1:eth2", "pe1:eth1"]
  - endpoints: ["ce2:eth2", "pe2:eth1"]

```

Enslave LAN ports to br0 in a deploy hook:

```

for n in eth1 eth2 eth3; do
  docker exec clab-dualhome-lan ip link set $n master br0
  docker exec clab-dualhome-lan ip link set $n up
done

```

FRR daemons

```

zebra=yes
bgpd=yes
staticd=yes
vrrpd=yes

```

CE1 config sketch

```
frr version 10.2.1
frr defaults traditional
hostname ce1
!
interface lo
 ip address 10.0.0.1/32
!
interface eth1
 ip address 192.168.50.2/24
 vrrp 1 ip 192.168.50.254
 vrrp 1 priority 200
!
interface eth2
 ip address 172.16.10.1/30
!
router bgp 65001
 bgp router-id 10.0.0.1
 no bgp ebgp-requires-policy
 neighbor 172.16.10.2 remote-as 65010
!
 address-family ipv4 unicast
  network 192.168.50.0/24
  neighbor 172.16.10.2 route-map FROM-PE in
  neighbor 172.16.10.2 route-map TO-PE out
 exit-address-family
!
 ip prefix-list SITE seq 5 permit 192.168.50.0/24
 ip prefix-list SITE seq 10 deny any
!
 ip prefix-list DEFAULT-OK seq 5 permit 0.0.0.0/0
 ip prefix-list DEFAULT-OK seq 10 deny any
!
 route-map TO-PE permit 10
  match ip address prefix-list SITE
 route-map TO-PE deny 20
!
 route-map FROM-PE permit 10
  match ip address prefix-list DEFAULT-OK
  set local-preference 200
 route-map FROM-PE deny 20
!
 line vty
```

`no bgp ebgp-requires-policy` is a lab convenience on some FRR versions—prefer explicit policies always in “production-shaped” labs.

CE2 config differences

- LAN IP .3, VRRP priority 100
- Uplink 172.16.20.1/30 to pe2
- **Outbound prepend** so pe2 is less preferred by others if both advertise:

```
route-map T0-PE permit 10
match ip address prefix-list SITE
set as-path prepend 65001 65001
```

- **Inbound** local-pref 100 (lower than ce1's 200) if both learn default:

```
route-map FROM-PE permit 10
match ip address prefix-list DEFAULT-OK
set local-preference 100
```

For dual-CE, each CE may only have one uplink—**VRRP** chooses which CE hosts use; each CE's BGP prefers its own default. Advanced: iBGP between ce1/ce2—optional stretch goal.

PE1 / PE2 sketch

```
hostname pe1
!
interface lo
 ip address 1.1.1.1/32
!
interface eth1
 ip address 172.16.10.2/30
!
router bgp 65010
 bgp router-id 1.1.1.1
 neighbor 172.16.10.1 remote-as 65001
!
address-family ipv4 unicast
 network 0.0.0.0/0
 network 1.1.1.1/32
 neighbor 172.16.10.1 route-map FROM-CE in
 neighbor 172.16.10.1 route-map T0-CE out
 exit-address-family
!
ip prefix-list CE-ALLOW seq 5 permit 192.168.50.0/24
ip prefix-list CE-ALLOW seq 10 deny any
!
```

```

route-map FROM-CE permit 10
  match ip address prefix-list CE-ALLOW
route-map FROM-CE deny 20
!
ip prefix-list ORIGINATE seq 5 permit 0.0.0.0/0
ip prefix-list ORIGINATE seq 10 permit 1.1.1.1/32
route-map TO-CE permit 10
  match ip address prefix-list ORIGINATE
!
line vty

```

pe2 mirrors with AS 65020, 2.2.2.2, 172.16.20.2.

Hijack drill: temporarily add network 203.0.113.0/24 on CE—PE must **not** install it.

Deploy and baseline verify

```

sudo containerlab deploy -t dualhome.clab.yml
# bridge hook as above

docker exec clab-dualhome-ce1 vtysh -c 'show vrrp'
docker exec clab-dualhome-ce2 vtysh -c 'show vrrp'
docker exec clab-dualhome-ce1 vtysh -c 'show bgp summary'
docker exec clab-dualhome-ce1 vtysh -c 'show ip route'
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni'
docker exec clab-dualhome-h1 ping -c 3 192.168.50.254
docker exec clab-dualhome-h1 ping -c 3 1.1.1.1
docker exec clab-dualhome-h1 traceroute -n 1.1.1.1

```

Predict (baseline)

- ce1 VRRP Master
- h1 reaches VIP and pe1 loopback via ce1
- pe1 has 192.168.50.0/24 only from CE
- pe2 may also have site prefix if ce2 session up

Drill 1 — Primary PE session loss

```
docker exec clab-dualhome-ce1 ip link set eth2 down
sleep 2
docker exec clab-dualhome-ce1 vtysh -c 'show bgp summary'
docker exec clab-dualhome-h1 ping -c 20 2.2.2.2
docker exec clab-dualhome-h1 traceroute -n 2.2.2.2
```

Predict → observe → fix

Predict	If false
Host still uses ce1 VIP (VRRP unchanged)	—
ce1 loses default → traffic blackholes unless ce2 takes VIP or ce1 has alternate	Add static floating / iBGP / kill VRRP master

Important teaching moment: if only ce1 is master and its uplink dies, **VRRP does not care** unless you track the uplink (BFD/link tracking) or lower priority. Options:

1. Shut ce1 LAN to force VRRP failover (crude).
2. Configure track/priority decrement when eth2 down (if supported).
3. Run dual default via both CEs with host-side multipath (rare).

Document which design you chose. Professional edges **tie FHRP priority to uplink health**.

Drill 2 — Force VRRP failover

```
docker exec clab-dualhome-ce1 ip link set eth1 down
sleep 3
docker exec clab-dualhome-ce2 vtysh -c 'show vrrp'
docker exec clab-dualhome-h1 ip neigh flush dev eth1
docker exec clab-dualhome-h1 ping -c 10 2.2.2.2
docker exec clab-dualhome-h1 traceroute -n 2.2.2.2
```

Predict

ce2 Master; path via pe2; short disruption.

Drill 3 — Policy rejection

```
# on ce1 temporarily
docker exec -it clab-dualhome-ce1 vtysh
# network 203.0.113.0/24 under bgp + dummy interface address if needed

docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni 203.0.113.0/24'
```

Predict

Empty / not best—filter works. If accepted, fix FROM-CE prefix-list.

Drill 4 — Name/time smoke (hygiene)

Optional: add dnsmasq A record vip.site.lab → 192.168.50.254 and dig from h1; confirm date -u roughly matches host.

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }

docker exec clab-dualhome-h1 ping -c 2 -W 1 192.168.50.254 || fail "vip"
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni' | grep -q '192.168.50.0' \
|| fail "pe1 missing site"
# hijack should be absent
if docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni' | grep -q '203.0.113.0'; then
    fail "hijack accepted"
fi
echo OK
```

Definition of done (this lab)

- ☐ Topology + configs in git
- ☐ Addressing plan + diagram
- ☐ VRRP master/backup documented

- ☐ eBGP policies on CE and PE
- ☐ Three drills with notes (PE loss, VRRP fail, hijack)
- ☐ `verify.sh` green on clean deploy
- ☐ Written note: how uplink loss interacts with VRRP

Optional SR Linux PE

Replace `pe1` with community SR Linux kind in Containerlab; keep CE on FRR. Same BGP objects: neighbor, import prefix allow-list, export default. Compare `show bgp` dialects in your journal—models stay identical.

Common mistakes

Mistake	Symptom
No bridge on LAN	VRRP never forms
Host uses .2 not VIP	No HA
PE accepts any prefix	Hijack succeeds
Uplink down but VRRP stays on blind CE	Blackhole
Asymmetric return without pe routes	One-way

Summary

- Dual-home labs combine **multihoming policy** and **first-hop HA**—design their interaction
- FRR eBGP + prefix-lists + VRRP are enough for a portfolio-quality edge
- Always test **policy negatives** (hijack) and **uplink vs VIP** failures separately
- Prefer link tracking or clear runbooks when master loses upstream
- This lab maps to capstone C2 in the ops part

Next part: **policy, QoS vocabulary, and hardening**—filters beyond BGP, control-plane protection, and checklists.

Policy, QoS, and Hardening

ACL Thinking

ACL Thinking

Access control lists are not a pile of lines—they are a **decision policy** evaluated in order. This chapter builds vendor-agnostic ACL decision patterns and implements them with Linux nftables/iptables and FRR filters in Containerlab (mid-2026 free tools).

Learning goals

By the end of this chapter you can:

- Write allow/deny policies with explicit default stance
- Place filters at the right plane (data plane vs route policy vs management)
- Implement Linux netfilter rules that match a written policy
- Apply FRR prefix-lists/route-maps without confusing them with packet ACLs
- Predict asymmetric ACL failures and fix with evidence

Concepts

Three different “filters” people mix up

Kind	Acts on	Example
Packet ACL / firewall	Data-plane packets	nftables drop tcp/23
Route policy	Routes in RIB/advertisements	prefix-list on BGP
Control-plane protection	Traffic to the CPU/RE	CoPP / policers (next chapter)

Using a BGP prefix-list will **not** stop a host scan. Dropping packets will **not** stop a bad route install (unless you couple systems).

Policy statement template

Before typing rules, write:

```

Stance: default deny (or default allow-pick deliberately)
Allow:
- established/related (if stateful)
- DNS to resolver R from clients
- HTTPS to internet from clients
- BGP from peer P on interface I
Deny:
- everything else (log sample)

```

First-match vs best-match

Most packet ACLs are **first match wins**. Order is part of the policy. Route filters also often first-match in lists. Never “add a deny at the top” without reading the whole list.

Stateful vs stateless

	Stateless	Stateful
Idea	Each packet alone	Track flows/conntrack
Return traffic	Must allow reverse explicitly	Often allow established
Lab tools	Simple iptables rules	iptables/nft conntrack

Direction and attachment

```

inbound on client LAN    inbound on uplink

```

Always specify **interface + direction + zone**. Diagram the edge:

```

[clients] --in--> | edge | --out--> [uplink]
                  |       |
                  +--- mgmt

```

Written policy for the lab

Edge role: small site router (Linux).

1. Default **drop** forwarded traffic not explicitly allowed.
2. Allow clients (10.60.0.0/24) to ping and HTTP to lab server 10.60.1.10.

3. Allow clients DNS to 10.60.0.53.
4. Allow SSH **only** from mgmt host 10.60.0.9 to edge.
5. Deny client access to edge SSH from other addresses.
6. Log dropped new TCP (rate-limited mentally—don't flood disks).

Topology YAML

```
name: acl-think

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iptables iputils curl
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils curl
    mgmt:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils openssh-client
    srv:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils busybox-extras
        # tiny http: nc -l -p 80 ...

  links:
    - endpoints: ["client:eth1", "edge:eth1"]
    - endpoints: ["mgmt:eth1", "edge:eth1"] # same LAN via bridge preferred
    - endpoints: ["edge:eth2", "srv:eth1"]
```

Prefer a bridge node so client + mgmt share 10.60.0.0/24:

```
lan:
  kind: linux
```

```

image: alpine:3.20
exec:
  - apk add --no-cache iproute2
  - ip link add br0 type bridge && ip link set br0 up
links:
  - endpoints: ["client:eth1", "lan:eth1"]
  - endpoints: ["mgmt:eth1", "lan:eth2"]
  - endpoints: ["edge:eth1", "lan:eth3"]
  - endpoints: ["edge:eth2", "srv:eth1"]

```

Addressing

Node	IP
edge eth1	10.60.0.1/24
edge eth2	10.60.1.1/24
client	10.60.0.10/24 via .1
mgmt	10.60.0.9/24 via .1
srv	10.60.1.10/24 via .1
DNS (optional)	10.60.0.53 on edge lo or dnsmasq

```

docker exec clab-acl-think-edge sh -c '
  sysctl -w net.ipv4.ip_forward=1
  ip addr add 10.60.0.1/24 dev eth1; ip link set eth1 up
  ip addr add 10.60.1.1/24 dev eth2; ip link set eth2 up
'

docker exec clab-acl-think-client sh -c '
  ip addr add 10.60.0.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.0.1
'

docker exec clab-acl-think-mgmt sh -c '
  ip addr add 10.60.0.9/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.0.1
'

docker exec clab-acl-think-srv sh -c '
  ip addr add 10.60.1.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.1.1
'

```

Implement with iptables (readable teaching form)

Flush carefully in lab only:

```

docker exec clab-acl-think-edge sh -c '
iptables -F
iptables -X
iptables -t nat -F
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# loopback + established
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# SSH to edge only from mgmt
iptables -A INPUT -p tcp -s 10.60.0.9 --dport 22 -j ACCEPT

# allow ping to edge from LAN (optional)
iptables -A INPUT -p icmp -s 10.60.0.0/24 -j ACCEPT

# forward: client/mgmt to server http + ping
iptables -A FORWARD -s 10.60.0.0/24 -d 10.60.1.10 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -s 10.60.0.0/24 -d 10.60.1.10 -p icmp -j ACCEPT

# explicit deny counters
iptables -A FORWARD -j LOG --log-prefix "FWD-DROP: " --log-level 4
iptables -A INPUT -j LOG --log-prefix "IN-DROP: " --log-level 4
'
```

If `conntrack` module unavailable in minimal Alpine, approximate with broad return allows **only for learning**, then move to an image with `conntrack`.

nftables sketch (same policy)

```

docker exec clab-acl-think-edge sh -c 'apk add --no-cache nftables; nft -f - <<EOF
flush ruleset
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        iif lo accept
        ct state established,related accept
        tcp dport 22 ip saddr 10.60.0.9 accept
        ip protocol icmp ip saddr 10.60.0.0/24 accept
    }
    chain forward {
        type filter hook forward priority 0; policy drop;
        ct state established,related accept
    }
}
```

```

    ip saddr 10.60.0.0/24 ip daddr 10.60.1.10 tcp dport 80 accept
    ip saddr 10.60.0.0/24 ip daddr 10.60.1.10 ip protocol icmp accept
}
chain output {
    type filter hook output priority 0; policy accept;
}
}
EOF'

```

Predict → observe → fix drills

Drill 1 — Allowed path

```

docker exec clab-acl-think-srv sh -c 'echo OK | nc -l -p 80 &'
docker exec clab-acl-think-client ping -c 2 10.60.1.10
docker exec clab-acl-think-client wget -q0- http://10.60.1.10 || \
    docker exec clab-acl-think-client nc -w2 10.60.1.10 80 </dev/null

```

Predict: success. **If fail:** forwarding, routes, rule order, server listen.

Drill 2 — Denied path

```

docker exec clab-acl-think-client sh -c 'nc -w2 10.60.1.10 22 || true'
docker exec clab-acl-think-edge iptables -L FORWARD -n -v

```

Predict: fail; counters/logs increment.

Drill 3 — SSH source restriction

```

# from mgmt should work if dropbear/sshd runs on edge
# from client should fail
docker exec clab-acl-think-client sh -c 'nc -w2 10.60.0.1 22 || echo blocked'

```

Drill 4 — Asymmetric paths

Allow only client→server tcp/80 but forget established return:

Predict: SYN works, data stalls if truly stateless wrong.

Observe: conntrack ESTABLISHED rule presence.

Fix: stateful allow or explicit reverse rule.

Drill 5 — Wrong plane

Install FRR on edge and **only** add a prefix-list denying 10.60.1.0/24. Client ping to server still works if connected routes exist.

Predict: route policy packet filter.

Observe: ip route still forwards; iptables counters unchanged for that myth.

Fix: teach the table of three filter kinds again.

FRR: route policy is still “ACL thinking”

```
ip prefix-list BOGON seq 5 deny 0.0.0.0/8 le 32
ip prefix-list BOGON seq 10 deny 10.0.0.0/8 le 32
ip prefix-list BOGON seq 100 permit any
!
route-map IMPORT deny 10
  match ip address prefix-list BOGON
route-map IMPORT permit 20
```

Same discipline: **written intent** → **ordered rules** → **negative tests**.

Object-group idea (vendor-agnostic)

Group hosts and services even if the engine lacks object-groups:

```
CLIENTS = 10.60.0.0/24
SERVERS = 10.60.1.10/32
WEB      = tcp/80
```

Expand into concrete rules; keep the group doc next to the ruleset in git.

Change control for ACLs

1. Write policy diff in markdown.
2. Add rules in lab.
3. Positive + negative tests.
4. Only then promote.
5. Never “temporary allow any” without a ticket/expiry note.

Verification checklist

Check	Intent
Policy doc matches counters	Rules do what you think
Default action explicit	No surprise open
Negative tests	Deny paths proven
Management path	You cannot lock yourself out without console plan
Logs sampled	Drops are visible

Common mistakes

Mistake	Symptom
Allow after broad deny	Never hits
Filter only one direction	One-way connectivity
Open SSH to world in lab configs committed	Habit risk
Logging every drop at full rate	Disk / CPU melt
Confusing RIB filter with packet filter	False confidence

Summary

- ACLs are **ordered policies** with an explicit default stance
- Separate **packet**, **route**, and **control-plane** filtering in your head
- Linux netfilter in Containerlab is enough to practice real data-plane policy
- Always pair positive allows with **negative** tests
- Write the policy in prose before the rule engine dialect

Next: **control-plane protection**—stop the CPU from drowning while the data plane still forwards.

Control-Plane Protection

Control-Plane Protection

The data plane can forward millions of packets while a handful of exception packets **kill the control plane**. Control-plane protection (CoPP / CPPr / RE protect—names vary) rate-limits and classifies traffic **to the device CPU**. This chapter builds the model with Linux and FRR-facing labs (Containerlab, mid-2026 open stack).

Learning goals

By the end of this chapter you can:

- Distinguish transit traffic from traffic destined to the router
- List common control-plane protocols worth protecting
- Apply Linux rate limits / policers toward local services
- Design a CoPP policy matrix (class → police → action)
- Predict symptoms of over-tight vs over-loose protection

Concepts

Two paths through a router

```
Transit (data plane):  in → FIB lookup → out
Receive (to CPU):      in → "this is for me" → process
```

Examples to CPU	Examples transit
BGP, OSPF, SSH, SNMP, NTP	Customer flows through the box
ARP to router MAC	Switched L2 through (L2 device)
ICMP echo to router IP	ICMP through to remote host
TTL expiry messages	Normal forwarded packets

Why unprotected CPUs melt

- Routing protocol floods / session storms
- Management scans (SSH/HTTPS)
- ICMP floods to the router IP
- ARP storms
- Fragment / exception paths

Symptoms: neighbor flaps, slow CLI, missed hellos → **self-inflicted outage** even if bandwidth remains.

Policy matrix (start simple)

Class	Match idea	Police	Notes
Critical routing	OSPF/BGP from known peers	generous	Never starve hellos blindly
Management	SSH/HTTPS from mgmt net	moderate	Jump host only
Monitoring	SNMP/telemetry agents	moderate	
Normal ICMP to RE	echo to router	low	
Default to RE	everything else	very low / drop	

Vendors implement class-maps + policers. Linux: nftables meter/limit, tc, or firewall rate modules.

Hardening adjacent controls

CoPP is **not** a substitute for:

- Interface ACLs that drop garbage early
- Management plane VRF / out-of-band
- Routing policy
- Authentication on protocols

Defense in depth: each layer different failure mode.

Lab topology

```
attacker ----|                |---- peer (OSPF/BGP)
mgmt  -----|  router  |
client -----|                |---- transit target
```

name: copp-lab

topology:

nodes:

rtr:

kind: linux

image: quay.io/frrouting/frr:10.2.1

binds:

- ./config/rtr:/etc/frr

also need iptables in image or side package

peer:

kind: linux

image: quay.io/frrouting/frr:10.2.1

binds:

- ./config/peer:/etc/frr

mgmt:

kind: linux

image: alpine:3.20

exec:

- apk add --no-cache iproute2 iputils openssh-client

attacker:

kind: linux

image: alpine:3.20

exec:

- apk add --no-cache iproute2 iputils

client:

kind: linux

image: alpine:3.20

exec:

- apk add --no-cache iproute2 iputils

target:

kind: linux

image: alpine:3.20

exec:

- apk add --no-cache iproute2 iputils

links:

- endpoints: ["rtr:eth1", "peer:eth1"]
- endpoints: ["rtr:eth2", "mgmt:eth1"]
- endpoints: ["rtr:eth3", "attacker:eth1"]

```
- endpoints: ["rtr:eth4", "client:eth1"]
- endpoints: ["rtr:eth5", "target:eth1"]
```

Addressing

Link	Subnet
rtr-peer	10.0.0.0/30 (.1 rtr, .2 peer)
rtr-mgmt	10.255.0.0/30 (.1 rtr)
rtr-attacker	10.66.0.0/30
rtr-client	10.10.0.0/24 (.1 rtr, .10 client)
rtr-target	10.20.0.0/24 (.1 rtr, .10 target)
rtr lo	192.0.2.1/32

Enable forward for transit client→target; BGP or static for peer experiments.

Baseline routing (static snack)

```
docker exec clab-copp-lab-rtr sysctl -w net.ipv4.ip_forward=1
# addresses omitted-set on each node per table
docker exec clab-copp-lab-client ip route add default via 10.10.0.1
docker exec clab-copp-lab-target ip route add default via 10.20.0.1
```

Optional OSPF between rtr and peer on eth1 (reuse interior-routing habits).

Linux receive-path protection (teaching CoPP)

Protect **SSH to the router** and **ICMP to the router**, without blocking transit.

```
# On rtr (needs iptables in container-install if FRR image lacks it)
docker exec clab-copp-lab-rtr sh -c 'command -v iptables || (apt-get update && apt-get install iptables)
2>/dev/null || true
```

FRR images vary; if package install is painful, run CoPP rules on an Alpine “router” with `ip_forward` and FRR only when needed. Pattern matters more than package manager.

```
docker exec clab-copp-lab-rtr sh -c '
# INPUT chain = traffic to local stack (control/management plane analogue)
iptables -N COPP 2>/dev/null || iptables -F COPP
iptables -F COPP

# BGP from peer only (TCP 179)
```

```

iptables -A COPP -p tcp -s 10.0.0.2 --dport 179 -j ACCEPT
iptables -A COPP -p tcp -s 10.0.0.2 --sport 179 -j ACCEPT

# OSPF (IP proto 89) from peer subnet
iptables -A COPP -p 89 -s 10.0.0.0/30 -j ACCEPT

# SSH from mgmt only, rate-limit new connections
iptables -A COPP -p tcp -s 10.255.0.0/30 --dport 22 -m conntrack --ctstate NEW -m limit --li
iptables -A COPP -p tcp -s 10.255.0.0/30 --dport 22 -m conntrack --ctstate ESTABLISHED -j AC

# ICMP to self: low rate
iptables -A COPP -p icmp --icmp-type echo-request -m limit --limit 5/sec -j ACCEPT

# established to self
iptables -A COPP -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# drop other to-self
iptables -A COPP -j DROP

iptables -I INPUT 1 -j COPP
'

```

Critical: do **not** put transit in INPUT. FORWARD remains separate:

```

docker exec clab-copp-lab-rtr sh -c '
iptables -P FORWARD ACCEPT
# or tighter ACL from previous chapter
'

```

Predict → observe → fix

Drill 1 — Transit still works under “attack” to RE

```

# flood ping to router IP from attacker
docker exec clab-copp-lab-attacker sh -c 'ping -f -c 1000 10.66.0.1 || ping -c 200 10.66.0.1
# simultaneous transit
docker exec clab-copp-lab-client ping -c 20 10.20.0.10

```

Predict: transit mostly fine; many attacker pings dropped by limit; CLI may still respond if limits work.

Observe: `iptables -L COPP -n -v` counters.

Fix if transit dies: rules mistakenly in FORWARD; CPU still overloaded—tighten attacker path earlier (interface ACL).

Drill 2 — SSH from non-mgmt

```
docker exec clab-copp-lab-attacker sh -c 'nc -w2 10.66.0.1 22 || echo blocked'
docker exec clab-copp-lab-mgmt sh -c 'nc -w2 10.255.0.1 22 || true'
```

Predict: attacker blocked; mgmt allowed (if sshd up).

Drill 3 — Over-tight BGP protection

Intentionally rate-limit TCP 179 too aggressively:

```
iptables -I COPP 1 -p tcp --dport 179 -m limit --limit 1/hour -j ACCEPT
iptables -I COPP 2 -p tcp --dport 179 -j DROP
```

Predict: BGP session flaps or won't establish.

Observe: vtysh -c 'show bgp summary'.

Fix: remove bad rules; document that **routing classes need headroom**.

Drill 4 — Peer-only OSPF

Send spoofed or neighbor attempt from attacker interface (concept): OSPF from non-peer should not be accepted.

Observe: no adjacency from wrong interface; INPUT drops proto 89 from wrong sources.

FRR-side levers (awareness)

Lever	Role
ttl-security / GTSM	BGP hop check
MD5/TCP-AO (as available)	Session auth
Interface ACLs + passive-interface	Reduce attack surface
Max prefix	Limit RIB blowups
BFD timers sanity	Avoid micro-flap storms

These complement packet CoPP; they do not replace it.

Building a CoPP document

```
Device role: lab edge
Classes:
- ROUTING_PEERS: list IPs/protos, police 1Mbps, prio high
```

```

- MGMT: 10.255.0.0/30 tcp/22, 100kbps
- ICMP_RE: 64kbps
- DEFAULT_RE: 32kbps drop excess
Exceptions: console always
Change window: test in lab before prod
Rollback: saved ruleset file in git

```

Store as copp-policy.md beside topology.

IPv6 notes

Same model: traffic to link-local and global router addresses hits receive path. ND/RA are control-plane-adjacent—filter carefully or you break neighbor discovery.

Optional SR Linux / NOS CoPP

Commercial and community NOS expose CoPP/CPM filter policies in YANG. Map your matrix classes to their filter entries; validate with protocol sessions + flood tests. Free path still proves the **idea** on Linux.

Verification checklist

Check	Intent
Transit ping during RE flood	Data vs control separation
Known peer protocols up	Not over-policed
Unknown src management down	Surface reduced
Counters move on floods	Policy hit
Rollback file exists	Safety

Common mistakes

Mistake	Symptom
Policing FORWARD as “CoPP”	Kills customers, not RE
Starving BGP/OSPF	Random reconvergence
No mgmt exception	Locked out
Only cloud dashboards, no on-box	Outage of mgmt path blind
Copy-paste prod CoPP into lab scale	Lab protocols die

Summary

- Control-plane protection polices **traffic to the CPU**, not transit
- Build a **class matrix** before touching syntax
- Linux INPUT rate limits teach the model; NOS CoPP is the same idea
- Always retest **routing adjacencies** after tightening
- Combine with mgmt ACLs, auth, and max-prefix for defense in depth

Next: **QoS vocabulary**—how queues and marks express intent without vendor exam laundry lists.

QoS Vocabulary

QoS Vocabulary

Quality of Service is a pile of vendor features if you start from menus. Start from **vocabulary and intent**: classify → mark → queue → schedule → (maybe) shape/police. This chapter stays conceptual with small Linux `tc` experiments in Containerlab—no paid QoS licenses (mid-2026).

Learning goals

By the end of this chapter you can:

- Explain classification, marking, queuing, scheduling, policing, shaping
- Map DSCP/PHB ideas without memorizing every codepoint table
- Build a minimal Linux `htb/fq_codel` lab and measure fairness under load
- Predict what happens when you police vs shape
- Know what *not* to promise with lab QoS

Concepts — the pipeline

packets in

Classify (match: 5-tuple, interface, DSCP, VLAN, app)

Mark (set DSCP / internal class / MPLS TC / VLAN PCP)

Police (drop/remark excess - hard limit)

Queue (buffers per class)

Schedule (which queue sends next: PQ, WRR, WFQ-like...)

Shape (smooth egress to rate - buffer then send)

wire

Not every hop does every stage. Trust boundaries matter: **remark at edges**, honor marks in the core only if you trust them.

Vocabulary table

Term	Meaning
Classification	Deciding which class a packet belongs to
Marking	Writing a field others can see (e.g. DSCP)
PHB	Per-hop behavior—what a class <i>should</i> experience
Policing	Enforce rate; excess drop or remark (little buffering)
Shaping	Enforce rate by delaying (buffering) excess
Queue depth	How much can wait; latency vs drop trade-off
Congestion avoidance	ECN, RED/WRED-like early drop/mark
Scheduling	Priority vs sharing bandwidth among classes
LLQ / priority queue	Low-latency class that can starve others if unbounded
Fair queueing	Prevent one flow from dominating (fq / fq_codel ideas)
Trust boundary	Where you accept or rewrite marks

DSCP / PHB (operator view)

Intent class	Typical idea	Notes
Network control	Highest care for protocols	Protect, don't just "EF everything"
Expedited (EF-like)	Low latency, low loss	Strict admission; small volume
Assured / AF-like	Business data tiers	Drop precedence inside class
Default / BE	Best effort	Most traffic
Scavenger	Below BE	Backups, bulk

Exact codepoints are standards-listed; **do not** treat exam bingo as design. Document *your* map:

EF-like → voice lab class (tiny)
AF31-like → interactive
CS6-like → routing (if marked)
BE → default

Policing vs shaping

	Police	Shape
Excess	Drop/remark	Delay in buffer
Latency	Can be lower for in-profile	Adds delay when congested
Use	Enforce contract, ingress	Match egress to circuit rate

Predict: shaping a 100 mbit stream to 10 mbit increases latency under load; policing 10 mbit drops packets and breaks TCP more brutally.

Linux lab topology

```
name: qos-vocab

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    - endpoints: ["r1:eth2", "h2:eth1"]
```

Addressing

Node	IP
h1	10.80.1.10/24 via 10.80.1.1
r1 eth1	10.80.1.1/24

Node	IP
r1 eth2	10.80.2.1/24
h2	10.80.2.10/24 via 10.80.2.1

```
docker exec clab-qos-vocab-r1 sysctl -w net.ipv4.ip_forward=1
# apply addresses on all nodes...
```

Experiment 1 — bottleneck without QoS

```
# limit r1 eth2 with a simple tbf shape to create congestion
docker exec clab-qos-vocab-r1 tc qdisc replace dev eth2 root tbf rate 5mbit burst 32kbit lat

docker exec -d clab-qos-vocab-h2 iperf3 -s
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -t 20 -P 4
```

Predict

Aggregate ~5 Mbit; latency rises; flows share roughly (TCP dynamics).

Observe

```
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -t 10 -J | head
docker exec clab-qos-vocab-r1 tc -s qdisc show dev eth2
```

Experiment 2 — two classes with HTB

Goal: bulk class 3 mbit ceiling; interactive class 2 mbit guaranteed-ish; total 5 mbit.

```
docker exec clab-qos-vocab-r1 sh -c '
tc qdisc del dev eth2 root 2>/dev/null || true
tc qdisc add dev eth2 root handle 1: htb default 20
tc class add dev eth2 parent 1: classid 1:1 htb rate 5mbit ceil 5mbit
tc class add dev eth2 parent 1:1 classid 1:10 htb rate 2mbit ceil 5mbit prio 1
tc class add dev eth2 parent 1:1 classid 1:20 htb rate 3mbit ceil 5mbit prio 2
tc qdisc add dev eth2 parent 1:10 handle 10: fq_codel
tc qdisc add dev eth2 parent 1:20 handle 20: fq_codel
# classify by destination port: "interactive" = 5202, bulk = 5201
tc filter add dev eth2 protocol ip parent 1:0 prio 1 u32 match ip dport 5202 0xffff flowid 1
tc filter add dev eth2 protocol ip parent 1:0 prio 2 u32 match ip dport 5201 0xffff flowid 1
'
```


Run two iperf servers/ports on h2:

```
docker exec -d clab-qos-vocab-h2 iperf3 -s -p 5201
docker exec -d clab-qos-vocab-h2 iperf3 -s -p 5202
docker exec -d clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -p 5201 -t 30 -P 2
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -p 5202 -t 20
```

Predict

Interactive class gets better latency / share under contention than pure bulk; total still capped ~5 mbit.

Observe

```
docker exec clab-qos-vocab-r1 tc -s class show dev eth2
docker exec clab-qos-vocab-r1 tc -s qdisc show dev eth2
```

Fix

Wrong filter → all traffic in default class. Check flowid and dport match (u32 host byte care).

Experiment 3 — police vs shape feel

```
# police ingress-ish on r1 eth1 (example)
docker exec clab-qos-vocab-r1 sh -c '
tc qdisc replace dev eth1 handle ffff: ingress
tc filter add dev eth1 parent ffff: protocol ip u32 match u32 0 0 \
  police rate 2mbit burst 20k drop flowid :1
'
```

Compare TCP goodput and retransmissions vs tbf shape on egress.

Predict: police drops → TCP sawtooth pain; shape delays → higher RTT, smoother rate.

Marking sketch (DSCP)

```
# set DSCP on packets from h1 (example iptables)
docker exec clab-qos-vocab-h1 sh -c '
apk add --no-cache iptables
iptables -t mangle -A OUTPUT -p tcp --dport 5202 -j DSCP --set-dscp 0x2e
'
```

```
# on r1 classify on DSCP instead of dport (tc flower/u32 match)
```

Predict

If r1 trusts and classifies EF-like mark into priority class, marked flows win under load.

Observe

```
docker exec clab-qos-vocab-r1 sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 -v -c 5 tc
```

What lab QoS cannot prove

- Hardware queue behavior of a specific ASIC
- Carrier hierarchical QoS at scale
- Exact voice MOS scores
- Multi-vendor PHB interoperability without careful design

It **can** prove: classification logic, bottlenecks, unfairness, and the difference between police/shape.

FRR / NOS note

FRR is not your main QoS engine. Real NOS (including SR Linux community) expose class-maps and queues in their models. Translate:

```
class VOICE → match DSCP EF → queue LLQ-like → police admit  
class BULK  → match default → shaped remainder
```

Same vocabulary.

Design mini-checklist

- ☐ Trust boundary defined
- ☐ Classes 4–5 for first design
- ☐ Admission for low-latency class

- Capacity math: sum guarantees link
- Congestion metrics (drop, latency, ECN) collected
- Failure mode: QoS mis-class worse than no QoS?

Predict → observe → fix template

Step	Example
Predict	Bulk should not starve interactive under 5 mbit cap
Observe	iperf + <code>tc -s</code> + ping RTT during load
Fix	filters, class rates, default class, ECN/fq_codel

Common mistakes

Mistake	Symptom
Mark without classifying later	Marks are cosmetics
Guarantees sum > link	Unexpected drops
Priority unbounded	Starvation of BE
QoS to “fix” bandwidth shortage	Still need capacity
Policing management/routing	Self-outage (see CoPP)

Summary

- QoS is a **pipeline of intents**, not a single CLI feature
- Classify → mark → queue → schedule → police/shape
- Linux `tc` + iperf3 in Containerlab is enough to feel congestion policy
- Document trust boundaries and class maps in git
- Keep routing/management classes out of scavenger police

Next: **hardening checklists**—turn models into repeatable gates for labs and edges.

Hardening Checklists

Hardening Checklists

Hardening is not a one-time “secure the router” checkbox. It is a **repeatable set of gates** you run after every design change. This chapter consolidates edge, control-plane, ACL, and ops hygiene into checklists you can attach to Containerlab labs (mid-2026 open stack: FRR, Linux, optional SR Linux notes).

Learning goals

By the end of this chapter you can:

- Run role-based hardening checklists (edge, P, lab host)
- Turn checklist items into automated `verify-hard.sh` probes
- Balance security with operability (break-glass paths)
- Record exceptions with expiry and owner
- Apply the same gates to dual-home and fabric labs

How to use these lists

1. **Pick the role** of the device or lab.
2. Mark each item: Pass / Fail / N/A / Exception.
3. Failures become backlog; exceptions need owner + date.
4. Automate the boring Pass/Fail checks.
5. Re-run after every topology or policy change.

Design → Implement → Functional verify → Hardening verify → Document

Checklist 0 — Lab hygiene (every topology)

#	Gate	Pass looks like
0.1	Topology in git	<code>.clab.yml</code> + configs committed
0.2	Addressing plan	No overlapping accidental spaces
0.3	Secrets	No production passwords in repo
0.4	Destroy works	<code>containerlab destroy -c clean</code>
0.5	Logs bound	Evidence survives destroy
0.6	Images free	Main path needs no paid license
0.7	Resource note	RAM/CPU expected documented

Checklist 1 — Management plane

#	Gate	Pass looks like
1.1	SSH/API not on all interfaces blindly	Listen limited or filtered
1.2	Mgmt sources limited	ACL/CoPP allow jump net only
1.3	Auth	Keys or strong lab passwords; no default vendor admin left
1.4	Banner / MOTD	Optional but marks non-prod
1.5	Out-of-band story	Console or dual path to recover
1.6	Telemetry endpoints known	Who scrapes what
1.7	DNS for mgmt names	Optional but reduces fat-finger

Linux probes

```
# who listens?
docker exec clab-X-r1 ss -lntp
# INPUT policy not wide open without reason
docker exec clab-X-r1 iptables -L INPUT -n -v | head
```

Checklist 2 — Control plane & routing

#	Gate	Pass looks like
2.1	Passive interfaces on access LANs	No accidental OSPF on user ports
2.2	BGP peer ACLs / TTL security as applicable	Only intended peers
2.3	Max-prefix on eBGP	Limit configured + action known
2.4	Prefix filters in and out	Negative hijack test fails closed

#	Gate	Pass looks like
2.5	Router-IDs unique	Documented
2.6	Auth on IGP/BGP where in scope	Or explicit exception
2.7	CoPP/receive ACL present on edges	Matrix documented
2.8	Loop prevention	No rediscovery of classic redistrib loops

FRR probes

```
docker exec clab-X-r1 vtysh -c 'show ip ospf interface'
docker exec clab-X-r1 vtysh -c 'show bgp summary'
docker exec clab-X-r1 vtysh -c 'show run' | grep -E 'prefix-list|route-map|max-prefix|passiv
```

Checklist 3 — Data plane filters

#	Gate	Pass looks like
3.1	Written policy exists	Prose before rules
3.2	Default stance explicit	Deny or allow documented
3.3	Positive tests	Needed paths work
3.4	Negative tests	Forbidden paths fail
3.5	Anti-spoof (uRPF-like idea)	As scope allows
3.6	Directed broadcast / dangerous services off	No chargen era nonsense
3.7	IPv6 filters parity	Not “v6 open because forgot”

Checklist 4 — Edge services

#	Gate	Pass looks like
4.1	DHCP pools correct	giaddr/relay tested
4.2	NAT scoped	Only intended interfaces
4.3	VRRP/anycast VIP documented	Single master observed
4.4	DNS dependency known	Outage drill done once
4.5	Time sync	chrony/ntp status OK
4.6	Logging on role changes	VRRP/BGP flap visible

Checklist 5 — Secrets, users, supply chain

#	Gate	Pass looks like
5.1	Image sources pinned	Digests/tags known
5.2	No shared root SSH everywhere	Or accepted lab-only risk noted
5.3	CI does not print secrets	Logs clean
5.4	Third-party scripts reviewed	Especially curl

Checklist 6 — Resilience & abuse

#	Gate	Pass looks like
6.1	Link failure drill	Recover within budget
6.2	Peer failure drill	Documented
6.3	Flood to RE drill	Transit survives
6.4	Config rollback	Known good in git
6.5	Capacity headroom	Lab not at 100% RAM always

Role profiles (quick)

Lab host (Containerlab hypervisor)

- Disk for images; unused labs destroyed
- Access limited to operators
- systemctl baseline documented
- Docker/podman not exposed to world

FRR edge CE

- Checklists 1–4 heavily
- Dual-home policies + VRRP tracking story
- Export only owned space

FRR P / fabric leaf

- Minimal services listening
- Underlay filters; no customer ACLs confusion
- ECMP + BFD timer sanity
- CoPP light but present

Linux bridge “switch”

- No IP on data plane unless needed
- STP/loop story if L2 loops possible
- Do not run random services on bridge NS

Exception register template

ID	Control	Reason	Owner	Expiry	Review
E-001	BGP without TTL security	Lab only single hop	you	2026-12-01	

Exceptions without expiry become permanent holes.

Automating gates — `verify-hard.sh`

```
#!/usr/bin/env bash
set -euo pipefail
LAB=${LAB:-clab-dualhome}
fail() { echo "FAIL: $" >&2; exit 1; }
pass() { echo "PASS: $"; }

# 2.4 negative: hijack absent on pe1
if docker exec ${LAB}-pe1 vtysh -c 'show bgp ipv4 uni' 2>/dev/null | grep -q '203.0.113.0';
    fail "hijack prefix present"
else
    pass "no hijack prefix"
fi

# 1.2 example: ssh port not open to all - soft check listening interfaces
```

```

docker exec ${LAB}-ce1 ss -lntp | grep -q ssh && pass "sshd present" || pass "no sshd (ok if

# 4.3 VIP answers
docker exec ${LAB}-h1 ping -c1 -W1 192.168.50.254 >/dev/null && pass "VIP" || fail "VIP"

# 0.1 topology file exists
test -f dualhome.clab.yml && pass "topology file" || fail "missing topology"

echo "HARDENING SMOKE OK"

```

Wire into Makefile:

```

smoke: ; ./verify.sh
hard:  ; ./verify-hard.sh

```

Predict → observe → fix (meta-drill)

Break one control on purpose:

```
# remove FROM-CE filter on pe1 in lab
```

Predict | `verify-hard.sh` fails hijack test when you advertise junk |
 Observe | script FAIL line |
 Fix | restore route-map; re-run |

Never disable a gate silently—disable via exception register.

Mapping to prior chapters

Chapter theme	Checklist
ACL thinking	3.x
CoPP	2.7, 6.3
QoS	don't police control into death; document classes
Dual-home edge	2.x + 4.x
Name/time/log	4.4–4.6, 0.5

Optional SR Linux

Export checklist to NOS-specific show commands in an appendix note. Same gates: peer authenticity, prefix limits, mgmt ACL, logging. Free community image is for dialect practice, not a second security model.

Definition of done — hardened lab

- ☐ Role checklist completed with Pass/Fail
- ☐ Exceptions registered
- ☐ `verify.sh` + `verify-hard.sh` green
- ☐ One intentional fail demonstrated (gate works)
- ☐ Rollback path known

Common mistakes

Mistake	Symptom
Checklist once, never again	Drift
Automating only happy path	False green
Lockout without console	Unrecoverable lab (or prod!)
Copying bank-grade CoPP into tiny lab	Protocols die
Paper checklists not in repo	Lost tribal knowledge

Summary

- Hardening is **repeatable gates**, not a mood
- Use role-based checklists + exception register
- Automate negative tests (hijack, flood survival, mgmt limits)
- Re-run after every material change
- Operability (break-glass) is part of security

Next part: **overlays and tunnels**—why we encapsulate, and how free stacks practice VXLAN/EVPN-class ideas.

Overlays and Tunnels

Why Encapsulate

Why Encapsulate

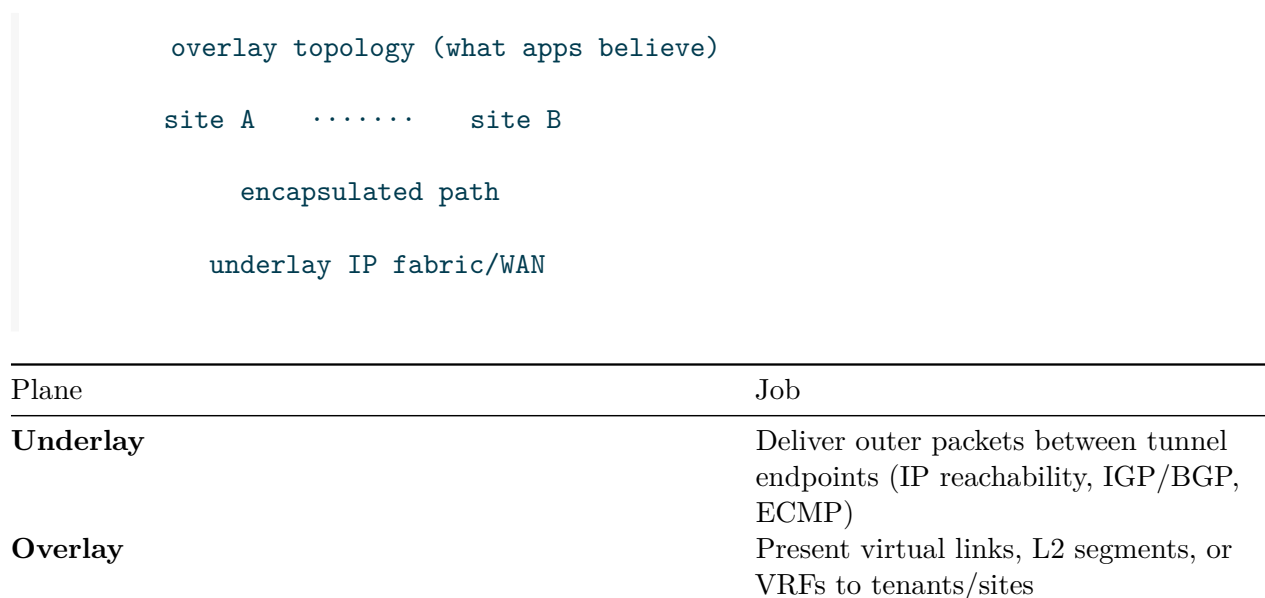
Encapsulation is not a fashion—it is how networks **reuse an underlay** while giving tenants or sites a different topology, address space, or policy domain. This chapter builds durable models before GRE/IPsec/VXLAN/EVPN syntax (Containerlab + FRR + Linux, mid-2026 free path).

Learning goals

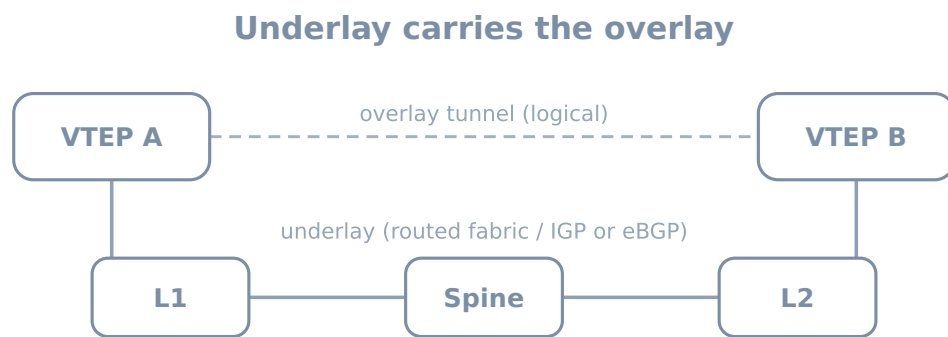
By the end of this chapter you can:

- Separate underlay from overlay responsibilities
- List reasons to tunnel or encapsulate (and reasons not to)
- Trace a packet through outer/inner headers
- Predict MTU, ECMP entropy, and troubleshooting failure modes
- Map encapsulation choices to common design problems

The core picture



Plane	Job
Control (overlay)	Who is where (EVPN-class, controller, static maps)
Data (overlay)	Encapsulation format (VXLAN, GRE, Geneve, IPsec...)



If overlay fails: check tunnel endpoints, then underlay reachability, then policy.

Free-lab friendly: FRR or SR Linux leafs + Linux endpoints in Containerlab.

Figure 1: Underlay vs overlay

Why people encapsulate

Driver	What encapsulation buys
Overlap addresses	Two tenants both use 10.0.0.0/8
L2 adjacency over L3	VM mobility, some clusters, legacy apps
Policy domains	Separate routing tables / VRFs
Traffic engineering	Steer logical topology physical
Multi-site stretch	Same segment in two DCs (careful!)
Security boundary	IPsec provides crypto + integrity
Abstraction	Apps ignore underlay renumbering

Why *not* to encapsulate

Temptation	Cost
Tunnel everything by default	MTU tax, state, debug opacity

Temptation	Cost
Stretch L2 globally	Failure domains explode
Overlay without underlay skill	“EVPN down” actually IGP down
Double NAT + tunnel + hairpin	Undebuggable piles

Rule: prefer plain IP routing until a requirement forces an overlay.

Encapsulation as Russian dolls

Example mental stack (VXLAN-ish):

```
[ Eth ][ IP underlay ][ UDP ][ VXLAN ][ Eth inner ][ IP inner ][ TCP ][ payload ]
```

Header	Owned by
Outer IP	Underlay endpoints (VTEPs / tunnel routers)
Outer L4	Often UDP for ECMP entropy (VXLAN)
Overlay header	VNI / keys / flags
Inner frame	Tenant

GRE classic:

```
[ Eth ][ IP ][ GRE ][ IP inner ][ ... ]
```

IPsec tunnel mode adds crypto headers and often transports inner IP.

Identifiers you must name

Identifier	Role
Tunnel endpoint IP	Underlay address of VTEP/router
VNI / VPN id / GRE key	Separates segments
VRF	Routing table isolation
Route target / RD (EVPN/MPLS-ish)	Control-plane membership
Mapping	End-host IP/MAC → endpoint

Without a clear mapping system (static, flood-and-learn, EVPN), overlays become amateur radio.

Encapsulation (what rides inside what)



Figure 2: Encapsulation

Failure domains

```
Underlay down → all overlays using it down
One VTEP down → locals on that leaf isolated from fabric overlay
VNI mis-map → blackhole or cross-tenant leak (severe)
MTU blackhole → big packets die, small pings live
```

Always ask: **is this underlay, overlay data, or overlay control?**

MTU tax

Every header steals bytes. If underlay MTU is 1500 and you add ~50–100 bytes, inner needs lower MTU or underlay needs jumbo.

Symptom	Likely
Ping works, TCP hangs	PMTUD broken + encapsulation
Only large transfers fail	MTU
DF set drops	Need clamp or larger underlay

Lab habit: set explicit MTU on overlay interfaces and test large pings.

```
ping -M do -s 1472 10.0.0.1 # classic Ethernet payload probe
# inside overlay, lower size until success; compute header tax
```

ECMP and entropy

Underlays load-balance on outer five-tuple. If outer UDP/TCP ports are fixed, many inner flows hash to **one** path → polarization.

Design	Entropy
VXLAN UDP source port from inner flow hash	Good
GRE without entropy helpers	Often poor
IPsec without cleverness	Can polarize

When debugging “uneven spines,” check **outer** headers, not only inner apps.

Control plane vs data plane (again)

	Data plane only	With control plane
Static GRE	Manual who-to-whom	—
Flood-and-learn VXLAN	Dynamic MAC via flood	Limited scale
EVPN-class	Advertises reachability	Mapping distributed

This book’s later chapters: site tunnels (mostly static/service), VXLAN data plane, EVPN intro where free images allow.

Mini lab — see encapsulation with tcpdump

```
name: encap-why

topology:
  nodes:
    a:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
    b:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
  under:
    kind: linux
    image: alpine:3.20
```

```

exec:
  - apk add --no-cache iproute2 iputils tcpdump
  - sysctl -w net.ipv4.ip_forward=1

links:
  - endpoints: ["a:eth1", "under:eth1"]
  - endpoints: ["b:eth1", "under:eth2"]

```

GRE between a and b across under

```

# addresses: a 10.0.0.1/24, under eth1 10.0.0.254; b 10.0.1.1/24, under eth2 10.0.1.254
# underlay route a b via under

docker exec clab-encap-why-a sh -c '
  ip tunnel add gre1 mode gre remote 10.0.1.1 local 10.0.0.1 ttl 64
  ip link set gre1 up
  ip addr add 172.16.0.1/30 dev gre1
'

docker exec clab-encap-why-b sh -c '
  ip tunnel add gre1 mode gre remote 10.0.0.1 local 10.0.1.1 ttl 64
  ip link set gre1 up
  ip addr add 172.16.0.2/30 dev gre1
'

```

Predict → observe → fix

```

docker exec clab-encap-why-a ping -c 2 172.16.0.2
docker exec clab-encap-why-under tcpdump -ni eth1 -c 5 proto gre
# or: tcpdump -ni eth1 proto 47

```

Predict: underlay capture shows outer IP a→b with GRE; inner ICMP not as plain on underlay without decap.

Observe: header stack.

Fix: if no gre, check tunnel remote/local and underlay ping 10.0.1.1 from a.

Design questions checklist

Before choosing an overlay:

1. What problem is pure IP routing failing to solve?
2. L2 stretch or L3 VPN?

3. Who allocates VNIs / VRFs?
4. What is the underlay routing design?
5. MTU strategy?
6. How do we debug (mirror, drop counters, inner/outer captures)?
7. Failure domain acceptable?

Mapping to the journey map

```
Models (planes, encap)
→ underlay skill (L2/L3/IGP/BGP)
→ edge services
→ overlays (this part)
→ fabrics (scale the underlay + overlay)
```

Skipping underlay competence makes overlay chapters mystical.

Common mistakes

Mistake	Symptom
Troubleshooting only inner	Miss underlay loss
Ignoring MTU	Mystery TCP issues
One giant L2 VNI	Broadcast storms / failure blast
No inventory of VNIs	Collisions and leaks
Encrypt + fragment chaos	Performance cliff

Summary

- Encapsulation reuses underlays to build different logical networks
- Always name **underlay**, **overlay data**, **overlay control**, and **identifiers**
- MTU and ECMP entropy are not footnotes
- Prefer routing until requirements force tunnels
- A tiny GRE lab + tcpdump makes the model real

Next: **site tunnels**—practical GRE/IPsec-style site-to-site patterns on free Linux/FRR labs.

Site Tunnels

Site Tunnels

Site-to-site tunnels connect islands of private addressing across a routed underlay (lab “Internet” or WAN). This chapter implements **GRE and policy-based / route-based IPsec-style** patterns with Linux in Containerlab, optionally fronted by FRR for routing (mid-2026 free tools).

Learning goals

By the end of this chapter you can:

- Build a GRE tunnel and route site prefixes across it
- Explain route-based vs policy-based VPN approaches
- Run a simple WireGuard or LibreSwan/strongSwan-style lab path (tool availability permitting)
- Fail underlay links and predict overlay behavior
- Clamp MTU and verify with large pings and tcpdump

When site tunnels are the right tool

Use	Not the first tool for
Two branches over untrusted WAN	DC fabric east-west (use fabric overlay)
Quick lab multi-site	Massive multi-tenant scale (EVPN fabrics)
Crypto requirement	Cleartext GRE alone on hostile nets

Topology — two sites + “Internet”

```
h1--siteA--\                               /--siteB--h2
            +----- inet -----+

name: site-tun

topology:
  nodes:
    sitea:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump iptables wireguard-tools || true
    siteb:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump iptables wireguard-tools || true
    inet:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
        - sysctl -w net.ipv4.ip_forward=1
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["h1:eth1", "sitea:eth1"]
    - endpoints: ["h2:eth1", "siteb:eth1"]
    - endpoints: ["sitea:eth2", "inet:eth1"]
    - endpoints: ["siteb:eth2", "inet:eth2"]
```

Addressing plan

Object	Address
Site A LAN	192.168.10.0/24 (sitea .1, h1 .10)
Site B LAN	192.168.20.0/24 (siteb .1, h2 .10)
sitea WAN	198.51.100.1/30
inet eth1	198.51.100.2/30
siteb WAN	203.0.113.1/30
inet eth2	203.0.113.2/30
GRE outer	WAN IPs
GRE inner	10.255.255.0/30 (.1 a, .2 b)

```
# site A
docker exec clab-site-tun-sitea sh -c '
    sysctl -w net.ipv4.ip_forward=1
    ip addr add 192.168.10.1/24 dev eth1; ip link set eth1 up
    ip addr add 198.51.100.1/30 dev eth2; ip link set eth2 up
    ip route add 203.0.113.0/30 via 198.51.100.2
'

docker exec clab-site-tun-h1 sh -c '
    ip addr add 192.168.10.10/24 dev eth1; ip link set eth1 up
    ip route add default via 192.168.10.1
'

# site B + h2 analogous with 192.168.20.0/24 and 203.0.113.1
docker exec clab-site-tun-inet sh -c '
    ip addr add 198.51.100.2/30 dev eth1; ip link set eth1 up
    ip addr add 203.0.113.2/30 dev eth2; ip link set eth2 up
'
```

Lab 1 — GRE + static routes

```
docker exec clab-site-tun-sitea sh -c '
    ip tunnel add gre1 mode gre remote 203.0.113.1 local 198.51.100.1 ttl 64
    ip link set gre1 up mtu 1400
    ip addr add 10.255.255.1/30 dev gre1
    ip route add 192.168.20.0/24 dev gre1
'

docker exec clab-site-tun-siteb sh -c '
    ip tunnel add gre1 mode gre remote 198.51.100.1 local 203.0.113.1 ttl 64
    ip link set gre1 up mtu 1400
    ip addr add 10.255.255.2/30 dev gre1
    ip route add 192.168.10.0/24 dev gre1
'
```


Predict

- Underlay: sitea pings 203.0.113.1
- Overlay: h1 pings h2
- tcpdump on inet shows GRE (proto 47), not plain inner ICMP as clear tenant IP without decap context

Observe

```
docker exec clab-site-tun-sitea ping -c 2 203.0.113.1
docker exec clab-site-tun-h1 ping -c 3 192.168.20.10
docker exec clab-site-tun-inet tcpdump -ni eth1 -c 8 proto 47
docker exec clab-site-tun-sitea ip route get 192.168.20.10
```

Fix

Symptom	Check
No underlay	inet addrs, routes
Underlay OK overlay fail	gre remote/local swap, routes to LAN via gre
One-way	missing return route

Lab 2 — MTU blackhole drill

```
docker exec clab-site-tun-h1 ping -c 2 -M do -s 1472 192.168.20.10
docker exec clab-site-tun-h1 ping -c 2 -M do -s 1372 192.168.20.10
```

Predict

Large DF pings fail; smaller succeed if gre mtu 1400.

Fix: lower inner interface MTU, TCP MSS clamp, or raise underlay MTU.

```
docker exec clab-site-tun-sitea iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN
```

Lab 3 — Underlay failure

```
docker exec clab-site-tun-inet ip link set eth2 down
docker exec clab-site-tun-h1 ping -c 5 192.168.20.10
docker exec clab-site-tun-inet ip link set eth2 up
```

Predict

Overlay dies with underlay; no magic. Multi-homed underlay would need dual tunnels + routing.

Route-based vs policy-based VPN

Model	Idea	Lab analogue
Route-based	Tunnel is an interface; RIB steers prefixes into it	GRE, WireGuard wg0, VTI
Policy-based	SPD selects traffic by ACL selectors for crypto	classic “interesting traffic” IPsec

Route-based composes better with dynamic routing (run OSPF/BGP over tunnel—carefully).

Lab 4 — WireGuard route-based (if packages available)

```
# Generate keys on both sides (persist in lab folder for real work)
docker exec clab-site-tun-sitea sh -c '
  apk add --no-cache wireguard-tools
  wg genkey | tee /tmp/a.key | wg pubkey > /tmp/a.pub
'
# exchange pubs out of band in lab notes
```

```
docker exec clab-site-tun-sitea sh -c '
  ip link add wg0 type wireguard
  ip addr add 10.255.255.1/30 dev wg0
  wg set wg0 private-key /tmp/a.key listen-port 51820 \
    peer B_PUBLIC_KEY endpoint 203.0.113.1:51820 allowed-ips 192.168.20.0/24,10.255.255.2/32
  ip link set wg0 up
  ip route add 192.168.20.0/24 dev wg0
'
```

Mirror on siteb with `allowed-ips 192.168.10.0/24`. Open UDP 51820 on path (inet forwards by default).

Predict → observe

```
docker exec clab-site-tun-h1 ping -c 3 192.168.20.10
docker exec clab-site-tun-inet tcpdump -ni eth1 -c 5 udp port 51820
docker exec clab-site-tun-sitea wg show
```

If `wireguard-tools` or kernel module missing in container, treat as **optional**—GRE still teaches site tunnels. Host-level WireGuard between namespaces is an alternate craft path from lab-craft chapters.

FRR over GRE (dynamic optional)

Enable `zebra + ospfd` or `bgpd` on FRR images replacing Alpine sites:

```
interface gre1
 ip ospf area 0
router ospf
 ospf router-id 1.1.1.1
```

Predict: LAN prefixes appear via OSPF over tunnel.

Risk: if underlay also runs OSPF without careful design, you can create recursive routing (tunnel endpoint learned *via* tunnel). **Keep underlay and overlay routing domains separate.**

Policy-based IPsec awareness

```
Selectors: src 192.168.10.0/24 dst 192.168.20.0/24
Peer: 203.0.113.1
```

Tools: `strongSwan`, `LibreSwan`—configs are verbose; for this book, understand selectors + SA state:

```
# illustrative
ip xfrm state
ip xfrm policy
```

Debug with state/policy tables + outer ESP captures (proto 50).

Security notes (vendor-agnostic)

- GRE alone is **not** confidential
- Authenticate/encrypt on untrusted underlays
- Filter what enters the tunnel (no free transit for world)
- Manage keys as secrets (not committed plaintext in public repos)

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-site-tun-h1 ping -c2 -W1 192.168.20.10
docker exec clab-site-tun-h2 ping -c2 -W1 192.168.10.10
echo OK
```

Common mistakes

Mistake	Symptom
Tunnel up, no LAN routes	Only gre peer pings
Recursive routing	Flapping tunnel endpoint
MTU ignored	App failures
NAT on underlay without keepalive	Idle VPN dies
Asymmetric interesting traffic	Policy VPN one-way

Summary

- Site tunnels extend private sites across a routed underlay
- GRE is the clearest teaching tool; WireGuard is a modern route-based crypto option
- Separate underlay reachability from overlay routing
- Always test **MTU** and **underlay failure**
- Prefer route-based models when combining with dynamic routing

Next: **VXLAN data plane**—L2 segments over L3 underlays with VNI encapsulation.

VXLAN Data Plane

VXLAN Data Plane

VXLAN encapsulates Ethernet frames in UDP/IP so you can stretch **L2 segments over an L3 underlay**. This chapter focuses on the **data plane**: VNI, VTEP, flood behavior, and a Linux VXLAN lab in Containerlab (mid-2026). Control planes (EVPN) come next.

Learning goals

By the end of this chapter you can:

- Define VTEP, VNI, and underlay vs overlay roles for VXLAN
- Configure Linux `ip link add type vxlan` with static flood lists or bridge
- Capture VXLAN UDP (port 4789) and relate outer/inner headers
- Predict MAC learning and flooding behavior without EVPN
- Handle MTU for VXLAN overhead

Concepts

Roles

Term	Meaning
VTEP	VXLAN Tunnel End Point—encap/decap device
VNI	VXLAN Network Identifier—segment ID (~24-bit space)
Underlay	IP network between VTEP loopbacks/addresses
Overlay	Ethernet segments identified by VNI

```
VM/host -- L2 -- [ VTEP ]==== underlay IP ====[ VTEP ] -- L2 -- VM/host
                VNI 100                        VNI 100
```

Header (mental)

```
Outer Eth | Outer IP | UDP dport 4789 | VXLAN (VNI) | Inner Eth | Inner payload
```

UDP source port often encodes entropy for ECMP.

Data-plane only learning

Without EVPN (or a controller):

1. Unknown unicast / broadcast / multicast may **flood** to all VTEPs in the flood list (head-end replication) or underlay multicast group.
2. Source MAC learning binds MAC → remote VTEP.
3. Scale limits and wasteful flood—why EVPN exists.

Multicast vs head-end replication

Mode	Idea
Underlay multicast	BUM to group; needs multicast routing
Head-end replication (HER)	Ingress VTEP unicasts BUM to each peer VTEP

Labs often use **HER** / **static remote VTEP list** for simplicity.

When VXLAN is appropriate

- DC fabric tenant segments
- Lab simulation of multi-leaf L2
- Separating many logical L2s without many VLANs on underlay

Not always needed for simple L3-only campuses.

Topology — two VTEPs + underlay + hosts

```
name: vxlan-dp
topology:
  nodes:
```

```

leaf1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 bridge iputils tcpdump
leaf2:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 bridge iputils tcpdump
spine:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils tcpdump
    - sysctl -w net.ipv4.ip_forward=1
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
h2:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils

links:
- endpoints: ["h1:eth1", "leaf1:eth1"]
- endpoints: ["h2:eth1", "leaf2:eth1"]
- endpoints: ["leaf1:eth2", "spine:eth1"]
- endpoints: ["leaf2:eth2", "spine:eth2"]

```

Addressing

Node	Underlay	Access
leaf1 lo	10.0.0.1/32	—
leaf2 lo	10.0.0.2/32	—
leaf1 eth2	10.1.1.1/30	—
spine eth1	10.1.1.2/30	—
leaf2 eth2	10.1.2.1/30	—
spine eth2	10.1.2.2/30	—
h1	—	10.10.10.10/24
h2	—	10.10.10.20/24
VNI	100	same L2 segment

Node	Underlay	Access
------	----------	--------

Underlay routes: leaf1 reaches 10.0.0.2 via spine; advertise loopbacks with statics:

```
docker exec clab-vxlan-dp-leaf1 sh -c '
ip addr add 10.0.0.1/32 dev lo
ip addr add 10.1.1.1/30 dev eth2; ip link set eth2 up
ip route add 10.0.0.2/32 via 10.1.1.2
ip route add 10.1.2.0/30 via 10.1.1.2
'
docker exec clab-vxlan-dp-spine sh -c '
ip addr add 10.1.1.2/30 dev eth1; ip link set eth1 up
ip addr add 10.1.2.2/30 dev eth2; ip link set eth2 up
ip route add 10.0.0.1/32 via 10.1.1.1
ip route add 10.0.0.2/32 via 10.1.2.1
'
# leaf2 similar with 10.0.0.2 and 10.1.2.1
```

VXLAN + bridge on leaf1

```
docker exec clab-vxlan-dp-leaf1 sh -c '
ip link add br100 type bridge
ip link set br100 up
ip link add vxlan100 type vxlan id 100 \
    local 10.0.0.1 dev eth2 dstport 4789
# static remote VTEP (HER)
bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst 10.0.0.2
ip link set vxlan100 up
ip link set eth1 up
ip link set eth1 master br100
ip link set vxlan100 master br100
'
```

leaf2:

```
docker exec clab-vxlan-dp-leaf2 sh -c '
ip link add br100 type bridge
ip link set br100 up
ip link add vxlan100 type vxlan id 100 \
    local 10.0.0.2 dev eth2 dstport 4789
bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst 10.0.0.1
ip link set vxlan100 up
ip link set eth1 up
ip link set eth1 master br100
```

```
ip link set vxlan100 master br100
,
```

Hosts (same subnet, no default needed for local L2 test):

```
docker exec clab-vxlan-dp-h1 sh -c '
ip addr add 10.10.10.10/24 dev eth1; ip link set eth1 up
'
docker exec clab-vxlan-dp-h2 sh -c '
ip addr add 10.10.10.20/24 dev eth1; ip link set eth1 up
'
```

Syntax note: bridge fdb / ip link add vxlan flags vary slightly by iproute2 version; consult man ip-link on your image. Some use vxlan ... remote 10.0.0.2 for point-to-point single peer.

Deploy checks

```
docker exec clab-vxlan-dp-leaf1 ping -c 2 10.0.0.2
docker exec clab-vxlan-dp-h1 ping -c 3 10.10.10.20
docker exec clab-vxlan-dp-leaf1 bridge fdb show
docker exec clab-vxlan-dp-spine tcpdump -ni eth1 -c 10 udp port 4789
```

Predict

- Underlay loopback ping works
- Host ping works over VNI 100
- Spine sees UDP/4789, not bare inner ARP as local LAN

Observe

```
docker exec clab-vxlan-dp-spine tcpdump -ni eth1 -vv -c 5 udp port 4789
docker exec clab-vxlan-dp-leaf1 ip -d link show vxlan100
```

Fix

Failure	Checks
Underlay down	static routes, spine forward
No FDB flood entry	remote VTEP missing
VNI mismatch	id 100 both sides

Failure	Checks
Host not in bridge	master br100

Drill — MTU

VXLAN overhead commonly ~50 bytes class. Set underlay MTU 1550 or overlay-aware 1450:

```
docker exec clab-vxlan-dp-leaf1 ip link set vxlan100 mtu 1450
docker exec clab-vxlan-dp-leaf2 ip link set vxlan100 mtu 1450
docker exec clab-vxlan-dp-h1 ping -c 2 -M do -s 1472 10.10.10.20
docker exec clab-vxlan-dp-h1 ping -c 2 -M do -s 1400 10.10.10.20
```

Predict

Too-large DF fails; tuned sizes pass.

Drill — wrong VNI isolation

Create VNI 200 on leaf2 only for a third host experiment—or change leaf2 to id 200 temporarily:

Predict

h1 cannot reach h2 (different segments).

Fix

Align VNI; document allocation.

Drill — MAC move / learning

```
docker exec clab-vxlan-dp-leaf1 bridge fdb show br100
docker exec clab-vxlan-dp-h1 ping -c 1 10.10.10.20
docker exec clab-vxlan-dp-leaf1 bridge fdb show br100
```

Predict

After traffic, remote MAC points at vxlan device / dst VTEP.

Flood scope risk

Static 00:00:00:00:00:00 FDB means BUM replicates to listed VTEPs. More leaves → more replication. This is why control planes advertise **who needs which VNI**.

FRR / SR Linux awareness

NOS VTEPs bind VNI to VLAN/IRB and underlay loopbacks. Data-plane counters and `show vxlan` equivalents matter operationally. Linux lab teaches headers; production uses integrated L2/L3.

Verification checklist

Check	Intent
Underlay VTEP reachability	Outer IP works
UDP 4789 on path	Encap happening
Same VNI	Segment match
FDB remote	Mapping exists
Host ARP/ping	Inner L2
MTU tests	Size tax handled

Common mistakes

Mistake	Symptom
VTEP IP not loopback stable	Flaps when link IP changes
No underlay route to remote VTEP	Silent blackhole
Bridging mis-order	Loops or no pass
Expecting EVPN routes without config	Only static works
One huge flood list forever	BUM explosion

Summary

- VXLAN is Ethernet-in-UDP/IP with **VNI** segment IDs
- VTEPs encap/decap; underlay only sees outer IP/UDP
- Static HER labs teach data plane before EVPN
- Verify with underlay ping, host ping, and spine captures

- Plan MTU and flood scope deliberately

Next: **EVPN control-plane intro**—distributing MAC/IP and VNI membership without pure flood-and-learn.

EVPN Control-Plane Intro

EVPN Control-Plane Intro

EVPN (Ethernet VPN) is a **BGP-based control plane** for overlay reachability—most often paired with VXLAN data planes in modern fabrics. This chapter is an **introduction to models and lab reality on free stacks** (FRR EVPN/VXLAN where available, optional SR Linux community). Syntax shifts; objects do not.

Learning goals

By the end of this chapter you can:

- Explain what EVPN advertises versus what VXLAN carries
- Name core route types at a practical level (2, 3, 5—awareness)
- Relate RD/RT-style membership ideas to “who joins which VNI”
- Sketch an FRR EVPN + VXLAN lab and know how to verify BGP EVPN tables
- Predict failure when underlay OK but EVPN session down

Why EVPN exists

Data-plane-only VXLAN:

- Floods BUM to all peers
- Learns MAC reactively
- Scales poorly; slow on moves

EVPN:

- Distributes MAC/IP bindings and VTEP locations via BGP
- Reduces unknown flooding (in good designs)
- Supports L2 and L3 (IRB) services in one framework

```
BGP EVPN (control)      VXLAN/GRE/... (data)
```

```
same fabric intent
```

Objects (vendor-agnostic)

Object	Role
VTEP IP	Underlay endpoint
VNI	L2 segment id in data plane
MAC-VRF / EVI	Service instance
RD	Disambiguates routes
RT (route target)	Import/export membership
Type-2 route	MAC (/IP) advertisement
Type-3 route	Inclusive multicast / flood tree membership ideas
Type-5 route	IP prefix (L3 VPN style overlay)
IRB / L3 VNI	Inter-subnet routing in fabric

You do not need full RFC memorization to operate: **session up** → **EVPN routes present** → **data plane VNI up** → **hosts pass**.

Control vs data verification split

Layer	Healthy signs
Underlay	VTEP loopbacks ping; ECMP present
BGP EVPN	Neighbors Established; AFI/SAFI EVPN negotiated
EVPN RIB	Type-2 for host MACs; correct next-hop VTEP
Data plane	VXLAN encap; host ping; FDB consistent

Minimal design story — 2 leaf

```
h1--leaf1----spine----leaf2--h2
 \      underlay      /
  BGP EVPN (often via spine route-reflector or eBGP unnumbered fabric)
```

Common free lab patterns:

1. **iBGP EVPN** to a route reflector (spine or dedicated).
2. **eBGP EVPN** unnumbered on fabric links (advanced; FRR supports variants).

Start with **iBGP** + **RR** mentally even if your first lab is two leaves peering directly.

FRR capability note (mid-2026)

FRR continues to expand EVPN/VXLAN integration. Treat the following as **structural config**—verify against your image’s vtysh help and docs:

```
# daemons: zebra, bgpd, staticd (and platform vxlan integration)
router bgp 65000
  bgp router-id 10.0.0.1
  neighbor 10.0.0.3 remote-as 65000
  neighbor 10.0.0.3 update-source lo
  !
  address-family l2vpn evpn
    neighbor 10.0.0.3 activate
    advertise-all-vni
  exit-address-family
  !
# VNI / bridge mapping is platform-specific:
# Linux FRR may integrate with kernel bridge/vxlan via zebra
```

Always confirm with:

```
vttysh -c 'show bgp l2vpn evpn summary'
vttysh -c 'show bgp l2vpn evpn'
vttysh -c 'show evpn vni'
vttysh -c 'show evpn mac vni all'
```

If a command is missing, your build differs—fall back to kernel VXLAN static (previous chapter) and read FRR release notes.

Topology sketch (EVPN lab)

```
name: evpn-intro

topology:
  nodes:
    leaf1:
      kind: linux
      image: quay.io/frROUTING/frr:10.2.1
      binds:
        - ./config/leaf1:/etc/frr
    leaf2:
      kind: linux
      image: quay.io/frROUTING/frr:10.2.1
      binds:
        - ./config/leaf2:/etc/frr
```

```

rr:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/rr:/etc/frr
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
h2:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils

links:
  - endpoints: ["leaf1:eth1", "rr:eth1"]
  - endpoints: ["leaf2:eth1", "rr:eth2"]
  - endpoints: ["h1:eth1", "leaf1:eth2"]
  - endpoints: ["h2:eth1", "leaf2:eth2"]

```

Underlay: give leaf1/leaf2/rr loopbacks 10.0.0.1/32, .2, .3 and point-to-point links with static or OSPF so loopbacks mesh. EVPN BGP between leaves and rr using loopbacks.

RR config idea

```

router bgp 65000
  bgp router-id 10.0.0.3
  neighbor 10.0.0.1 remote-as 65000
  neighbor 10.0.0.1 update-source lo
  neighbor 10.0.0.2 remote-as 65000
  neighbor 10.0.0.2 update-source lo
  address-family l2vpn evpn
    neighbor 10.0.0.1 activate
    neighbor 10.0.0.1 route-reflector-client
    neighbor 10.0.0.2 activate
    neighbor 10.0.0.2 route-reflector-client
  exit-address-family

```

Predict → observe → fix drills

Drill 1 — Session before service

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn summary'
```

Predict: Established to RR.

If Idle: underlay loopback ping, update-source, firewall, daemons.

Drill 2 — Host MAC appears as EVPN route

```
docker exec clab-evpn-intro-h1 ping -c 2 10.10.10.20
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn'
docker exec clab-evpn-intro-leaf2 vtysh -c 'show evpn mac vni all'
```

Predict: Type-2-like entries; next-hop remote VTEP.

Fix: VNI advertise config, bridge binding, host traffic to trigger learning if required.

Drill 3 — Kill EVPN, keep underlay

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'conf t' -c 'router bgp 65000' -c 'neighbor 10.0.
docker exec clab-evpn-intro-h1 ping -c 5 10.10.10.20
```

Predict: underlay pings between leaves still work; overlay host traffic fails or relies on stale/f flood only.

Observe: BGP down, EVPN table ages out.

Fix: no neighbor ... shutdown.

Drill 4 — RT mismatch (concept)

If export/import RTs do not match, routes exist on one leaf but are not imported on the other.

Predict: show bgp l2vpn evpn has routes locally originated but remote missing or not installed.

Fix: align VRF/VNI RT policy.

L2 VNI vs L3 VNI (awareness)

Service	Host experience	EVPN idea
L2 VNI	Same subnet two leaves	MAC routes (type-2)
L3 VNI / IRB	Different subnets routed in fabric	Type-5 prefixes + symmetric/asymmetric IRB models

Do not implement every IRB mode on day one. Get **same-subnet across two leaves** solid first.

Optional SR Linux community path

Nokia SR Linux is a common Containerlab free/community kind for EVPN-VXLAN labs. Workflow:

1. Underlay eBGP or OSPF
2. Overlay BGP EVPN
3. `mac-vrf` / `vxlan-interface` style objects

Map to the same verification split. Use SR Linux documentation for exact CLI; journal the object mapping:

```
VNI    vxlan-interface
MAC-VRF bridge table
bgp-evpn advertisement
```

Security / isolation checklist

- VNI allocation authority
- RT import discipline (no accidental cross-tenant)
- Underlay ACL optional for UDP/4789 between VTEPs only
- Control-plane protection on BGP

What this intro deliberately skips

- Full multihoming Ethernet segments (ESI) deep dive
- Complex IRB interop matrices
- Multicast optimized BUM underlays
- DCI stretch best practices

Those are fabric/scale topics once the skeleton works.

Common mistakes

Mistake	Symptom
EVPN before underlay	Endless BGP down
Wrong AFI/SAFI	Session up but no EVPN
Data plane VNI advertised	Control OK, ping fail
Treating EVPN as L2 switch only	Miss L3 service models later
No RR scaling plan	Full mesh pain

Summary

- EVPN is BGP control for overlay membership and MAC/IP reachability
- VXLAN (or similar) remains the data plane
- Verify **underlay** → **BGP EVPN** → **EVPN RIB** → **host data** in order
- FRR and SR Linux are free-ish paths to practice; check your image's feature set
- Master two-leaf same-subnet before IRB gymnastics

Next: **overlay lab**—a single integrated workout combining underlay, VXLAN, and (if available) EVPN verification.

Overlay Lab

Overlay Lab

This chapter is a **portfolio lab**: build a small multi-site / multi-leaf overlay on free tools, break it on purpose, and prove each plane. Choose track A (GRE sites), track B (VXLAN static), or track C (EVPN if your FRR/SR Linux build supports it). Mid-2026 defaults: Containerlab + FRR + Alpine.

Learning goals

By the end of this chapter you can:

- Deliver one complete overlay lab folder that rebuilds from git
- Show captures proving encapsulation
- Run a failure matrix (underlay, tunnel endpoint, VNI/policy)
- Document MTU and security stance
- Meet the book's definition of done for overlay work

Pick a track

Track	Focus	Difficulty
A — Dual-site GRE/WG	Site tunnels + routing	
B — VXLAN HER two-leaf	L2 overlay data plane	
C — EVPN two-leaf + RR	Control + data	

Do **A then B** if new to overlays. Attempt C only after B host pings work.

Definition of done (all tracks)

- ☐ README.md with diagram + addressing
- ☐ *.clab.yml + configs/scripts in git

- ☐ Clean containerlab deploy on cold start
- ☐ `verify.sh` green
- ☐ Capture artifact (pcap or logged tcpdump) showing outer header
- ☐ 3 failure drills with notes
- ☐ MTU test recorded
- ☐ Retrospective: what was underlay vs overlay

Track A — Dual-site GRE (reference build)

Reuse topology from **Site Tunnels**; package it cleanly.

```
labs/overlay-a/
  site-tun.clab.yml
  addressing.md
  scripts/bringup.sh
  verify.sh
  drills.md
  captures/.gitkeep
```

Success metrics

- h1 h2 continuous ping
- inet tcpdump shows GRE
- Underlay link down stops overlay; restore recovers
- DF large ping behavior documented

`verify.sh`

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-site-tun-h1 ping -c2 -W1 192.168.20.10
docker exec clab-site-tun-h2 ping -c2 -W1 192.168.10.10
docker exec clab-site-tun-sitea ip link show gre1 | grep -q UP
echo OK-A
```


Failure matrix A

#	Action	Expect
A1	inet eth2 down	overlay down
A2	delete gre on sitea	blackhole
A3	remove LAN route	gre peer lives, hosts die
A4	lower mtu wrongly	large packet fail

Track B — VXLAN two-leaf (reference build)

Package **VXLAN data plane** chapter topology as `labs/overlay-b/`.

Success metrics

- Underlay VTEP lo ping
- h1 h2 same subnet across leaves
- Spine capture UDP/4789
- Wrong VNI experiment isolates
- FDB shows remote MAC after traffic

bringup excerpt

```
#!/usr/bin/env bash
set -euo pipefail
# called after containerlab deploy
sudo containerlab deploy -t vxlan-dp.clab.yml
# apply addresses + vxlan/bridge (idempotent scripts per node)
./scripts/addr.sh
./scripts/vxlan.sh
./verify.sh
```

verify.sh B

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo FAIL:$* >&2; exit 1; }
docker exec clab-vxlan-dp-leaf1 ping -c1 -W1 10.0.0.2 || fail underlay
```

```
docker exec clab-vxlan-dp-h1 ping -c2 -W1 10.10.10.20 || fail overlay
echo OK-B
```

Failure matrix B

#	Action	Expect
B1	spine forward off	underlay fail → overlay fail
B2	remove FDB flood entry	BUM/unknown fail
B3	VNI mismatch	isolation
B4	host unbridge eth1	local access fail

Capture helper

```
docker exec clab-vxlan-dp-spine sh -c \  
  'tcpdump -ni eth1 -c 20 udp port 4789 -w /tmp/vx.pcap'  
# docker cp out to captures/vx.pcap
```

Track C — EVPN intro (stretch)

Package EVPN chapter with explicit **feature probe**:

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn summary' \  
  || { echo "EVPN not available-document fallback to Track B"; exit 0; }
```

Success metrics

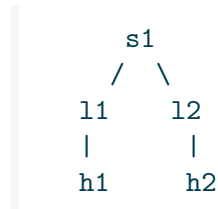
- BGP EVPN Established
- EVPN MAC routes visible
- Host ping works
- Neighbor shutdown kills overlay learning path
- Underlay remains pingable

Failure matrix C

#	Action	Expect
C1	RR down	leaves lose routes (no alternate)
C2	advertise-all-vni removed	control stops updates
C3	underlay lo route removed	BGP over loopback dies
C4	RT mismatch experiment	import empty

Integrated “mini fabric snack” (optional merge)

If ambitious, combine track B underlay style with dual spines from the fabrics part later. **Do not** expand scope until verify is green on two leaves.



Underlay ECMP + VXLAN HER to both directions is enough complexity for one week.

Observability expectations

Tool	Use
ping / traceroute	Inner vs outer endpoints
tcpdump / tshark	Prove encap
bridge fdb / wg show / ip xfrm	State tables
vysh show bgp l2vpn evpn	Control plane
ip route get	Which path outer takes

Hardening lite (overlay)

- ☐ Underlay allows only needed outer ports between VTEPs
- ☐ No accidental route leaking inner prefixes to “inet”
- ☐ Keys for WireGuard/IPsec not in public git
- ☐ VNI inventory file (vnis.md)

Lab report template

```
# Overlay lab report
Track: B
Date: 2026-...
Underlay: static loopbacks via spine
Overlay: VXLAN VNI 100 HER
Evidence:
- captures/vx.pcap
- verify.sh OK
Drills: B1-B4 notes...
What surprised me:
Next: EVPN or dual-spine
```

Predict → observe → fix (capstone habit)

For every drill row, write three lines in `drills.md`. If observe predict, you learned something—that is the point.

Time-box guidance

Block	Time
Deploy + address	30–60 min
Overlay up	1–2 h
Captures + verify	30 min
Failure matrix	1–2 h
Writeup	30 min

If blocked >1 h on EVPN syntax, drop to Track B and file an issue note—do not thrash.

Common mistakes

Mistake	Symptom
Mixing all tracks half-done	Nothing verifies
No cold-start test	“Works on my dirty ns”
Capturing on wrong node	Never see UDP 4789
Skipping underlay prove	Overlay debug theater
Giant jumbo scope	No finish

Summary

- One finished overlay lab beats three partial ones
- Tracks $A \rightarrow B \rightarrow C$ escalate control-plane complexity
- Always prove **encap with packets** and **failures with a matrix**
- Keep VNI/tunnel inventory and MTU notes in git
- Next part scales underlays into **fabrics and multi-area design**

When this lab is green, you are ready for Clos-style underlays and multi-implementation leaves.

Fabrics and Multi-Area

Clos Leaf-Spine

Clos Leaf-Spine

Modern data-center fabrics prefer **scale-out leaf-spine (Clos-inspired)** topologies over deep access-aggregation trees. This chapter builds the underlay model and a free Containerlab fabric snack with FRR + Linux endpoints (mid-2026).

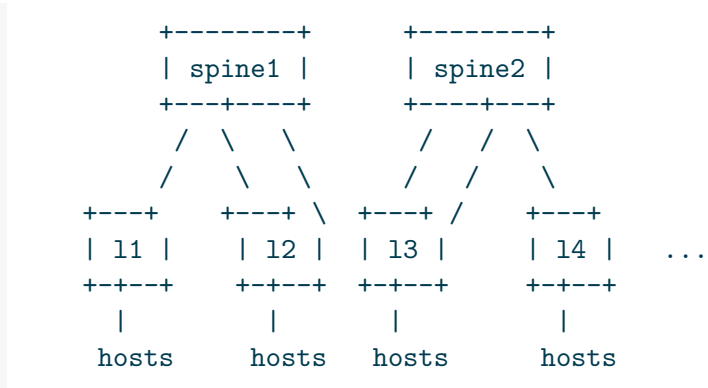
Learning goals

By the end of this chapter you can:

- Explain leaf, spine, and east-west vs north-south roles
- Design a 2-spine × 2-leaf IP underlay with ECMP
- Choose eBGP vs OSPF underlay trade-offs at a model level
- Deploy a Clos-ish lab and verify multipath
- Predict behavior when one spine or one leaf uplink fails

Why Clos-style fabrics

Classic tree pain	Leaf-spine idea
Oversubscription at aggregation	East-west via spines with planned ratio
STP blocking / L2 sprawl	L3 underlay to leaf; overlay for L2 if needed
Hard horizontal scale	Add leaves/spines with more links
Big failure domains	Limit L2; contain blasts



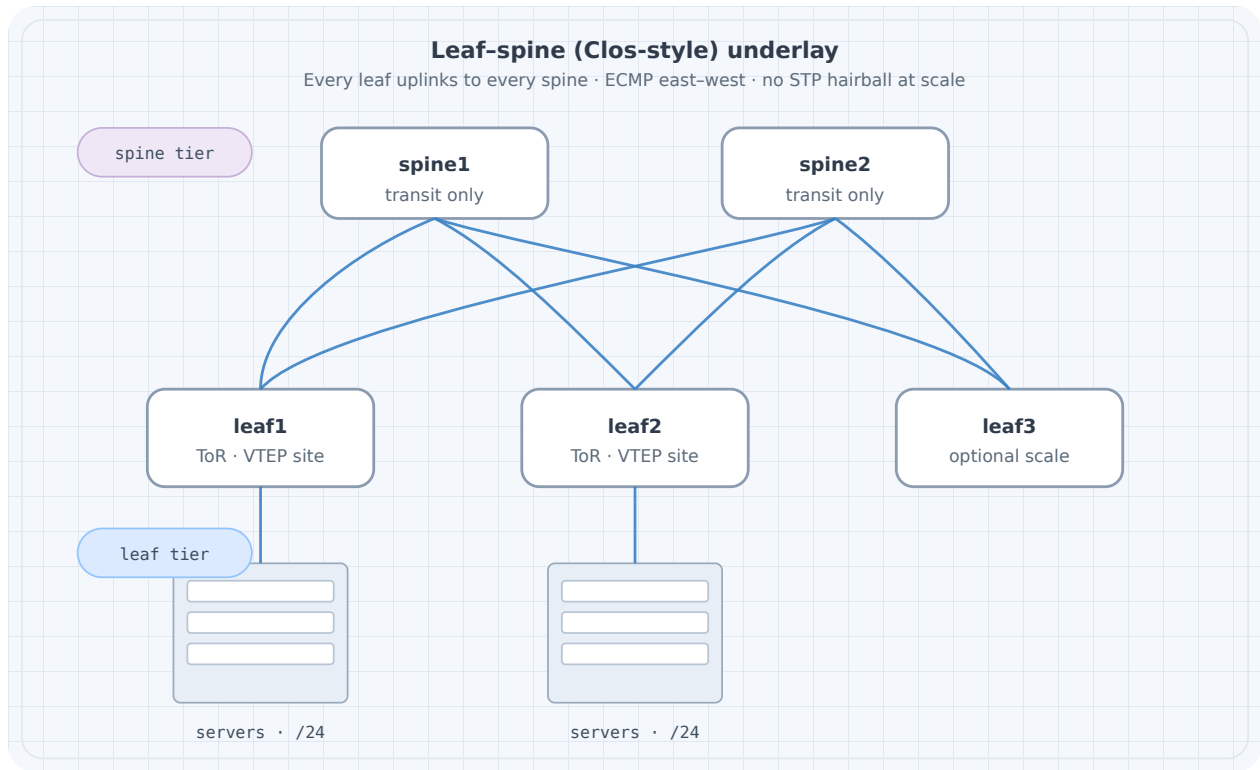


Figure 1: Leaf-spine Clos underlay

Small lab: **2 spines × 2 leaves** (diagram shows a third leaf for scale intuition).

Roles

Role	Function
Leaf	Connects servers; VTEP if overlay; default GW / anycast GW often here
Spine	Transit underlay only (usually no endpoints)
Super-spine / border	Optional higher tier or DC edge (later)

Spines should not become “core with lots of features” on day one—**move packets between leaves**.

Oversubscription (awareness)

```
oversubscription (sum leaf downlink speeds) / (sum leaf uplink speeds)
```

Lab ignores real bandwidth; production designs document the ratio.

Underlay protocol choices

Underlay	Pros	Cons
eBGP (often unnumbered)	Scale, policy, isolation	More session design
OSPF / IS-IS	Simple adjacency model	Large single domain care
Static	Tiny labs	No scale

This book's first fabric: **OSPF or eBGP on FRR**—pick one and stay consistent. eBGP fabric is popular; OSPF is fine for 2×2 learning.

Addressing plan (2×2)

Node	Loopback	Links
s1	10.0.0.1/32	to l1, l2
s2	10.0.0.2/32	to l1, l2
l1	10.0.0.11/32	to s1, s2; host eth
l2	10.0.0.12/32	to s1, s2; host eth
h1	192.168.1.10/24	via l1
h2	192.168.2.10/24	via l2

Point-to-point /31 or /30 on fabric links—example /30:

Link	Subnet
l1–s1	10.1.1.0/30
l1–s2	10.1.2.0/30
l2–s1	10.1.3.0/30
l2–s2	10.1.4.0/30

Host subnets via leaf: advertise leaf loopbacks + host subnets into underlay **or** keep hosts only for L3 via leaf and use overlay later.

For pure underlay ECMP lab: ping between leaf loopbacks and between hosts with leaf default routing + OSPF advertised LANs.

Topology YAML

```
name: clos22

topology:
  nodes:
```

```

s1:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/s1:/etc/frr
s2:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/s2:/etc/frr
l1:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/l1:/etc/frr
l2:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/l2:/etc/frr
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.1.10/24 dev eth1
    - ip link set eth1 up
    - ip route add default via 192.168.1.1
h2:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.2.10/24 dev eth1
    - ip link set eth1 up
    - ip route add default via 192.168.2.1

links:
  - endpoints: ["l1:eth1", "s1:eth1"]
  - endpoints: ["l1:eth2", "s2:eth1"]
  - endpoints: ["l2:eth1", "s1:eth2"]
  - endpoints: ["l2:eth2", "s2:eth2"]
  - endpoints: ["h1:eth1", "l1:eth3"]
  - endpoints: ["h2:eth1", "l2:eth3"]

```

FRR daemons

```
zebra=yes
ospfd=yes
# or bgpd=yes for eBGP underlay
```

OSPF underlay sketch (I1)

```
frr version 10.2.1
hostname l1
!
interface lo
 ip address 10.0.0.11/32
 ip ospf area 0
!
interface eth1
 ip address 10.1.1.1/30
 ip ospf network point-to-point
 ip ospf area 0
!
interface eth2
 ip address 10.1.2.1/30
 ip ospf network point-to-point
 ip ospf area 0
!
interface eth3
 ip address 192.168.1.1/24
 ip ospf area 0
!
router ospf
 ospf router-id 10.0.0.11
 passive-interface eth3
!
line vty
```

Spines: loopbacks + fabric links in area 0; **no host interfaces**. 12 mirrors with its addresses. Point-to-point OSPF avoids DR drama on p2p links.

eBGP underlay sketch (optional alternate)

```
router bgp 65101
  bgp router-id 10.0.0.11
  neighbor 10.1.1.2 remote-as 65100
  neighbor 10.1.2.2 remote-as 65100
  address-family ipv4 unicast
    network 10.0.0.11/32
    network 192.168.1.0/24
    neighbor 10.1.1.2 activate
    neighbor 10.1.2.2 activate
    maximum-paths 4
  exit-address-family
```

Spines use ASN 65100 (or each spine unique ASN—design choice). Leaves unique ASNs. Enable multipath.

Deploy and ECMP verify

```
sudo containerlab deploy -t clos22.clab.yml

docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'
docker exec clab-clos22-l1 vtysh -c 'show ip route 10.0.0.12'
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-clos22-h1 ping -c 3 192.168.2.10
docker exec clab-clos22-h1 traceroute -n 192.168.2.10
```

Predict

- Each leaf: 2 OSPF neighbors (both spines)
- Route to remote leaf/host shows **multiple nexthops** (ECMP) if multipath installed
- traceroute may show s1 or s2

Observe multipath

```
docker exec clab-clos22-l1 ip route show 192.168.2.0/24
# or
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0/24'
```

FRR/zebra may need `maximum-paths` under OSPF:

```
router ospf
maximum-paths 4
```

Drill 1 — Spine loss

```
docker exec clab-clos22-s1 ip link set eth1 down
docker exec clab-clos22-s1 ip link set eth2 down
sleep 3
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-clos22-h1 ping -c 20 192.168.2.10
```

Predict → observe → fix

Predict | ECMP shrinks to remaining spine; brief loss |

Observe | single nexthop via s2 |

Fix | if blackhole, check l2 uplink to s2 and SPF |

Restore interfaces after.

Drill 2 — One leaf uplink down

```
docker exec clab-clos22-l1 ip link set eth1 down
docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'
docker exec clab-clos22-h1 traceroute -n 192.168.2.10
```

Predict

Still reachable via other spine; neighbor count 1.

Drill 3 — Polarization awareness

Send many parallel iperf flows; without entropy, paths may skew. Underlay traceroute from different sources may stick to one spine. Note in journal; overlays later add UDP entropy.

Pattern	Path
---------	------

East-west vs north-south

Pattern	Path
East-west h1 h2	leaf → spine → leaf
North-south to Internet	leaf → border/spine path to edge

Do not attach WAN carelessly to all spines without a border design.

Overlay placement (preview)

VTEPs usually on **leaves**. Spines stay underlay-only. That separation keeps failure domains clean—connects to VXLAN/EVPN chapters.

Verification checklist

Check	Intent
All fabric adjacencies	Full mesh leaf-spine links up
Loopback mesh	Any lo to any lo
ECMP present	2 nexthops where topology allows
Host traffic	Data plane
Spine failure	Continuity

Common mistakes

Mistake	Symptom
Hosts on spines	Weird roles, scale pain
L2 across spines	Back to tree problems
Different OSPF areas accidentally	Missing routes
No p2p network type	Extra DR complexity
Expecting overlay without VTEP	Hosts same L2 fantasy

Summary

- Leaf-spine Clos-style fabrics scale east-west with L3 underlays
- Spines transit; leaves attach endpoints (and VTEPs)
- 2×2 FRR lab is enough to feel ECMP and spine loss
- Pick OSPF or eBGP underlay deliberately
- Keep overlays on leaves in later designs

Next: **multi-area OSPF**—when a single area is no longer the right underlay shape (campus/WAN more than fabric).

Multi-Area OSPF

Multi-Area OSPF

Single-area OSPF is the right first lab. Multi-area OSPF exists to **bound flooding, summarize, and structure large domains**. This chapter extends FRR OSPF into a small multi-area topology in Containerlab (mid-2026 free path)—useful for campus/WAN; fabrics often prefer BGP underlays instead.

Learning goals

By the end of this chapter you can:

- Explain area, ABR, ASBR, and LSA-type roles at operator level
- Build a three-router multi-area lab (backbone 0 + spoke area)
- Summarize at an ABR and predict reachability holes
- Contrast stub-ish ideas without drowning in edge-case flags
- Verify with database, route, and failure drills

Concepts

Why areas

Single area	Multi-area
Every router full topology	Topology flooding scoped
SPF on full graph	Smaller SPF domains
Simple	Needs backbone rules

Backbone rule (practical)

- Area **0** is the backbone
- Non-backbone areas attach via **ABRs** that touch area 0
- Virtual links exist for scars—not for greenfield labs

Roles

Role	Meaning
ABR	Router with interfaces in multiple areas; can summarize
ASBR	Injects external routes (redistribution)
Internal	All interfaces one area

LSA awareness (not flash cards)

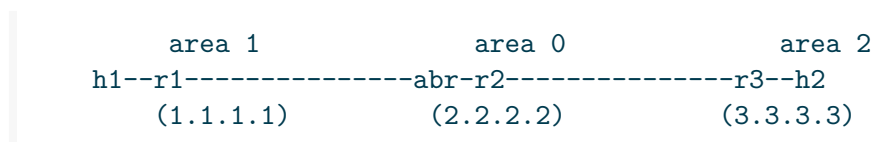
Type idea	Carries
Router / network	Topology inside area
Summary (ABR)	Inter-area prefixes
External	From outside OSPF

You operate with: `show ip ospf database`, `show ip ospf border-routers`, `show ip route`.

Summarization

ABR can advertise `10.1.0.0/16` instead of many `/24`s. Mis-summary → **blackholes** if holes exist inside the aggregate.

Topology



Node	Areas	Notes
r1	1 only	internal
r2	0 + 1 + 2	ABR
r3	2 only	internal

Links:

Link	Subnet	Area
r1-r2	10.0.12.0/24	1
r2-r3	10.0.23.0/24	2
r1 LAN	192.168.1.0/24	1
r3 LAN	192.168.3.0/24	2

Link	Subnet	Area
r2 lo	2.2.2.2/32	0

Optional second area-0 link later; one ABR is enough to learn.

Topology YAML

```
name: ospf-ma

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.3.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.3.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
```

- endpoints: ["h1:eth1", "r1:eth2"]
- endpoints: ["h2:eth1", "r3:eth2"]

FRR configs

r1 (area 1)

```
hostname r1
!
interface lo
 ip address 1.1.1.1/32
 ip ospf area 1
!
interface eth1
 ip address 10.0.12.1/24
 ip ospf area 1
!
interface eth2
 ip address 192.168.1.1/24
 ip ospf area 1
!
router ospf
 ospf router-id 1.1.1.1
 passive-interface eth2
!
line vty
```

r3 (area 2)

Mirror with 3.3.3.3, 10.0.23.2/24 area 2, LAN 192.168.3.1/24 area 2.

r2 (ABR)

```
hostname r2
!
interface lo
 ip address 2.2.2.2/32
 ip ospf area 0
!
interface eth1
 ip address 10.0.12.2/24
 ip ospf area 1
```

```

!
interface eth2
 ip address 10.0.23.1/24
 ip ospf area 2
!
router ospf
 ospf router-id 2.2.2.2
 area 1 range 192.168.0.0/16
 area 2 range 192.168.0.0/16
!
line vty

```

Careful: broad 192.168.0.0/16 summary is a teaching weapon—it can hide holes. Prefer tight ranges:

```

area 1 range 192.168.1.0/24
area 2 range 192.168.3.0/24

```

Or summarize only when you own contiguous blocks.

Deploy and baseline

```

sudo containerlab deploy -t ospf-ma.clab.yml

docker exec clab-ospf-ma-r2 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-ma-r2 vtysh -c 'show ip ospf border-routers'
docker exec clab-ospf-ma-r1 vtysh -c 'show ip ospf database'
docker exec clab-ospf-ma-r1 vtysh -c 'show ip route'
docker exec clab-ospf-ma-h1 ping -c 3 192.168.3.10

```

Predict

- r2 has neighbors in area 1 and 2
- r1 sees inter-area path to 192.168.3.0/24 via r2
- Database on r1 does **not** contain full area 2 topology detail (summary only)

Observe

Compare `show ip ospf database` on r1 vs r2. r2 should show more topology.

Drill 1 — Summarization blackhole

On r2:

```
area 1 range 192.168.0.0/16
```

Do **not** put a real 192.168.99.0/24 anywhere, but inject a static on r1 into OSPF... or simpler: summarize too wide then withdraw a component.

Teaching sequence:

1. Advertise 192.168.1.0/24 only.
2. Add `area 1 range 192.168.0.0/16`.
3. On r3, install a static to 192.168.99.1 via null or missing next hop and redistribute—or ping 192.168.99.1 from r3 side expecting blackhole from r1 if summary exists without component.

Simpler blackhole lab:

```
# r2 summarizes 192.168.0.0/16 from area 1 while only .1.0/24 exists
# from r3 ping 192.168.1.10 works; ping 192.168.50.10 fails (good)
# If r3 thinks 192.168.50.0 is under summary toward area1, traffic blackholes at ABR
```

Predict

Summaries without covering specifics attract traffic to ABR then drop.

Fix

Tight ranges; discard routes; or don't summarize.

Drill 2 — ABR failure

```
docker stop clab-ospf-ma-r2
docker exec clab-ospf-ma-h1 ping -c 5 192.168.3.10
```

Predict

Total partition—single ABR is SPOF.

Design lesson

Redundant ABRs for real campus; dual attachment to area 0.

Drill 3 — Area mismatch

Put r1 eth1 in area 0 by mistake:

```
interface eth1
 ip ospf area 0
```

Predict

Adjacency fails or forms wrong; routes missing.

Observe

show ip ospf neighbor empty/wrong.

Fix

Restore area 1 both sides.

Stub area ideas (awareness)

Stub/totally stubby/NSSA reduce external LSAs into an area. FRR supports variants—use when external flood is noisy. For this chapter, know **why** (less type-5 in area) more than every flag combo.

```
router ospf
 area 1 stub
```

Retest default injection behavior when you try it.

Multi-area vs fabric BGP

Multi-area OSPF	Clos eBGP underlay
Campus/WAN hierarchy	DC scale-out
ABR summarization	Prefix + policy at leaves
Flooding domains	AS boundary policy

Do not force multi-area OSPF onto a leaf-spine if eBGP underlay is the house style.

Check	Command
-------	---------

Verification checklist

Check	Command
Neighbors per area	<code>show ip ospf neighbor</code>
ABR role	<code>show ip ospf border-routers</code>
DB scope	<code>show ip ospf database</code>
Inter-area routes	<code>show ip route IA flags</code>
Data plane	<code>ping h1 h2</code>

Common mistakes

Mistake	Symptom
Discontiguous area 0	Weird inter-area failures
Summary too broad	Blackholes
Expecting full topology remote area	Confusion reading DB
Redistribute without filters	Loops/externals everywhere
Single ABR production	Outage on maintenance

Summary

- Multi-area OSPF bounds flooding and enables ABR summarization
- Area 0 backbone rules still matter in greenfield design
- Always verify **database scope** and **inter-area routes**, not only neighbors
- Summaries can blackhole—tight ranges and drills
- Prefer BGP fabrics for Clos; multi-area OSPF for hierarchical campus/WAN

Next: **fabric with FRR**—package a tighter leaf-spine implementation path and verification kit.

Fabric with FRR

Fabric with FRR

This chapter turns the Clos model into an **FRR-operable fabric kit**: config layout, unnumbered eBGP option, verification scripts, and failure drills. Open stack only—Containerlab + FRR + Alpine (mid-2026). Overlay remains optional on leaves.

Learning goals

By the end of this chapter you can:

- Organize a fabric lab repo for N leaves $\times$ M spines
- Implement OSPF or eBGP underlay consistently on FRR
- Use `maximum-paths` and read `multipath` in RIB/FIB
- Smoke-test a fabric with a single `verify.sh`
- Know where to bolt VXLAN/EVPN on leaves later

Lab repo layout

```
labs/fabric-frr/  
  clos22.clab.yml  
  addressing.md  
  config/  
    s1/frr.conf daemons  
    s2/...  
    l1/...  
    l2/...  
  scripts/  
    render.sh      # optional template expander  
    verify.sh  
    fail-spine.sh  
  captures/  
  README.md
```

Templating (jinja/make/python) pays off at 4×4 ; 2×2 can be hand-written.

Design defaults for this book

Choice	Default
Size	2 spine × 2 leaf
Underlay	OSPF area 0 p2p or eBGP
Loopbacks	10.0.0.x/32
Hosts	one per leaf, /24
Overlay	off (underlay first)
Images	quay.io/frrouting/frr:10.2.1 + alpine:3.20

eBGP unnumbered (modern fabric style)

When link-local + IPv6 RA or RFC5549-style patterns available, unnumbered BGP reduces IP inventory. FRR support has matured—verify on your image:

```
interface eth1
  ip address 10.1.1.1/30
!
router bgp 65101
  neighbor fabric peer-group
  neighbor fabric remote-as external
  neighbor fabric capability extended-nextthop
  neighbor eth1 interface peer-group fabric
!
address-family ipv4 unicast
  neighbor fabric activate
  network 10.0.0.11/32
exit-address-family
```

If interface peering is painful in your build, use explicit /30 neighbors (Clos chapter)—**same topology math**.

Numbered eBGP fabric (reliable teaching)

Node	ASN
s1	65100
s2	65100
l1	65101
l2	65102

Spines same ASN (or not—both designs exist). Leaves unique.

Spine s1 excerpt

```
hostname s1
!
interface lo
 ip address 10.0.0.1/32
!
interface eth1
 ip address 10.1.1.2/30
!
interface eth2
 ip address 10.1.3.2/30
!
router bgp 65100
 bgp router-id 10.0.0.1
 neighbor 10.1.1.1 remote-as 65101
 neighbor 10.1.3.1 remote-as 65102
!
 address-family ipv4 unicast
  network 10.0.0.1/32
  neighbor 10.1.1.1 activate
  neighbor 10.1.3.1 activate
  maximum-paths 4
 exit-address-family
!
line vty
```

Leaf l1 excerpt

```
hostname l1
!
interface lo
 ip address 10.0.0.11/32
!
interface eth1
 ip address 10.1.1.1/30
!
interface eth2
 ip address 10.1.2.1/30
!
interface eth3
 ip address 192.168.1.1/24
!
router bgp 65101
 bgp router-id 10.0.0.11
```

```

neighbor 10.1.1.2 remote-as 65100
neighbor 10.1.2.2 remote-as 65100
!
address-family ipv4 unicast
  network 10.0.0.11/32
  network 192.168.1.0/24
  neighbor 10.1.1.2 activate
  neighbor 10.1.2.2 activate
  maximum-paths 4
exit-address-family
!
line vty

```

daemons: zebra=yes, bgpd=yes.

OSPF alternate (same wiring)

Reuse Clos chapter OSPF configs; add:

```

router ospf
  maximum-paths 4

```

Containerlab topology

Same as Clos chapter clos22—keep one file, swap configs via binds.

Operational show kit

```

# sessions / neighbors
docker exec clab-clos22-l1 vtysh -c 'show bgp summary'
docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'

# multipath
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0/24'
docker exec clab-clos22-l1 ip route show 192.168.2.0/24

# loopback mesh
for d in 10.0.0.1 10.0.0.2 10.0.0.11 10.0.0.12; do
  docker exec clab-clos22-l1 ping -c1 -W1 $d || echo fail $d
done

# hosts

```

```
docker exec clab-clos22-h1 ping -c2 192.168.2.10
```

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }
P=clab-clos22

docker exec $P-l1 ping -c1 -W1 10.0.0.12 || fail "l1->l2 lo"
docker exec $P-l1 ping -c1 -W1 10.0.0.1 || fail "l1->s1 lo"
docker exec $P-h1 ping -c2 -W1 192.168.2.10 || fail "h1->h2"
docker exec $P-h2 ping -c2 -W1 192.168.1.10 || fail "h2->h1"

# multipath soft check: at least one route line with multiple nexthops or two entries
R=$(docker exec $P-l1 vtysh -c 'show ip route 192.168.2.0/24' || true)
echo "$R" | grep -Eq '192.168.2.0|via' || fail "no route to h2 subnet"

echo OK-FABRIC
```

Failure scripts

```
#!/usr/bin/env bash
# fail-spine.sh s1
set -euo pipefail
SP=${1:-s1}
docker exec clab-clos22-$SP ip link set eth1 down
docker exec clab-clos22-$SP ip link set eth2 down
echo "$SP uplinks down; run ping/traceroute; then restore"
```

```
#!/usr/bin/env bash
# restore-spine.sh s1
SP=${1:-s1}
docker exec clab-clos22-$SP ip link set eth1 up
docker exec clab-clos22-$SP ip link set eth2 up
```

Predict → observe → fix

Drill	Predict	Observe
Spine down	Paths via other spine	single nexthop

Drill	Predict	Observe
Leaf uplink down	Still works	1 BGP/OSPF neighbor
Wrong ASN	Session down	show bgp summary Active/Idle
Missing network stmt	Host subnet absent	RIB missing

Policy on fabric (light)

Even underlays need hygiene:

```
# do not accept default from random peers in fabric
# do not advertise lab management prefixes
ip prefix-list FABRIC-L0 seq 5 permit 10.0.0.0/24 le 32
route-map EXPORT-FABRIC permit 10
  match ip address prefix-list FABRIC-L0
```

Apply out toward spines if you practice policy discipline.

Adding overlay later (hook points)

On each leaf:

1. Keep underlay BGP/OSPF as-is
2. Add VTEP on loopback
3. EVPN AF on BGP or static VXLAN to other leaf loopbacks
4. Spines remain ignorant of VNIs

```
leaf = underlay router + VTEP + (optional) IRB
spine = underlay router only
```

Scale checklist 2x2 → 4x4

Item	Action
IP plan	Spreadsheet / addressing.md generator
Configs	Templates
RAM	FRR light; watch host memory
verify	Loop all leaf pairs
Cabling matrix	Automate clab links generation

Observability

- Enable FRR logging to bound files
- Graph: `containerlab graph -t clos22.clab.yml`
- Optional: scrape with your ops chapter tools

Optional SONiC / SR Linux leaf

Replace one leaf image with community NOS; keep underlay protocol same. Document CLI mapping for `show bgp` / `show ip route`. Goal: **multi-implementation confidence** (next chapter deepens this).

Common mistakes

Mistake	Symptom
Spines advertising host defaults weirdly	Traffic trombones
maximum-paths left at 1	No ECMP
Overlay on spine	Operational regret
No cold deploy test	Config drift in running only
Same router-id	BGP/OSPF chaos

Summary

- FRR is enough to run a real teaching fabric underlay
- Standardize repo layout, ASNs, and verify scripts
- Prove ECMP and spine loss before overlays
- Leaves are the VTEP attachment point later
- Policy-light underlays still need prefix discipline

Next: **multi-implementation lab**—FRR + Linux + optional SR Linux in one topology.

Multi-Implementation Lab

Multi-Implementation Lab

Vendor-agnostic skill means the **same design** survives different CLIs. This capstone-style lab mixes **FRR, Linux endpoints, and optional Nokia SR Linux (community)** in one Containerlab topology (mid-2026 free/community images only).

Learning goals

By the end of this chapter you can:

- Run a leaf-spine underlay with heterogeneous leaf implementations
- Map show-command dialects to the same verification questions
- Keep addressing, ASN, and success metrics identical across nodes
- Document differences without abandoning models
- Produce a multi-NOS lab report

Design principle

Same:

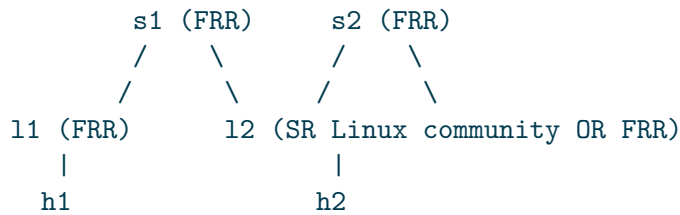
- topology roles (spine/leaf/host)
- ASNs / areas / loopbacks
- prefixes advertised
- failure drills

Different:

- config language
- show syntax
- feature knobs names

If behavior diverges, **stop and explain** (bug, knobs, or design mismatch)—do not “fix it” with silent statics on one leaf only.

Recommended topology



If SR Linux image pull is unavailable, run **l2 as FRR** and still complete the dual-leaf discipline—or use VyOS/SONiC community kinds if present in your Containerlab kind list. The **process** matters.

Addressing / ASN (fixed)

Node	Kind	lo	ASN
s1	FRR	10.0.0.1/32	65100
s2	FRR	10.0.0.2/32	65100
l1	FRR	10.0.0.11/32	65101
l2	SR Linux or FRR	10.0.0.12/32	65102
h1	Alpine	192.168.1.10/24	—
h2	Alpine	192.168.2.10/24	—

Fabric links: same /30 map as Clos chapter.

Topology YAML (FRR + optional srl)

```
name: multi-imp

topology:
  nodes:
    s1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/s1:/etc/frr
    s2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/s2:/etc/frr
    l1:
```

```

kind: linux
image: quay.io/frrouting/frr:10.2.1
binds:
  - ./config/l1:/etc/frr
l2:
  # Option A - all FRR:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/l2:/etc/frr
  # Option B - SR Linux community (uncomment/adjust per containerlab docs):
  # kind: nokia_srlinux
  # image: ghcr.io/nokia/srlinux:latest    # pin a known free tag in real labs
  # binds: ...
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
    - ip route add default via 192.168.1.1
h2:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils
    - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
    - ip route add default via 192.168.2.1

links:
  - endpoints: ["l1:eth1", "s1:eth1"]
  - endpoints: ["l1:eth2", "s2:eth1"]
  - endpoints: ["l2:eth1", "s1:eth2"]
  - endpoints: ["l2:eth2", "s2:eth2"]
  - endpoints: ["h1:eth1", "l1:eth3"]
  - endpoints: ["h2:eth1", "l2:eth3"]

```

Pin image tags in real work; `latest` is a lab convenience only. Consult [Containerlab kinds](#) for current SR Linux free image references.

FRR side

Reuse fabric-with-FRR spine/leaf configs. Spines must peer to both leaves regardless of leaf NOS.

SR Linux mapping (conceptual)

Intent	FRR-ish	SR Linux-ish
Router id / lo	<code>interface lo + BGP router-id</code>	<code>system / network-instance loopback</code>
Interface IP	<code>ip address</code>	<code>interface subinterface IPv4</code>
eBGP neighbor	<code>neighbor A.B.C.D remote-as</code>	<code>bgp neighbor under network-instance</code>
Advertise lo + LAN	<code>network statements</code>	<code>export policy + prefixes</code>
Multipath	<code>maximum-paths</code>	<code>ECMP / multipath knobs</code>
Verify BGP	<code>show bgp summary</code>	<code>show network-instance default protocols bgp ... (version-specific)</code>
Verify route	<code>show ip route</code>	<code>show route / fib show variants</code>

Exact paths change by SR Linux release—**write the mapping table in your lab README** with the commands that worked for you.

Verification questions (dialect-free)

Answer these on **every** leaf and spine:

1. Are fabric links up at L1/L2?
2. Is the underlay protocol adjacent to both spines?
3. Is my loopback in the fabric RIB on the far leaf?
4. Is the remote host subnet reachable with ECMP?
5. Do hosts pass bidirectional ICMP?
6. After killing s1, does traffic continue?

FRR answers

```
docker exec clab-multi-imp-l1 vtysh -c 'show bgp summary'
docker exec clab-multi-imp-l1 vtysh -c 'show ip route 192.168.2.0/24'
```

SR Linux answers (illustrative)

```
# container name pattern depends on clab naming
docker exec clab-multi-imp-l2 sr_cli 'show interface'
docker exec clab-multi-imp-l2 sr_cli 'show network-instance default route-table'
# adjust to your image's CLI entrypoint (sr_cli / srlinux)
```

predict → observe → fix matrix

Drill	Predict	FRR observe	SRL observe
Deploy cold	all sessions up	bgp summary	bgp show
h1→h2 ping	ok	ping	ping from h2 side
s1 down	ok via s2	single path	single path
shutdown l2	h2 isolated	route gone on l1	route gone
LAN network			
MTU 1400	large ping fail	ping -M do	same
fabric			

verify.sh (heterogeneous)

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $" >&2; exit 1; }
P=clab-multi-imp

docker exec $P-h1 ping -c2 -W1 192.168.2.10 || fail "h1->h2"
docker exec $P-h2 ping -c2 -W1 192.168.1.10 || fail "h2->h1"
docker exec $P-l1 ping -c1 -W1 10.0.0.12 || fail "l1->l2 lo"

# FRR leaf sessions
docker exec $P-l1 vtysh -c 'show bgp summary' | grep -q '65100' \
|| fail "l1 missing spine asn in summary"

echo OK-MULTI
```

Extend with SRL-specific checks when l2 is SR Linux.

Overlay optional add-on

Once underlay multipath works on both implementations:

- Static VXLAN between l1 and l2 loopbacks (Linux vxlan on FRR node may need extra tooling; pure FRR leaf might use other integration)
- Or EVPN only if **both** leaves support it—heterogeneous EVPN is an advanced interop project, not a first lab

Declare non-goals if overlay skipped.

Hardening gates (reuse part 08)

- ☐ Only fabric prefixes advertised
- ☐ Mgmt not exposed on fabric links
- ☐ Image tags pinned
- ☐ destroy/recreate clean

Lab report template

```
# Multi-implementation fabric
Date:
Images: FRR x.y, SR Linux z (or FRR-only fallback)
Underlay: eBGP numbered
Results:
- cold verify.sh: PASS/FAIL
- spine failure: notes
- CLI mapping table: attached
Differences found:
Next improvements:
```

When interop hurts

Symptom	Approach
Session up, no routes	policy / AF / network statements
ECMP one side only	multipath knobs asymmetric
Hosts one-way	RPF, missing return prefix
SRL slow boot	wait; recheck before debug spiral

Time-box interop: 2 hours then fall back to dual FRR with a written “SRL blocked on X” note.

Common mistakes

Mistake	Symptom
Different ASNs than plan	sessions down
Comparing wrong show outputs	false failure
Fixing only FRR side	asymmetric design
Unpinned SRL image	lab broke next month
Skipping host bidirectional test	false green

Summary

- Multi-implementation labs prove **models over CLI**
- Freeze addressing and success metrics; vary only NOS
- Maintain a personal show-command dictionary
- Prefer underlay interop first; EVPN interop later
- FRR-only fallback still completes the fabric curriculum

You now have published depth from edge services through fabrics. Return to ops automation capstones and harden the labs you actually keep.